

AccessMatrix Common Login Page Deployment Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - AccessMatrix CLP Deployment Guide.....	4
2. Introduction to AccessMatrix CLP Deployment.....	5
3. AccessMatrix CLP Deployment.....	11
3.1. Deployment Methods.....	12
3.2. Deployment Configurations.....	15
4. AccessMatrix CLP Application Integrations.....	26
4.1. CLP for AccessMatrix Applications.....	27
4.2. CLP for SAML Identity Provider.....	34
4.3. CLP as OIDC OpenID Provider.....	41
4.4. CLP for WSA Login.....	45
5. AccessMatrix CLP UI Customizations.....	47
6. AccessMatrix CLP Authentication Methods.....	53
6.1. 1FA (Password Authentication).....	55
6.2. 2FA (Email/SMS OTP).....	61
6.3. Integrated Windows Authentication (WIA).....	64
6.4. 2FA (Token OTP).....	83
6.5. OneSpan Scan and Login.....	85
6.6. OneSpan Push to Login.....	88
6.7. YESsafe Scan and Login.....	92
6.8. SingPass Mobile.....	97
6.9. FIDO2 Authentication.....	102
6.10. External OP Authentication via OIDC.....	109
6.11. End-to-End Encryption Authentication (E2EEA).....	113
6.12. Certificate-based Authentication.....	119
7. AccessMatrix CLP Re-Authentication Method.....	126
7.1. Reauthentication Overview.....	127
7.2. System Password Basic.....	130
7.3. VascoTokenLogin.....	132

7.4. Reauthentication Login Modules – Summary Table.....	134
7.5. SMSOTPLLogin.....	136
7.6. OIDCRelyingPartyLoginModule.....	138
7.7. FIDO2TokenLogin.....	142
7.8. RadiusLogin.....	144
7.9. PseudoE2EEALoginModule.....	146
8. AccessMatrix CLP Other Authentication Configurations.....	148

1. Preface - AccessMatrix CLP Deployment Guide

This document guides you on the deployment and configuration of AccessMatrix Common Login Page (CLP) as a web-based client authentication component for different applications that require a login page.

Intended Reader

It is intended as a reference for the security administrators and i-Sprint's Professional Service personnel to carry out the following operations:

- Integrate CLP with AccessMatrix server for authentication purpose
- Customize the User Interface (UI) of CLP and its configuration properties
- Deploy CLP to different web application servers
- Integrate CLP with the supported AccessMatrix web applications

Document Scope

This document only provides the necessary steps to configure and set up Common Login Page and will not provide documentation on operating the AccessMatrix system and its supported web applications.

For feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- [AccessMatrix Common Administration Guide](#)
- [AccessMatrix Common Deployment Guide](#)

2. Introduction to AccessMatrix CLP Deployment

This document provides the instructions on the deployment, integration and maintenance of the AccessMatrix Common Login Page (CLP) for AccessMatrix Server.

About AccessMatrix CLP

Common Login Page (CLP) provides a web-based authentication front end that is ready to be deployed to serve a variety of authentication methods backed by the AccessMatrix Server: for example, password, out-of-band OTP (SMS or Email), or hardware/software token-based OTP authentication methods. For enterprise deployment, CLP also supports integrated windows authentication to provide seamless authentication for Windows-based environments.

For easier integration with other applications, CLP (backed by AccessMatrix UAM) supports federated single sign-on as a SAML Identity Provider and an OpenID Connect OpenID Provider. This means CLP can be easily used as the Identity Provider for cloud-based applications, such as Google G Suite, Microsoft Office 365, Facebook Workday, Salesforce, Zendesk. Other applications that support SAML and OpenID Connect can also integrate with CLP based on these standards.

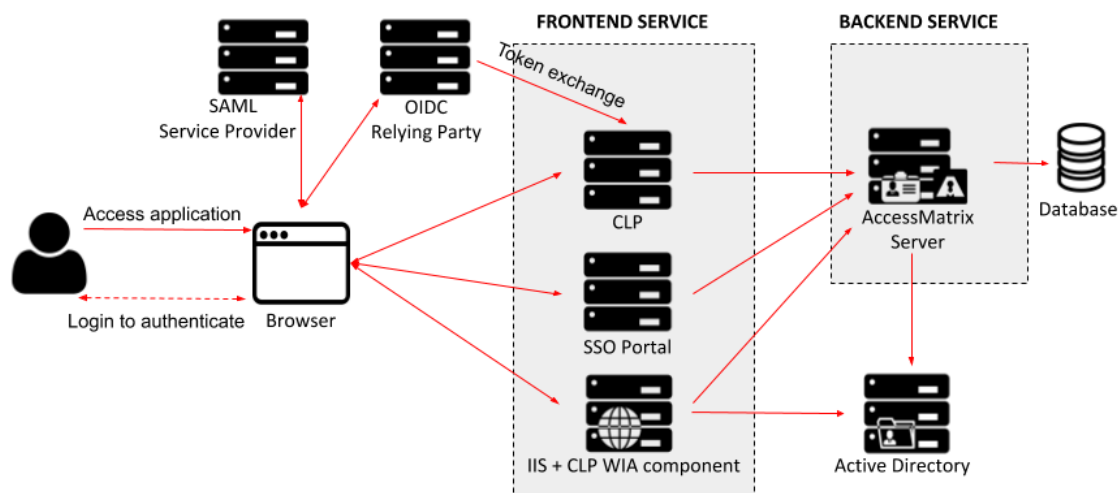
Web-based applications that are protected by AccessMatrix Web Service Agent (WSA) can optionally use CLP to handle user authentication to the protected web applications. Naturally, other AccessMatrix applications, such as USO or SSO Portal and UAS Token Activation Portal, are also integrated with CLP.



Note: CLP is compatible with the Google Chrome and Microsoft Edge browsers.

Architecture

A high-level overview of CLP and its implementations for Federated SSO, USO and Windows-based authentication are illustrated in the architecture diagram below.



CLP is a user-facing thin client application and deployed as a front-end server. It relies on AccessMatrix Server's back-end services for its authentication and single sign-on processing.

CLP does not store any state about user session or state in CLP server; thus, it is easily scaled horizontally when required.

On the client side, the communication between end user, applications (which include SSO Portal, SAML Service Provider and OIDC Relying Party application) and CLP are done via web browser.

The main functions of CLP are to authenticate users and to allow them single sign-on to their web applications, which include SSO Portal, SAML Service Provider and OIDC Relying Party application. These are normally done via web browser. There is no direct communication required between the other web applications to CLP except for web applications that use OpenID Connect and need to call the respective endpoints in CLP.

For enterprise deployment that requires the use of Windows Integrated Authentication (WIA), WIA can be optionally configured in CLP as the authentication option of a client application. That way, CLP will automatically recognize users as they log in with their Windows domain user credentials. CLP WIA component can be deployed on Microsoft's Internet Information Services (IIS) to achieve this. In the event of user authentication using WIA, CLP WIA component in the IIS server will call AccessMatrix Server to trust the identity of the authenticated AD user, and subsequently facilitate user authentication and web SSO with CLP.

Application Integrations

The CLP's primary objective is to provide a front end for user authentication for other applications. To achieve this objective, CLP also needs to be easily integrated with other applications.

- **Integrating AccessMatrix Applications (USO/SSO Portal, USO Self-Service Facility, UAS Token Activation Portal)**

AccessMatrix Applications and CLP support CLP's own protocol to handle login requests and responses backed by AccessMatrix Server. The integration can be easily configured as the applications are predefined. Refer to the [CLP for AccessMatrix Applications](#) section for more details.

- **Integrating SAML-based Applications (Service Provider)**

CLP supports SAML SSO and acts as Identity Provider. This integration uses AccessMatrix UAM to process the SAML Request and to generate Response securely. CLP has a built-in SAML SSO URL that accepts SAML Request (HTTP-Redirect or HTTP-POST binding). CLP will challenge the user with authentication if the user does not have a valid session. With a valid session, CLP generates its corresponding SAML Response and sends the response back to the ACS URL of the application (Service Provider). Refer to the [CLP for SAML Identity Provider](#) section for more details.

- **Integrating OIDC-based Applications (Relying Party)**

CLP supports OIDC Authentication and acts as OpenId Provider. This integration uses AccessMatrix UAM to process the OIDC Authentication/Authorization Request and to produce OIDC tokens (authorization code, access token, ID token). CLP bundles UAM OAuthProxy. This means CLP has OIDC endpoints, such as authorization, token, introspection. When receiving an OIDC authentication request, CLP will challenge the user with authentication if the user does not have a valid session. With a valid session, CLP generates its corresponding response (depending on the requested grant type) and sends it back to the redirect URI of the application (Relying Party). Refer to the [CLP as OIDC OpenID Provider](#) section for more details.

- **Integrating UAM WSA Reverse Proxy**

WSA Reverse Proxy provides web access control to protected resources. CLP can be deployed to handle user authentication. This means when a user, who is not yet authenticated, is trying to access a WSA-protected resource, WSA will prompt the user for authentication by redirecting the user to CLP. Once the user has been authenticated, then CLP will redirect back to the WSA-protected resource. Refer to the [CLP for WSA Login](#) section for more details.

Supported Authentication Methods

AccessMatrix system supports a wide variety of authentication methods to address the business requirements for authentication mechanisms. In AccessMatrix, authentication realm is used to associate users' credentials for authentication based on a lookup module and the login module(s) where user entries can be created into a local repository of AccessMatrix or integrated from Lightweight Directory Access Protocol (LDAP)-compliant directory. Authentication realm(s) would then be assigned to users for authentication purpose. For more details, refer to [AccessMatrix Common Administration Guide - Manage User Authentication Service](#) section.

AccessMatrix CLP supports the following authentication methods:

- **One-factor Authentication (1FA) - Password Authentication**

This method provides a password-based authentication to log in to CLP. This can be using a basic password or an LDAP password depending on the configured authentication realm. When the user tries to access CLP, the user will be prompted for 1FA or password. After successful authentication, the user is allowed to proceed and considered as logged in user. Refer to the [1FA \(Password Authentication\)](#) section for the configuration details.

- **Two-factor Authentication (2FA) - Email/SMS OTP**

This method uses Email/SMS OTP as a 2nd-factor authentication mechanism. When the user tries to access CLP, the user will be prompted for the 1st-factor authentication, normally a password-based authentication (username and password). Once the user has been successfully authenticated, the user will be prompted for the email/SMS OTP as the 2nd-factor authentication. Only when the user has been successfully authenticated using email/SMS OTP that the user is allowed to proceed and considered as logged in user. Refer to the [2FA \(Email/SMS OTP\)](#) section for the configuration details.

- **Two-factor Authentication (2FA) - Token OTP**

This method uses a token OTP (e.g. an OTP generated from OneSpan Digipass hardware token) as a 2nd-factor authentication mechanism. When the user tries to access CLP, the user will be prompted for the 1st-factor authentication, normally a password-based authentication (username and password). Once the user has been successfully authenticated, the user will be prompted for the token OTP as the 2nd-factor authentication. Only when the user has been successfully authenticated using token OTP that the user is allowed to proceed and considered as logged in user. Refer to the [CLP with 2FA \(Token OTP\)](#) section for the configuration details.

- **Windows Integrated Authentication (WIA)**

This method integrates CLP with Windows authentication via Microsoft Internet Information Services (IIS), so that users do not need to enter their primary AccessMatrix password anymore. Essentially, AccessMatrix automatically authenticates users who have been authenticated by Windows. Refer to the [Integrated Windows Authentication \(WIA\)](#) section for the configuration details.

- **OneSpan MAS for Mobile Scan and Login**

This method uses the OneSpan MAS (also known as DIGIPASS) for Mobile app as a digital token to identify and authenticate the user. CLP will display a Crontosign image. The user will then scan the Crontosign image using the app to log in to CLP using the digital token. Refer to the [OneSpan Scan and Login](#) section for the configuration details.

- **OneSpan MAS for Mobile Push to Login**

This method uses the OneSpan MAS (also known as DIGIPASS) for Mobile app to send a notification prompt on the user's device for authentication. The user completes the authentication process by tapping on their mobile device. Refer to the [OneSpan Push to Login](#) section for the configuration details.

- **SingPass Mobile**

This method uses SingPass Mobile for user authentication. CLP will display a QR Code as the authentication method. The user will then scan the QR Code using SingPass app to log in to CLP using OIDC, then successfully log in to the target application. Refer to the [SingPass Mobile](#) section for the configuration details.

- **FIDO2 Authentication**

This method uses a FIDO2 Authenticator (e.g. Windows Hello) for user authentication. The user will first be prompted for their User ID. Once the user has entered their User ID, they will be prompted to log into CLP via their registered FIDO2 sign-in method (e.g. Windows Hello Fingerprint). Refer to the [FIDO2 Authentication](#) section for the configuration details.

- **External OP Authentication via OIDC**

This method uses an external OIDC OpenID Provider (OP) for user authentication. The user will be redirected to the external OP's login page for authentication. Once they have successfully logged in via the external OP's page, they will be logged in to CLP. Refer to the [External OP Authentication via OIDC](#) section for the configuration details.

- **End-to-End Encryption Authentication (E2EEA)**

This method uses end-to-end encryption authentication for user authentication. The user will be

prompted for their User ID and Password. Once the user has entered their User ID and password, they will be logged into CLP. Refer to the [End-to-End Encryption Authentication \(E2EEA\)](#) section for the configuration details.

- **Certificate-based Authentication**

This method uses client certificates for user authentication. The user will be prompted to select the certificate to use for certificate-based authentication. Once the certificate has been authenticated, they will be logged in to CLP. Refer to the [Certificate-based Authentication](#) section for the configuration details.

3. AccessMatrix CLP Deployment

The following pages provide detailed instructions on how to deploy and configure the AccessMatrix Common Login Page (CLP) component:

- [Deployment Methods](#)
- [Deployment Configurations](#)

3.1. Deployment Methods

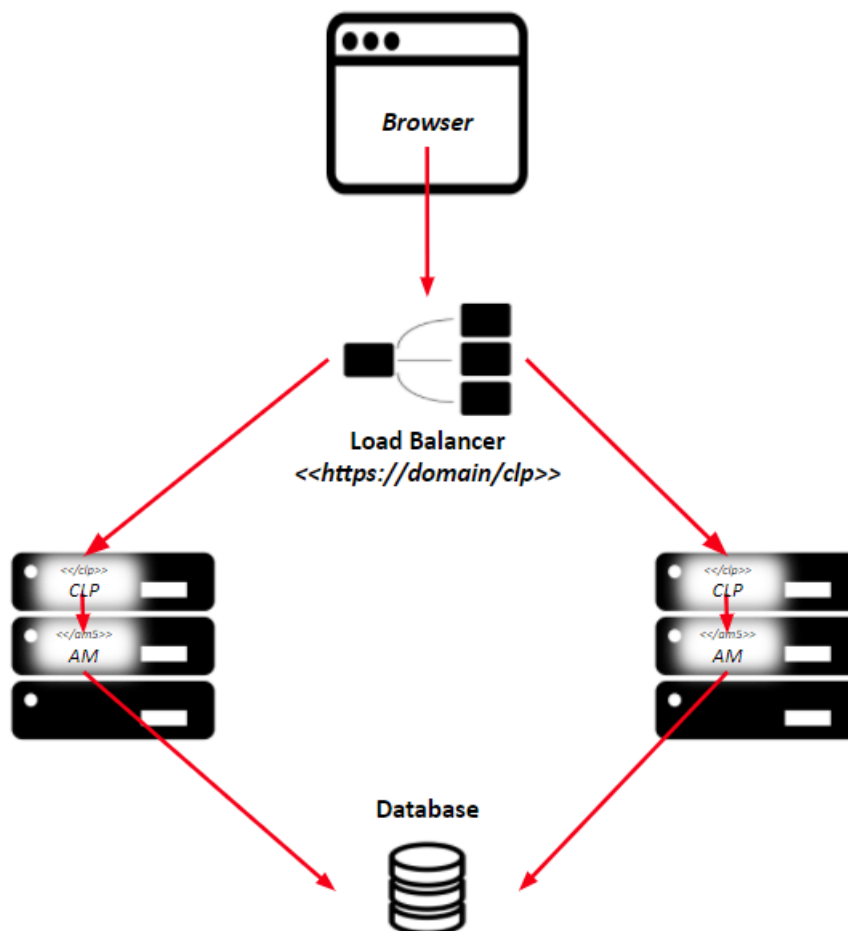
CLP component can be installed either by running the AccessMatrix installation package (via executable installer or zip archive) or by installing the CLP Web ARchive (WAR) package. There are two methods of deploying CLP:

- [Bundling CLP with AccessMatrix server.](#)
- [Deploying as a standalone instance](#) from the AccessMatrix server.

i **Note:** In this guide, <CLP_HOME> is used to refer to the installation directory of AccessMatrix CLP component (i.e. /opt/acmatrix/tomcat/webapps/clp).

Deploy AccessMatrix's Bundled CLP

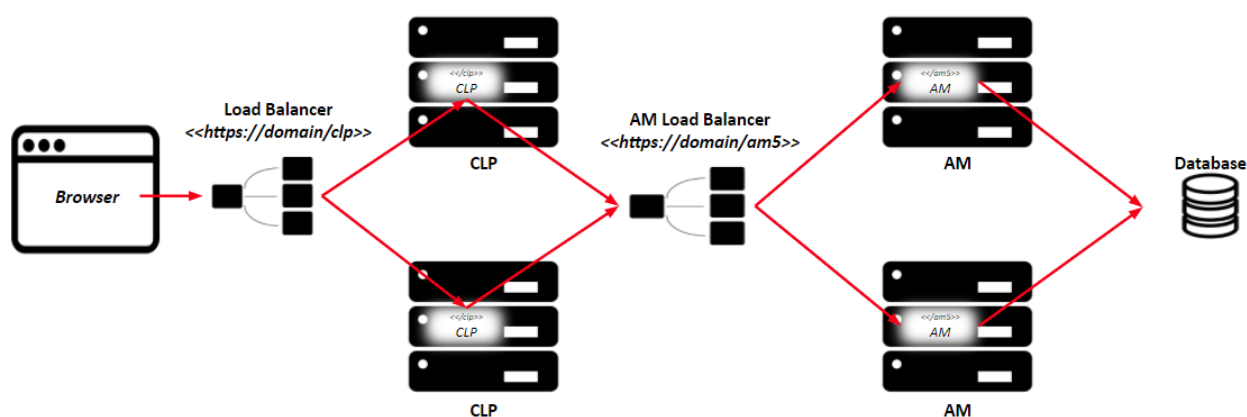
By default, installing the AccessMatrix installation package (via executable installer or zip archive) will deploy Common Login Page together with the AccessMatrix server and bundled Apache Tomcat server. An example deployment use case is shown below:



As shown in the diagram above, for multiple AccessMatrix instances that consist of the AccessMatrix application servers are deployed with Common Login Page (CLP) running together. A load balancer is needed in front of the CLP instances to distribute the workload among the instances. CLP will only communicate to the AccessMatrix Server in the same instance in this type of deployment.

Deploy Standalone CLP

The AccessMatrix Common Login Page can be deployed as Web ARchive (WAR) package onto JAVA-based web or application server (such as Apache Tomcat Server) and deployed as a separate instance from the AccessMatrix server. A load balancer is needed in front of the CLP instances if more than one is deployed for use. Another load balancer is needed between the CLP instances and AccessMatrix Servers if more than one AccessMatrix Servers are used. This deployment use case is shown below:



Note: The AccessMatrix installation package comes with a CLP WAR file (e.g. **AccessMatrix-x.x.x.xxxx-GA-clp.war**). Copy the WAR file to the server where the web or application server is installed. The WAR file may be renamed to **clp.war** for easy reference.

Deploy CLP on Apache Tomcat Server

To deploy the CLP Web Archive:

1. Navigate to Apache Tomcat's installation directory, e.g. */ApacheTomcat/apache-tomcat-9.0.35*.
 2. Extract the CLP WAR file contents to Apache Tomcat's webapps folder.
 3. Configure CLP to establish connection to the AccessMatrix server as described in [Connect to the AccessMatrix Server](#) section.
 4. Start the Apache Tomcat server.
 5. Launch the CLP application.
-



Note: If changes are introduced to the CLP Web Archive in the future released version of CLP, the administrator is required to redeploy the Web Archive.

3.2. Deployment Configurations

CLP Configuration Files

The table below lists the configurations that must be set up for Common Login Page:

Configuration File	Description
CLP Web Application Configuration (web.xml)	This file contains CLP settings such as AccessMatrix REST API URL configuration, SingPass redirect/callback URL, connection pool configuration, WIA redirect URL, SAML response URL, and common logout URL.
AccessMatrix Server Configuration (server.xml)	This file contains the AccessMatrix server configurations. For better security, you should configure the server to require a trusted certificate from the CLP agent.
CLP Configuration (am.commonlogin.config.xml)	This file contains the CLP configurations for authentication realm-specific login, segment-based login, reauthentication enablement, password change and self-service password reset enablement. Depending on your company requirements, you may or may not need to configure all of them.
CLP Logging Configuration (clplog4j2.properties)	This file contains the configurations of log level and log output file location. Refer to Configure CLP Log Properties section for configuration details.

Connect to the AccessMatrix Server

The configuration below establishes the connection between CLP and AccessMatrix server. To configure:

1. Navigate to `<CLP_HOME>/WEB-INF` and open **web.xml**.
2. Look for the `restAPIURL` element and modify the value to refer to the relevant AccessMatrix server as shown below:

XML
<pre><context-param> <param-name>restAPIURL</param-name> <param- value>http://localhost:8080/am5/rest</param-value> </context-param></pre>

To support client-authenticated SSL (a mandatory configuration for Production environment), the following settings should be configured:

-
- Modify the `restAPIURL` value to use secure connection to the AccessMatrix server as shown below:

XML

```
<context-param>
  <param-name>restAPIURL</param-name>
  <param-
value>https://localhost:8443/am5/rest</param-value>
</context-param>
```

- Configure the following client keystore properties:

XML

```
<context-param>
  <param-name>am.hc4.ssl.keystore</param-name>
  <param-value>/opt/acmatrix/tomcat/conf/clp-
client.keystore</param-value>
</context-param>
<context-param>
  <param-name>am.hc4.ssl.keystore-passphrase</param-name>
  <param-
value>AesEncryptor:pWtZZ0N6bfIvSlBBF0zXC0ZwSlKny/GexGVFpaqWI1Q
=</param-value>
</context-param>
```



Note: Refer to the [List of Configurable Parameters](#) section for the description of the properties.

3. Save the changes.
4. Restart the CLP server for changes to take effect.



Note: Refer to the [Connection Pool Settings](#) section if there's a need to fine tune the connection to AccessMatrix server.

Configure Connection Pool Settings (Optional)

Connection pooling refers to the method of creating a pool of connections and caching those connections so that it can be reused again. It improves the response time of a web-based application. This setting is used to fine tune the connection to AccessMatrix server.

To configure the connection pool parameters:

1. Navigate to `<CLP_HOME>/WEB-INF` and open **web.xml**.
 2. Look for the Connection Pool Configuration keywords and modify the parameter values, if necessary:
-

XML

```
<!-- Connection Pool Configuration -->
<context-param>
  <param-name>am.hc4.timeout.conn</param-name>
  <param-value>20000</param-value>
</context-param>
<context-param>
  <param-name>am.hc4.pool.maxconnperroute</param-
name>
  <param-value>50</param-value>
</context-param>
```



Note: You may add more connection pool parameters by referring to [List of Configurable Parameters](#) section.

3. Save the changes.
4. Restart the CLP server for changes to take effect.

Configure AccessMatrix Server Settings (Optional)

This section describes the recommended AccessMatrix server configurations you should make for enhanced security in your CLP deployment. These steps require the use of a non-interactive user, referred to as an agent, to enforce 2-way SSL authentication between AccessMatrix and CLP. Before starting with the steps below, you should have prepared an end-entity SSL certificate file for the agent, issued by a CA or its intermediate CA. Likewise, prepare the certificates of the CA and intermediate CA. The files should be placed into the directory `<AM_ROOT>/tomcat/conf/`.

Create Agent for Certificate-based Authentication

1. Navigate to **Operation Dashboard > User & Segment > User**, and click **Create**.
2. Provide an appropriate ***User Id** (e.g. "clpagent") and ***User Name** (e.g. "CLP Agent").
3. Under ***Interactive**, select "No".
4. Under **Distinguished Name**, specify the user's DN (e.g. "CN=clpagent, OU=Security, O=i-sprint, L=Singapore, ST=Singapore, C=SG"). Ensure these fields and values match those in the agent's certificate.
5. Click the **Save** button.

Import Certificates Into AccessMatrix Truststore

1. Open the Windows CLI.
-

-
2. Navigate to the directory `<AM_ROOT>/tomcat/conf/`, which contains the AccessMatrix truststore **testtrust.keystore**.
 3. Import the agent certificate into **testtrust.keystore** (the default password is "passphrase") using the command `keytool -import -v -trustcacerts -alias <agent certificate alias> -keypass passphrase -file <agent certificate file> -keystore testtrust.keystore`. For example:

```
keytool -import -v -trustcacerts -alias clpagent -keypass passphrase -file
agentcert.pem -keystore testtrust.keystore
```



Note: If a different AccessMatrix truststore is in use, import the certificate into the that truststore instead.

4. Next, import the CA and intermediate CA certificates into the same truststore by repeating the process described in step 3.

Configure AccessMatrix Server File

1. Navigate to `<AM_ROOT>/tomcat/conf/` and open **server.xml**.
2. Locate the `<Connector Port="8443".....>` element:

XML

```
<Connector port="8443" minSpareThreads="10"
protocol="com.isprint.amutil.tomcat.AMCustomNIO2Protocol"
    enableLookups="false" disableUploadTimeout="true"
    keepAliveTimeout="900000" maxKeepAliveRequests="-1"
    acceptCount="100" maxThreads="200"
    scheme="https" secure="true" SSLEnabled="true"
    maxParameterCount="1000"
>
    <SSLHostConfig certificateVerification="optional"
sslProtocol="TLS" protocols="TLSv1.2,TLSv1.3"
ciphers="TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_RSA_WITH_A
ES_128_GCM_SHA256,TLS_AES_256_GCM_SHA384,TLS_AES_128_GCM_SHA256"
truststoreFile="conf/testtrust.keystore"
truststorePassword="AesEncryptor:txNbulL6Ip9JtvEFzUZ1UsK2LzK8LJUyF2
RQgyTe2RU=" truststoreType="JKS">
        <Certificate
certificateKeystoreFile="conf/testserver.keystore"
certificateKeystorePassword="AesEncryptor:txNbulL6Ip9JtvEFzUZ1UsK2L
zK8LJUyF2RQgyTe2RU=" certificateKeystoreType="JKS"/>
    </SSLHostConfig>
</Connector>
```

Update the value of the `certificateVerification` field to "required" to enforce 2-way SSL authentication.

Note: If a different AccessMatrix truststore is in use, you should also ensure that the `truststoreFile` and `truststorePassword` fields are updated to the appropriate values.

3. Save the changes.

Disable Client Certificate Simulation

1. Navigate to `<AM_ROOT>/tomcat/webapps/am5/WEB-INF/classes/` and open **amsystem.properties**.
2. Locate the line `com.isprint.am.server.xmlrpc.XmlRpcServlet.$simCert.file=conf/testagent.cer` and ensure it is commented out:

Properties (.properties files)

```
# Uncomment the following to simulate the client certificate
authentication for POC or quick test without the need to set up
actual client cert authentication.
# This is normally used for http protocol, under https protocol, it
will not work if client certificate is present
# For security reasons, do NOT enable this in UAT or production
environments; instead mutual SSL authentication with a valid agent
certificate should be set up.
# == REST, XML-RPC, SOAP API ==
#
com.isprint.am.server.xmlrpc.XmlRpcServlet.$simCert.file=conf/testa
gent.cer
```

3. Save the changes.

Configure Session Token Cookie

Upon CLP login, a session token cookie (WSASID cookie) is generated. This session cookie is used to verify user identity and the session's validity. By default, the CLP session token cookie is removed when the browser used for the SSO session is closed. This means that the SSO session is removed upon closing the browser, and users must log in again after relaunching the browser.

This section describes configurations related to this CLP session token cookie.

Configure Session Token Cookie Domain and Path

The CLP cookie can be set to a domain and a context path.

To configure:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the **SessionCookie** element as shown below:

XML
<pre><SessionCookie> <domain></domain> <path></path> <SameSite></SameSite> </SessionCookie></pre>

3. Specify the domain value to allow the session cookie be set in the specified domain. By default, the domain value is empty, thus setting the session cookie in the default domain.
4. Specify the context path value. By default, path is set to `/`, thus allowing the session cookie to be set in all context paths in the current domain.

i	Notes: ○ For WSA, specify <code>/</code> to allow the applications behind WSA Reverse Proxy to access the session cookie. ○ For SAML and OAuth, it is recommended to use <code>/clp</code> or <code>/<customizedclpcontext_path></code> for better security control.
----------	---

5. Specify the **SameSite** value if required. By default, **SameSite** is empty, thus it would depend on browser's default behavior.

i	Note: You are recommended to configure "Strict" or at least "Lax". It is not safe to use "None".
----------	---

6. Save the changes made.
7. Restart the CLP server for the new changes to take effect.

Configure Session Token Cookie to Use AccessMatrix Session Token Lifetime

i	Note: Currently, this feature is only supported for the SAML protocol.
----------	---

You can configure the CLP cookie to use the lifetime of AccessMatrix's session token instead. As long as the AccessMatrix session token has not expired, this will allow the SSO session to be persisted even after closing the browser. This means that users do not have to log in again after closing and relaunching their browsers. To configure this, perform the following steps:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
-

-
2. Under the authentication realm that you intend to use for CLP login, uncomment the `<useSessionTokenLifetimeForCookie>` setting and set the value to "true". For example:

XML

```
<AuthnRealm>
  <id>PwdRealm</id>
  <idLabel>Lbl_PwdRealm</idLabel>

  <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

  <loginModuleIdForChangePassword>SystemPasswordBasic</loginModuleIdForChangePassword>
  <stateForChangePassword>login.chgpwd</stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>SystemPasswordBasic</handler>
    <loginState>login.start</loginState>
  </loginModule>

  <useSessionTokenLifetimeForCookie>true</useSessionTokenLifetimeForCookie><!-- This is to define whether to use the AM5 session token lifetime expiry for cookie session expiry. Default:false -->
</AuthnRealm>
```

3. Save the changes.
4. Restart the CLP server for changes to take effect.

Configure the lifetime duration of the AccessMatrix session token via the AccessMatrix Admin Console. The session token lifetime settings are located under the realm-based session policy assigned to the authentication realm you are using, or under **Global Setting** if no policy is assigned in the authentication realm. To configure this, perform the following steps:

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** and select the authentication realm you intend to use for CLP login.
 2. Check the **Realm-Based Session Policy** setting to determine where you should make the configurations:
 - a. If there is already a session policy specified, you can navigate to its settings page by:
 - i. Clicking on it;
-

-
- ii. Scrolling down in the pop-up window and clicking on the **View in main panel** button; and
 - iii. Clicking the **Edit** button in the **View Realm-Based Session Policy** page you are brought to.
 - b. If there is no session policy specified, navigate to **Administration Dashboard > Configuration > Global Setting**, click the **Edit** button, and switch to the **Session** tab.
 3. Under the **Session Lifetime (in seconds)** attribute, specify the desired session lifetime in seconds.
 4. Under the **Session Idle Time (in seconds)** attribute, specify the same value as above.

! Important Note: You MUST configure **Session Idle Time** with the same value as **Session Lifetime**. If **Session Idle Time** is shorter than **Session Lifetime**, the SSO session will expire before the full **Session Lifetime** duration is over.

5. Save the changes.

Configure CLP Log Properties (Optional)

CLP uses Apache Log4j 2 as the logging utility. It defines the log level and log output file location.

To configure:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **clplog4j2.properties**.
2. Modify the log level, if necessary.
3. Save the changes.
4. Restart the CLP server for changes to take effect.

List of Configurable Parameters

Below are the list of connection pool parameters supported by CLP that can influence the connection operations to AccessMatrix server.

Parameter Name	Description
Connection Pool	

Parameter Name	Description
am.hc4.pool.maxconn_per_route	The total number of connections limit to a single route (host and port). Default: 50
am.hc4.pool.maxconn_total	The total maximum connections available in the connection pool. Default: 100
am.hc4.pool.ttl_ms	Defines the maximum life span of persistent connections regardless of the allowed time-to-live returned by server. Default: 180000 milliseconds
Retry	
am.hc4.retry.times	The maximum number of times a method will be retried. Default: 1
am.hc4.retry.request_sent_retry	Set to "true" to retry methods that have successfully sent their requests. Default: false
Timeout	
am.hc4.timeout.conn	Determines the timeout in milliseconds until a connection is established. Default: 10000 milliseconds
am.hc4.timeout.socket	Determines the timeout when waiting for receiving or sending data (socket read/write timeout). Default: 30000 milliseconds
am.hc4.timeout.conn_request	Maximum wait time to get a connection (waiting for connection pool to return a connection). Default: 30000 milliseconds
SSL	
am.hc4.ssl.ciphers-suites	Specifies the ciphers that facilitate secure sockets layer/transport layer (SSL/TLS).
am.hc4.ssl.protocols	The networking protocol used to secure connections between web clients and web servers over an insecure network.
am.hc4.ssl.keystore	The Java keystore file that contains the key used for client-authentication SSL.

Parameter Name	Description
am.hc4.ssl.keystore-passphrase	The password of the Java keystore. This password or passphrase can be a clear password or an encrypted password value. Refer to the Common Deployment Guide - Encryptor Utility Tool section for the encryption steps.
Proxy	
am.hc4.proxy.host	The host name or IP address of a proxy server that you want to connect to.
am.hc4.proxy.port	The number of the port that the proxy server uses to listen for client connections.
am.hc4.proxy.username	The user name that you use to access the proxy server.
am.hc4.proxy.password	The password that you use to access the proxy server.
am.hc4.proxy.ntlm	Indicate whether NTLM proxy authentication is enabled. Default: false
am.hc4.proxy.ntlm.domain	The domain where credentials are intended to authenticate with.
am.hc4.proxy.ntlm.host	The host or workstation name of the computer originating the authentication request.
Socket	
Advanced Java Socket object configurations. In most cases, there is no need to configure this. Refer to Java SocketOptions for more details, including the default values. Some settings may not work on certain Operating Systems (OS) and require setting them at the OS-level instead.	
am.hc4.socket.receive-buffer-size	Sets the size of the receive buffer for newly created sockets.
am.hc4.socket.send-buffer-size	Sets the size of the send buffer for newly created sockets.
am.hc4.socket.keep-alive	Indicate whether or not to have socket keep alive turned on.
am.hc4.socket.linger	Sets the information about the linger state of the associated socket.
am.hc4.socket.timeout	Defines the socket timeout in seconds, which is the time waiting for data - after establishing the connection. It is the maximum period of inactivity between two consecutive data packets. This setting is the same as am.hc4.timeout.socket.
am.hc4.socket.tcp-no-delay	Sets the socket TCP_NODELAY flag.

For more details on the behavior of the configuration, refer to [HTTP Client 4.5](#) documentation.

4. AccessMatrix CLP Application Integrations

The following pages provide detailed information on the supported AccessMatrix Common Login Page (CLP) application integrations:

- [CLP for AccessMatrix Applications](#)
- [CLP for SAML Identity Provider](#)
- [CLP as OIDC OpenID Provider](#)
- [CLP for WSA Login](#)

4.1. CLP for AccessMatrix Applications

Below are the main steps to set up CLP for AccessMatrix Applications such as USO Server, USO SSF, and UAS TAP:

1. Ensure that the [prerequisites](#) are met.
2. Edit the [Common Login Page Configuration](#) to configure the AccessMatrix application URL.
3. Edit the AccessMatrix application to configure the [CLP URL](#).
4. Configure the [CLP authentication realm](#) of the AccessMatrix applications.

Prerequisites

- Prior to CLP configuration, it is assumed that CLP WAR file is already configured to connect to the AccessMatrix server as described in [Connect to the AccessMatrix Server](#) section.
- Ensure that the system time between AccessMatrix server and CLP Application's server are synchronized, if deployed in separate servers.
- Ensure that the application-specific configuration is already configured in CLP. Refer to [Configure Session Token Cookie Domain and Path](#) section.

Edit Common Login Page Configuration

To integrate AccessMatrix web applications with CLP, you need to configure the client type and product target URL in Admin Console so that the AccessMatrix system will know which application is calling CLP for authentication. To configure:

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Common Login Page**.
2. Click **Edit** to modify the CLP configuration for a specific AccessMatrix web application, if necessary.
3. Save the changes.

The table below lists the attributes and its descriptions for the configurations.

CLP Configuration Page Attribute Description

	Note: ' * ' denotes mandatory attribute.
---	---

Attribute Name	Description
*Configuration Id	Unique Id of configuration, i.e. "CommonLoginPageConfiguration".
*Configuration Name	Name of configuration, e.g. "CommonLoginPageConfiguration".
Description	Description of configuration, e.g. "Common Login Page Configuration".
Version	Configuration entry version.
Implementation	Type of configuration implementation, i.e. "Common Login Page Configuration".
Attributes Tab	
Client Type and Product Target URL	Refers to the client type and the product target path. Some default configurations are already provided: • USO, /usoserver/start.uso • SSF, /usossf/start.ssf • UCM, /ucmweb/index.jsp • TAP (Token Activation Portal), /uastap/clp.jsp By default, this field is already filled with a relative path of the applications. However, modify the product target URL if the IP and port are provided.
Token Validity Period (in seconds)	The difference between the time the login request is issued and the time the login request is received must be within this configured time. The value should not be less than 300. Default: 540
Cache	
Session Nonce Cache Size	The number of session nonce to be kept in the cache. The cache is used to verify the login session. The value should not be less than 1000. Default: 5000

Configure the CLP URL

USO Server

Set the CLP URL in the USO Server application by following the steps below:

1. Navigate to `<USOSERVER_HOME>/assets/` and open **clp_url.js**.



Note: `<USOSERVER_HOME>` is used to refer to the installation directory of USO Server application (i.e. `/opt/acmatrix/tomcat/webapps/usoserver`).

2. Specify the angular constant `AM_CLP_URL`, if necessary.

JavaScript

```
(function() {  
    ...  
    angular  
        .module('usoagent')  
        .constant('AM_CLP_URL', '/clp')  
        ...  
})();
```

3. Save the changes and restart USO Server for changes to take effect.

USO Self-Service Facility (SSF)

Set the CLP URL in the USO SSF application by following the steps below:

1. Navigate to `<USOSSF_HOME>/assets` and open **clp_url.js**.



Note: `<USOSSF_HOME>` is used to refer to the installation directory of USO Server application (i.e `/opt/acmatrix/tomcat/webapps/usossf`).

2. Specify the angular constant `AM_CLP_URL`, if necessary.

JavaScript

```
(function() {  
    ...  
    angular  
        .module('usossf')  
        .constant('AM_CLP_URL', '/clp/axmxsso');  
})();
```

3. Save the changes and restart USO SSF server for changes to take effect.

UAS Token Activation Portal (TAP)

Set the CLP URL in the UAS TAP application by following the steps below:

1. Navigate to `<UASTAP_HOME>/core` and open **index.config.js**.



Note: `<UASTAP_HOME>` is used to refer to the installation directory of UAS TAP application (i.e. `/opt/acmatrix/tomcat/webapps/uastap`).

2. Specify the url, if necessary.

JavaScript

```
var CONFIG = {  
    CLP: {
```

```
        automatic: true,  
        url: "/clp/axmxsso"  
    }  
}
```

3. Save the changes and restart UAS TAP server for changes to take effect.

Configure the CLP Authentication Realm

USO Server

For USO server application, there is an option to set the default authentication realm in CLP. This can be done by following the steps below:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Enterprised SSO > USO Server Configuration**.
2. Click **Edit** to modify the USO Server configuration on the right pane.
3. Specify the default authentication realm for USO users by setting the attribute value for **Default authentication for USO users** attribute.



Note: By default, this attribute is already set with 40151 as the authentication realm. You may select another authentication realm from the drop-down list.

4. You have the option to use this authentication realm as CLP's default authentication realm by setting "Yes" as the value of the **Specify Default Authentication to CLP** attribute. This will bypass the selection of authentication realm in CLP.
5. Save the changes and restart AccessMatrix server for changes to take effect.

UAS Token Activation Portal (TAP)

For UAS TAP application, a default authentication realm in CLP can be set by following the steps below:

1. Navigate to `<UASTAP_HOME>/WEB-INF` and open **web.xml**.



Note: `<UASTAP_HOME>` is used to refer to the installation directory of UAS TAP application (i.e. `/opt/acmatrix/tomcat/webapps/uastap`).

2. Look for the CLP/REALM_ID param-name and specify the authentication realm in the param-value, e.g. "40151" as shown below:

XML

```
<init-param>
  <param-name>CLP/REALM_ID</param-name>
  <param-value>40151</param-value>
</init-param>
```

You can specify multiple authentication realms, just separate the authentication realm values with a comma.

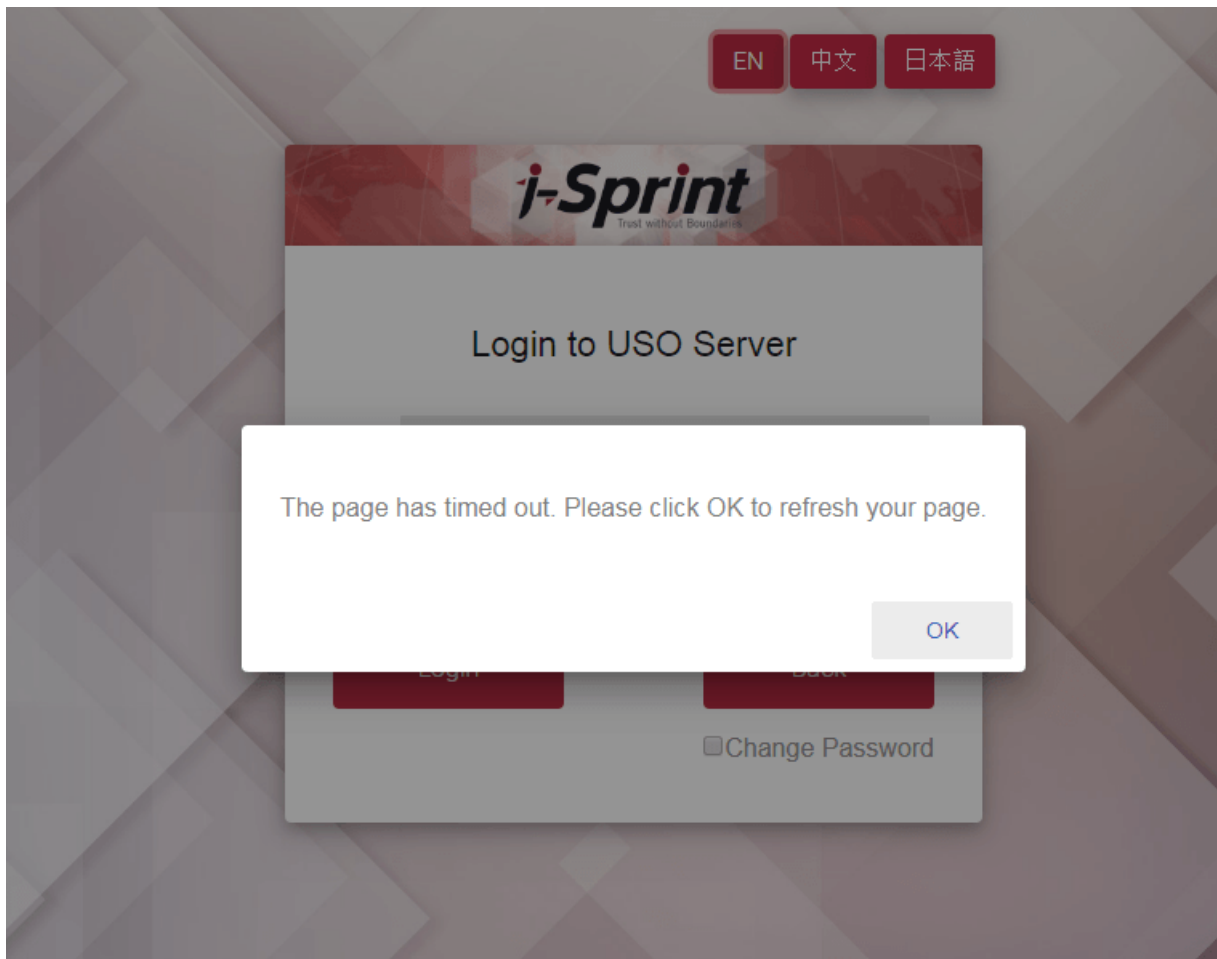
3. Save the changes and restart UAS TAP for changes to take effect.

Session Management

When the AccessMatrix web applications, such as USO Server, USO SSF and UAS TAP, use CLP for the login requests, the AccessMatrix server maintains the CLP login request session by having an expiry timeout to ensure the security of the login process. See how CLP manages the CLP login request timeout below:

CLP Web Application Page at idle state

AccessMatrix web applications using CLP, such as USO Server, shows a timeout dialog box after the login request has expired. This timeout happens when the login page is left idle for a long time. The CLP timeout setting is configurable in the Admin Console (see **Token Validity Period (in seconds)** as described in [Common Login Page Configuration](#) section). When the timeout dialog box is displayed, click the **OK** button to redo the login request. See below.



CLP Session Token

As mentioned above, when an AccessMatrix web application sends a login request to CLP, a CLP login request session is created. User must complete a login request before it expires. The CLP login request also cannot be reused. It contains nonce which is for one-time use. This is to prevent replay attack and also to avoid request forgery. The login request session also contains the originating requester so that once the user has completed the login process in CLP, CLP can redirect the user back to the originating application. The URL of the originating application is registered in AccessMatrix, configurable via the Admin Console, to ensure that the user login response is sent to the correct AccessMatrix web application URL.

CLP Nonce Cache

During the generation of the CLP token, AccessMatrix stores the session nonce into CLP nonce cache, and removes it from the cache after a login request is completed. When there are multiple AccessMatrix server instances, the nonces need to be accessible by all the AccessMatrix Server instances that are handling CLP requests. Normally Ehcache is used as the CLP nonce cache. In this case, Ehcache

replication needs to be configured. Please refer to [Common Deployment Guide - Cache Configuration - Replication](#) to learn how to setup Ehcache replication. For cloud-based deployment, normally Redis cache is used. In this case, as long as AccessMatrix Server is configured to use Redis cache, there is no additional configuration required for this CLP nonce cache.

User Re-Authentication

[Configure USO Application Reauthentication Attributes](#)

[USO Administration Guide - USO Admin Console Administration](#)

4.2. CLP for SAML Identity Provider

The following sections will provide configuration steps for setting up CLP as the SAML Identity Provider (IdP) Front End.

Supported SAML Flows

CLP supports standard **Web Browser SSO flows**, which can be initiated by either the Service Provider (SP) or the Identity Provider (IdP). The request can be in either HTTP-Redirect or HTTP-POST formats in Service Provider initiated flow. Please refer [SAML Web SSO Configuration](#) to know more about the set up. An illustration is provided below:-

i **Note:** All responses from the Identity Provider are returned in HTTP-POST.

Entity	Request Type
Service Provider	HTTP-Redirect or HTTP-POST
Identity Provider	HTTP-Redirect or HTTP-POST

Prerequisites

- Prior to CLP configuration, it is assumed that CLP WAR file is already configured to connect to the AccessMatrix server as described in [Connect to the AccessMatrix Server](#) section.
- Ensure that the system time between AccessMatrix UAM server, CLP Application's server and SAML Service Provider (SP) are synchronised, if deployed in separate servers.
- Ensure that the SAML-specific configuration is already configured in CLP. Refer to [Configure Session Token Cookie Domain and Path](#) section.

Configuring CLP as the SAML Identity Provider (IdP) Front End

By using CLP as Identity provider(IdP) front end, the administrator may perform the following steps:

1. [Configure SAML Identity Provider \(Idp\)](#)
 2. [Configure SAML Service Provider \(SP\)](#)
 3. [Configure SAML SP Redirection Page \(OPTIONAL\)](#)
 4. Configure User Authentication
 5. [Configure User Re-Authentication](#)
-

6. Configure Login Policy (OPTIONAL)

Configure SAML Identity Provider (IdP)

To configure IdP when CLP work as frontend for IdP:

1. Navigate to Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > **Identity Provider**
2. Click the Create button and select "UAM as Identity Provider" .
3. Provide the details required:
 - **Identity Provider Id** - Unique Id of Identity Provider, e.g. "MyIdP".
 - **Identity Provider Name** - Name of Identity Provider, e.g. "Identity Provider".
 - **IdP issuer** - A kind of Id, normally in the URL form to represent the Identity Provider entity.
 - **Single-Sign-On (SSO) URL** - Sometimes it is called login URL or SSO login URL. This URL is given by the IdP and used to authenticate the user. The IdP looks for valid login credentials or checks for valid session cookies. In this case, AccessMatrix server acts as the back end of the identity provider and CLP serves as the front end to authenticate the SAML user. By default, the URL should look like this:
`http://<CLP_Domain>:<port>/clp/samlSso`

For more details, please see [Configure SAML Identity Provider \(IdP\)](#) section.

Configure SAML Service Provider (SP)

To configure SP when CLP work as frontend for IdP.

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Service Provider.**
 2. Click the **Create** button. Select "UAM as Identity Provider" .
 3. Provide the details required:
 - **Service Provider Id** - Unique Id of Identity Provider, e.g. "samlsp". The value to be retrieved from the user object as user identity.
-

-
- **Service Provider Name** - Name of Identity Provider, e.g. "Service Provider".
 - **SAML SP issuer ("Id used as issuer" attribute)**
Service Provider provides a standard issuer URL or sometimes it allows the user to choose or use the domain-specific issuer.
 - **Value or User Response to return as NameId value**
 - **Assertion Consumer Service (ACS) URLs**
The Service Provider's URL to send the SAML Response to. Normally, the Service Provider will provide this information in the documentation or as part of its configuration. Some cloud services may provide a unique ACS URL for each account/tenant.
(eg. `http://<CLP_Domain>:<port>/`)

More information, please refer to [Configure SAML Service Provider \(SP\)](#) section.

- **IdP issuer URL** - This URL denotes the identity of the IdP.
 - **SP issuer URL** - SP gives a standard issuer URL or sometimes it would allow the user to use the user's domain-specific issuer.
 - **Single-Sign-On (SSO) URL** - Sometimes it is called login URL or SSO login URL. This URL is given by the IdP and used to authenticate the user. The IdP looks for valid login credentials or checks for valid session cookies. In this case, AccessMatrix server acts as the back end of the identity provider and CLP serves as the front end to authenticate the SAML user. By default, the URL should look like this: `http://<CLP_Domain>:<port>/clp/samlssso`.
 - (Optional) **Signing Certificate** - This will be used to trust SAML messages originating from IdP software.
 - (Optional) **Allow All Users to Access** - Set to "false" to disable access for all users. You can also restrict access for specific users or groups instead:
 - To configure access for a specific user, refer to **SAML Application Privilege Tab** in [AccessMatrix Common Administration Guide - Manage Users](#).
 - To configure access for a group, refer to **SAML Application Privilege Tab** in [AccessMatrix Common Administration Guide - Manage Groups](#).
-

Configure SAML SP Redirection Page (OPTIONAL)

i Note: After SAML Single Sign-On (SSO) processing is done, CLP will use the SAML Response Page to send SAML response via HTTP-POST (when SAML SSO is successful) or to display an error (if an error is encountered). If there is a need to customize or to use a different SAML Response page, the `samlResponsePageUrl` param-value in the configuration below can be changed. Refer to the default **samlredirect.jsp** as a reference.

After SSO processing, CLP forwards the SAML response or displays an error to SAML Response page. The SAML Response page must be a JSP where the `SAMLAuthnProcessor` servlet will dispatch using a forward call. By default, `samlredirect.jsp` is used.

The following request attributes can be accessed within the page:

Request Attribute	Description
<code>samlResponseHtmlEncoded</code>	The SAML Response payload to send to the SAML SP ACS URL.
<code>relayStateHtmlEncoded</code>	Refers to the relay state sent as part of the SAML request.
<code>acsUrl</code>	The SP URL that receives the SAML response from SAML IdP.
<code>spName</code>	Refers to the SP display name.
<code>errorMessage</code>	Refers to the error message for a failed case.

To configure the SAML Authentication Processor:

1. Navigate to `<CLP_HOME>/WEB-INF` and open **web.xml**.
2. Look for the `SAMLAuthnProcessor` servlet class and configure as shown below:

XML

```
<servlet>
  <servlet-name>SAMLProcessor</servlet-name>
  <servlet-
class>com.isprint.am.commonlogin.init.SAMLAuthnProcessor</servlet
-class>
  <init-param>
    <param-name>loginPageUrl</param-name>
    <param-value>/index.jsp</param-value>
  </init-param>
  <init-param>
```

```
<param-name>samlResponsePageUrl</param-name>
<param-value>/samlredirect.jsp</param-value>
</init-param>
</servlet>
```

3. By default, the `samlredirect.jsp` page is configured as the `samlResponsePageUrl` value. However, this value can be modified to point to the customer's actual SAML Service Provider (SP) redirect page, if necessary.
4. Save the changes.
5. Restart the CLP server for the changes to take effect.

User Authentication

The administrator must configure **authentication realms** for user authentication. Steps:

~~The SAML IdP configuration will require configuration of multiple authentication realms. These authentication realms will be used for user authentication. Subsequently, make sure that the users of the service provider are assigned with the corresponding authentication realms.~~

1. Configure User Lookup Module. Refer to [Manage User Authentication Service - Configure User Lookup Module Section](#).
 2. Configure Login Module. Refer to [Manage User Authentication Service - Configure Login Module Section](#).
 3. Configure Authentication Realm. Refer to [Manage User Authentication Service - Configure Authentication Realm](#).
 4. Configure Login Account. Refer to [Manage User Authentication Service - Configure Login Account](#).
 5. **(Optional)** Configure Login Policy. Refer to [Configure Login Policy](#) to whitelisted Authentication Realms.
 6. Configure the Authentication Realm in CLP. Refer [Configure the Authentication Realms in CLP](#).
 7. ~~To configure the authentication realms of users, refer to [AccessMatrix Common Administration Guide - Configure Authentication Realm](#) section.~~
 8. ~~To assign the authentication realms:–~~
 - ~~For specific users, refer to **Login Account** tab in [AccessMatrix Common Administration Guide - Manage Users](#).~~
-

-
- For groups, refer to **Login Account** tab in [AccessMatrix Common Administration Guide - Manage Groups](#).
 - For the corresponding SAML configurations on AccessMatrix server, refer to [UAM SAML Implementation Guide - UAM as SAML Identity Provider](#).

User Re-Authentication

The administrator may proceed to perform the following step for user re-authentication. Steps:

1. Configure Login Module. Refer to [Manage User Authentication Service - Configure Login Module Section](#).
2. (Optional) Configure Login Policy. Refer to [Configure Login Policy](#) to whitelisted Reauthentication Login Modules.
3. Configure Reauthentication Login Modules in CLP. Refer [Configure the Re-Authentication Methods](#).

Configure Login Policy (OPTIONAL)

1. Navigate to **Security Policy > Unified SSO > Federated SSO > Application Policy > Identity Provider Policy**
 2. Click the **Create** button and select "**SAML IdP Policy**".
 3. Define:
 - Enforce Policy = true
 - Whitelisted Authentication Realms
 - Whitelisted Reauthentication Login Modules
 4. Apply the policy to target Service Provider. Navigate **Configuration > Unified SSO > Federated SSO > SAML > Service Provider**
 - a. Select the Service Provider under "SPTemplateUamAsIdP" template.
 - b. Under "Policy" tab, assigned the policy just created under "Application Policy".
 5. Make sure the the Realms and Login Modules whitelisted in policy is enabled in CLP.
-



Note: If no policy is enabled, or if realms/login modules are not whitelisted, CLP will fall back to the default configuration defined in `am.commonlogin.config.xml`.

Log out SAML IdP's User Session in CLP

The `/clp/logout` endpoint is used to log out AccessMatrix session token cookie (WSASID cookie) via web browser. Upon receiving this, CLP will display a confirmation page for the user to log out. Optionally, your application may use this endpoint to trigger logout for the IdP's user session.



Note: This does not refer to SAML single logout.

4.3. CLP as OIDC OpenID Provider

i **Note:** The license for using AccessMatrix as OpenID Provider (OP) is controlled and checked based on the number of the registered OAuth Client(s) - under **OAuth Client** of **UAM FEATURE**. Ensure that the number of OAuth Client(s) you intend to register does not exceed the configured value of this license checking.

AccessMatrix UAM supports OpenID Connect version 1.0 and CLP is designed to be the front-end server of the OP. Below is the list of OIDC Endpoints supported by CLP:

Name of OIDC Endpoint	Endpoint URL
Authorization endpoint	https://<CLP_Domain>/<path>/auth
Token endpoint	https://<CLP_Domain>/<path>/token
User info endpoint	https://<CLP_Domain>/<path>/userinfo
JWKS endpoint	https://<CLP_Domain>/<path>/jwks.json
Revocation endpoint	https://<CLP_Domain>/<path>/revoke
Introspection endpoint	https://<CLP_Domain>/<path>/introspect
OpenID Provider Configuration /Discovery endpoint	https://<CLP_Domain>/<path>/well-known/openid-configuration

CLP as OIDC OpenID Provider Setup Steps

Below are the main steps to enable CLP as OpenID Provider:

1. [Configure CLP as the OpenID Provider](#) in Admin Console.
2. In order to integrate your application to use CLP for OIDC authentication or SSO, firstly, you need to [register your application](#) in Admin Console. Your application can use OpenID Discovery endpoint to get URLs of the supported OpenID Provider endpoints and some other useful metadata info. In addition to that, your application might also want to call JWKS endpoint to obtain the RSA public key for signature verification of user's OIDC Tokens, e.g. ID Token and, if applicable, Access Token too.

i **Note:** As mentioned in the CLP deployment steps, make sure that the setting of the session cookie's domain and path are properly configured to ensure that the CLP session cookie is accessible and allows SSO. If OIDC

Silent Authentication is needed, set the SameSite value to "None" as well. CLP also allows some customization related to the use of CLP as OpenID Provider. You may consider some CLP customizations, as described in the Customizations for CLP as OP section.

Configure CLP as OP

Perform the steps below to set up the identity provider in Admin Console:

1. Create the RSA Asymmetric keys to be used for signing of the OIDC tokens, e.g. ID Token, optionally, access token too.
2. Define the URLs of the OpenID Connect endpoints based on your CLP deployment's URL and context path.
3. Configure the scopes and user consent, if required.

Refer to [UAM OAuth and OIDC Implementation Guide - UAM as OIDC OpenID Provider](#) for the detailed steps of the above-mentioned configurations.

Register the Application (Relying Party)

OIDC Application/Relying Party will receive OIDC tokens as a response from CLP once the user has an existing session in CLP or has been authenticated via CLP. CLP supports the following OIDC authentication grants with user authentication:

- Authorization Code (deprecated)
- Authorization Code with PKCE
- Implicit (deprecated)

You need to create an OAuth Client in Admin Console as part of the registration process. Refer to [UAM OAuth and OIDC Implementation Guide - OAuth Client](#) for the detailed steps of the configurations.

Customizations for CLP as OP (OPTIONAL)

CLP as an OpenID Provider allows the customization of the following components:

- Display content of scopes
 - OIDC error page
-

Customize Display Content of Scope(s)

Perform the steps to customize the display content of scope(s) for OIDC User Consent Page:

1. Navigate to `<CLP_HOME>/assets/oauth` and open **oauth_consent.json**.
2. You will see some predefined OIDC scopes and descriptions as shown below:

```
JSON
{
  "profile": "Personal information",
  "email": "Email",
  "phone": "Phone number"
}
```

3. You may modify the file to change the text of the existing scopes or add a new scope and text.
For example:

- Change the text of phone scope to Contact No:

```
JSON
{
  "profile": "Personal information",
  "email": "Email",
  "phone": "Contact No"
}
```

- Add a new scope DOB with Date of Birth as its text value:

```
JSON
{
  "profile": "Personal information",
  "email": "Email",
  "phone": "Contact No",
  "DOB": "Date of Birth"
}
```

4. Save the changes.
5. Restart the CLP server for changes to take effect.

OIDC Error page

To change the error label of an OIDC error page, refer to [Customize Texts/Labels](#) section for the steps. However, if you intend to display your own error page to end users when a problem is encountered while using OIDC authentication, then follow the steps below:

-
1. Navigate `<CLP_HOME>/WEB-INF` and open **web.xml**.
 2. Look for the `OAuthRequestServlet` element as shown below:

XML

```
<!-- oauth-proxy Configuration -->
<servlet>
  <servlet-name>OAuthRequestServlet</servlet-
name>
  ...
  <init-param>
    <param-name>errorPageUrl</param-name>
    <param-value>/oautherror.jsp</param-value>
  </init-param>
</servlet>
```

3. By default, `/oautherror.jsp` is the value of `errorPageUrl`. Modify this value to point to your error page URL.
4. Save the changes.
5. Restart the CLP server for changes to take effect.

Logout OP's User Session in CLP

The `/clp/logout` endpoint is used to log out AccessMatrix session token cookie (WSASID cookie) via web browser. Upon receiving this, CLP will display a confirmation page for the user to log out. Optionally, your application may use this endpoint to trigger logout for the OP's user session.

Configure CORS for OIDC Endpoints

Refer to [Cross-Origin Resource Sharing \(CORS\) for OIDC Endpoints](#) for more details on the steps required.

4.4. CLP for WSA Login

Web Service Agent (WSA) is a component that intercepts all HTTP requests into a web or application server. In this case, the Common Login Page (CLP) is used to authenticate users before reaching the web or application server.

The following sections will provide configuration steps for setting up CLP for WSA login.

Prerequisites

- Ensure that the AccessMatrix UAM Server is successfully integrated with WSA Reverse Proxy (RP) Component. Refer to the [UAM WSA Reverse Proxy Implementation Guide](#) for the detailed implementation steps of WSA RP. Complete the testing by using the WSA RP demo files bundled with the WSA RP zip file as a target web application.
- Prior to CLP configuration, it is assumed that CLP WAR file is already configured to connect to the AccessMatrix server as described in [Connect to the AccessMatrix Server](#) section.
- Ensure that the system time between AccessMatrix server, WSA application's server and WSA server are synchronised, if deployed in separate servers.
- Ensure that the WSA-specific configuration is already configured in CLP. Refer to [Configure Session Token Cookie Domain and Path](#) section.

Configure CLP for WSA

1. Navigate to **Access Agent page by selecting Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent** on the left pane.
 2. On the right pane, select the Web Security Agent (i.e mydemo) that is already created during integration between AccessMatrix UAM Server and WSA RP Component in Search Access Agent Result. Refer to [UAM WSA RP Implementation Guide - Deploy Sample Application of WSA Reverse Proxy](#) for the web application configuration.
 3. Click the **Edit** button to modify the **Login URL** attribute of mydemo application to point to CLP's WSA endpoint. This login URL can be configured in 2 ways:
 - Using the default URL, e.g. `http://<CLP_Domain>:<port>/clp/wsa`. When WSA intercepts this URL, the default CLP main page with authentication realm selection will be displayed.
 - (Optional) Specifying the realm ID in URL to allow direct access to login page, e.g. `http://<CLP_Domain>:<port>/clp/wsa?realmId=40151`. When WSA intercepts this URL, CLP
-

login page will be displayed directly without requiring selection of the authentication realm, if the provided realm is already configured in CLP. However, if the realm ID is not configured in CLP yet, the CLP main page with authentication realm selection will be displayed.

4. Save the changes.

Configure CLP for WSA in WSA RP Component

1. Configure **wsarp.conf** file in WSA server, add reverse proxy URL mapping for CLP under `<VirtualHost server.wsa.com:80>`:

XML

```
<Location "/clp">
  ProxyPass http://<CLP_Domain>:<port>/clp
  ProxyPassReverse http://<CLP_Domain>:<port>/clp
</Location>
```

2. Save the changes.
3. Restart the WSA server for the changes to take effect.

Perform WSA Login

1. Open a web browser and access the web link, for example,
<http://server.wsa.com/wsatest/private/index.jsp>.
2. Make sure it is redirected to the Common Login Page for the authentication process.
3. If WSA has been started and successfully initialized, the user is redirected to the specified CLP login URL.
4. After user login successfully, CLP redirects the user to the desired application page.

5. AccessMatrix CLP UI Customizations

AccessMatrix Common Login Page (CLP) supports UI customization which allows users to set their preferences on what they want to see or how they want information to be organized or displayed. CLP allows customization of the following components:

- [Page Background image](#)
- [Logo](#)
- [Authentication Realm value](#)
- [Texts or labels](#)
- [Logout page](#)

The CLP WAR file contains the following folders and sub-folders:

Folder/File Name	Sub-folder	Description	Usage
assets	css	Contains the CSS files for CLP customization (button, background image, etc.)	UI customization
	i18N	Contain the English, Japanese, and Chinese labels	UI customization
	ico	Contain the Icons' images.	UI customization
	img	Contain the background images.	UI customization
	logo	Contain the logo image.	UI customization
	oauth	Used for OAuth login. Scope label for user consent.	UI customization
	font	Contains the font CSS.	UI customization
scripts	-	Angular JS files	-
styles	-	Angular JS CSS files	-
WEB-INF	classes/ am.commonlogin.config.xml	Contain configurations of AccessMatrix SSL certificate	UI customization General

Folder/File Name	Sub-folder	Description	Usage
		connection, Authentication Realms, etc.	Configuration Authentication Realm Configuration SAML Specific Configuration
	classes/clplog4j2.properties	Contain configurations of log level and log output file location.	General Configuration
	lib	Libraries used for CLP	-
	tld	JSP tag library	-
	wia	Contain the ASP file for Windows Integrated Authentication.	-
	web.xml	Contain filter mapping and servlet mapping.	-
	weblogic.xml	A configuration file to load all classes from WEB-INF of CLP application instead of loading classes from the system.	-
error.jsp	-	The page used to display the OIDC RP error message.	OIDC RP Error Page
iisredirect.jsp	-	Used for Windows Authentication login.	IIS Redirect Page
include.jsp	-	A common page that includes javascript references.	-
index.jsp	-	Refer to the CLP main jsp page.	CLP Main Page
logout.jsp	-	Used for CLP logout confirmation page.	CLP Logout Page
logoutCancelled.jsp	-	Used when a logout process is cancelled.	CLP Logout Page
logoutSuccess.jsp	-	Used when a logout process is successful.	CLP Logout Page

Folder/File Name	Sub-folder	Description	Usage
oautherror.jsp	-	Used to display errors thrown from OAuth use cases.	OAuth Error Page
op.html	-	Used for OAuth login.	-
redir.jsp	-	Used for OAuth login.	OAuth Redirect Page
samlredirect.jsp	-	Used for SAML login.	SAML Redirect Page
ssoredirect.jsp	-	Used for Kerberos login and OIDC RP login.	NTLM Redirect Page and OIDC RP Redirect Page

Customize the Page Background Image

To change the page background image of Common Login Page:

1. Navigate to `<CLP_HOME>/assets/css`.
2. Open the **branding.css** file.
3. Uncommenting the provided code for `.home-bg` class to customize page background image.


Note: Please remove the existing `.home-bg` class that applies an empty background.

CSS

```

.home-bg {
  background:
url("../assets/img/backgrounds/background.jpg");
  background-position: center;
  background-size: cover;
  height: 100%;
  display: flex;
  flex-direction: column;
}

```

4. Replace the **background.jpg** file with the new image file found on the same location (`<CLP_HOME>/assets/img/backgrounds`).
5. Save the changes.
6. Restart the CLP server for changes to take effect.

The new page background should take effect once the Common Login Page is launched.

Customize the Logo

1. Navigate to `<CLP_HOME>/assets/css`.
2. Open the **branding.css** file and edit the `#brand-logo` ID to customize CLP's logo.
3. Replace the **accessMatrixLogo.png** file with the new logo image found on the same location (`<CLP_HOME>/assets/logo`).
4. Save the changes.
5. Restart the CLP server for changes to take effect.

The new logo should take effect once the Common Login Page is launched.

Customize the Display Value of CLP Authentication Realm

To customize the display value of CLP authentication realms:

1. Navigate to `<CLP_HOME>/assets/i18n`.
2. Open and edit **message_en_US.json** file for English labels, **message_ja_JP.json** file for Japanese labels, and **message_zh_CN.json** for Chinese labels.
3. To add a new label, modify the file and add `"Lbl<name>": "<description>"`, e.g. `"Lbl_VascoOtp": "Vasco_OTP"`.
4. Save the changes.
5. To use this new label, navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
6. Look for the id of the authentication realm that you wanted to customize, e.g. VascoOTP.
7. Specify the value of the `idLabel`. For example:

XML

```
<AuthnRealm>
  <id>VascoOTP</id>
  <idLabel>Lbl_VascoOtp</idLabel>
  ...
</AuthnRealm>
```

8. Save the changes.
-

-
- Restart the CLP server for changes to take effect.

The new label will be used as the CLP authentication realm value on the login page.

Customize Texts/Labels

The description of the texts/labels in CLP can be customized. For example, the administrator can display a contact information on the error page.

To customize the description of texts/labels:

- Navigate to `<CLP_HOME>/assets/i18n`.
- Open and edit **message_en_US.json** file for English labels, **message_ja_JP.json** file for Japanese labels, and **message_zh_CN.json** for Chinese labels.
- Modify the description of an existing label, e.g. `Lbl_OAuthErrorDefault`.
- Save the changes.
- Restart the CLP server for changes to take effect.

Customize the Logout Page

To customize or use a different CLP logout page:

- Navigate to `<CLP_HOME>/WEB-INF` and open **web.xml**.
- Look for the `CommonLogout` element as shown below:

XML

```
<servlet>
  <servlet-name>CommonLogout</servlet-name>
  <servlet-
class>com.isprint.am.commonlogin.init.CommonLogout</servlet-
class>
  <init-param>
    <param-name>logoutConfirmPageUrl</param-name>
    <param-value>/logout.jsp</param-value>
  </init-param>
  <init-param>
    <param-name>logoutSuccessfulPageUrl</param-
name>
    <param-value>/logoutSuccess.jsp</param-value>
  </init-param>
  <init-param>
    <param-name>logoutCancelledPageUrl</param-name>
```

```
<param-value>/logoutCancelled.jsp</param-value>
</init-param>
</servlet>
```

3. By default, `logout.jsp` is configured as the `logoutConfirmPageUrl` value, `logoutSuccess.jsp` as the `logoutSuccessfulPageUrl`, and `logoutCancelled.jsp` as the `logoutCancelledPageUrl` value. Modify these URLs values to point to the customer's actual logout pages.
 - **logoutConfirmPageUrl** - this URL will display the logout confirmation page if the user has a valid session and visited `http://<CLP_Domain>:<port>/clp/logout`.
 - **logoutSuccessfulPageUrl** - this URL will display the logout success page if the logout process is successful.
 - **logoutCancelledPageUrl** - this URL will display the logout cancellation page if the user opted to cancel the logout process.
 4. Save the changes.
 5. Restart the CLP server for changes to take effect.
-

6. AccessMatrix CLP Authentication Methods

The following pages provide detailed information on the support AccessMatrix Common Login Page (CLP) authentication methods:

- [1FA](#)
- [2FA \(Email/SMS OTP\)](#)
- [2FA \(Token OTP\)](#)
- [Integrated Windows Authentication \(WIA\)](#)
- [OneSpan Scan and Login](#)
- [OneSpan Push to Login](#)
- [YESsafe Scan and Login](#)
- [SingPass Mobile](#)
- [FIDO2 Authentication](#)
- [External IdP Authentication via OIDC](#)
- [End-to-End Encryption Authentication \(E2EEA\)](#)
- [Certificate-based Authentication](#)



Note: 1FA also provides details on general configurations in the CLP config file that are applicable to various other authentication methods. Please refer to the [Authentication Realm Configurations](#) section for more information.

Depending on the authentication protocol, different authentication methods are supported. The following compatibility table provides an overview of this:

Authentication Method / Protocol	AccessMatrix Applications	SAML	OIDC	WSA
SystemPasswordBasic	Supported	Supported	Supported	Supported
2FA (Email/ SMS OTP)	Supported	Supported	Supported	Supported
2FA (Token OTP)	Supported	Supported	Supported	Supported
Integrated Windows Authentication (WIA)	Supported	Supported	Supported	-
YESsafe Scan and Login	Supported	Supported	Supported	Supported

Authentication Method / Protocol	AccessMatrix Applications	SAML	OIDC	WSA
OneSpan Scan & Login	Supported	Supported	Supported	Supported
OneSpan Push to Login	Supported	Supported	Supported	-
Singpass Mobile	Supported	Supported	Supported	Supported
FIDO2 Authentication	Supported	Supported	Supported	Supported
External IdP Authentication via OIDC	Supported	Supported	Supported	Supported
End-to-End Encryption Authentication (E2EEA)	Supported	Supported	Supported	Supported
Certificate-Based Authentication	Supported	Supported	Supported	-

6.1. 1FA (Password Authentication)

Configure the Login Module

The following steps will create a new login module. Alternatively, you can choose to use the pre-created login module "SystemPasswordBasic".

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select "System Password Basic" from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** (e.g. "PwdLM") and **Login Module Name** (e.g. "Password-based Login Module").
4. Configure the rest of the login module attributes, if necessary.
5. Save the changes.

Configure the Authentication Realm

The following steps will create a new authentication realm in Admin Console. Alternatively, you can choose to use the pre-created authentication realm "40151".

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane.
2. Click **Create** and provide the **Authentication Realm Id** (e.g. "PwdRealm") and **Authentication Realm Name** (e.g. "Password-based Realm").
3. Specify the **User Lookup Module** value that will be used to validate the user identity, e.g. "SystemLookup".
4. Specify the **First(1st) Login Module** that will be used for user authentication, e.g. "Password-based Login Module" (PwdLM).
5. Save the changes and assign this authentication realm to a user.

Configure the Authentication Realm in CLP

The following steps will add the newly created Authentication Realm to CLP:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML

```
<AuthnRealm>
  <id>PwdRealm</id>
  <idLabel>Lbl_PwdRealm</idLabel>

  <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

  <loginModuleIdForChangePassword>SystemPasswordBasic</loginModuleIdForChangePassword>
  <stateForChangePassword>login.chgpwd</stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>SystemPasswordBasic</handler>
    <loginState>login.start</loginState>
  </loginModule>

  <!--
  <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>--><!-- This is to define whether to use the AM5 session token
  lifetime expiry for cookie session expiry. Default:false -->
</AuthnRealm>
```



Note: Refer to the [Authentication Realm Configurations](#) section for the detailed information of the configuration above.

3. Add the idLabel value (Lbl_PwdRealm) into **message_en_US.json/message_zh_CN.json** to properly display the label of the authentication realm in CLP. Refer to the [Customize the Display Value of CLP Authentication Realm](#) section for the configuration steps.
4. Save the changes.
5. Restart the CLP server for changes to take effect.

Authentication Realm Configurations

This section provides detailed explanations for each of the available authentication realm configurations.

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.

-
2. Look for the AuthnRealms element. Under this element is the AuthnRealm element containing the authentication realm configurations. An example is shown below:

XML

```
<AuthnRealms>
  <defaultRealmId>40151</defaultRealmId>
  <AuthnRealm>
    <id>sysToken</id>
    <idLabel></idLabel>

    <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

    <loginModuleIdForChangePassword>SystemPasswordBasic</loginModuleIdForChangePassword>

    <stateForChangePassword>login.chgpwd</stateForChangePassword>
    <resetPassword>
      <userStoreId>DefaultStore</userStoreId>
      <loginModuleId>SystemPasswordBasic</loginModuleId>
      <!-- <forcePasswordChange>true</forcePasswordChange> --
    > <!-- User is required
      to change password the next time the user logs in after
      resetting password -->
    </resetPassword>
    <loginModule>
      <handler>SystemPasswordBasic</handler>
      <loginState>login.start</loginState>
      <externalLinks>
        <externalLink>
          <order>1</order>
          <url>http://google.com</url>
          <label>Lbl_ExLink_1</label>
        </externalLink>
        <externalLink>
          <order>2</order>
          <url>https://xx</url>
          <label>Lbl_ExLink_2</label>
        </externalLink>
      </externalLinks>
    </loginModule>

    <!--
    <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>--><!-- This is to define whether to use the AM5 session token
    lifetime expiry for cookie session expiry. Default:false -->
  </AuthnRealm>
</AuthnRealms>
```

Below is the list of elements and attributes for Authentication Realm configuration:

AuthnRealms - This refers to all the authentication realms that are displayed as authentication methods upon user login.

-
- **defaultRealmId** - The default realm Id to be used as a default authentication realm value for user authentication.

i **Note:** If only one AuthnRealm is configured, the authentication methods' drop-down box is not displayed. In this case, the defaultRealmId is set with the id value of the configured AuthnRealm.

AuthnRealm - This contains the configuration of an Authentication Realm as shown below:

- **id** - This refers to the Authentication Realm Id, e.g. 40151.

i **Note:** If idLabel value is empty, then CLP will use the id value to be displayed as authentication realm value on the login page.

- **idLabel** - This refers to the label used to represent the authentication realm, e.g. Lbl_40151. Refer to [Customise the Display Value of CLP Authentication Realm](#) section for the configuration steps.

i **Note:** The translated value will be shown in the drop-down list of the authentication realm selections.

- **firstLoginModuleNeedsChallenge** - Set this value to "true" if the first login module requires server to generate OTP (like SMS/Email OTP) and throw challenge (like Challenge-Response of token) before the user can enter the 1FA.
 - **loginModuleIdForChangePassword** - This refers to the login Module Id to be used for Change Password. To support change password, perform the steps described in [Enable Change Password](#) section to enable change password and in [Configure Login Module ID for Change Password](#) to configure the login module for change password.
 - **stateForChangePassword** - This refers to the state for change password, i.e. login.chpwd. Setting this state will display a page that requires user to enter existing password, new password and the confirmed new password.
 - **resetPassword** - This refers to the self-service password reset for password-based login module only. To support reset password, perform the steps described in [Configure Self-Service Password Reset](#).
 - **userStoreId** - This refers to the user store to lookup for user, such as DefaultStore, ActiveDirectory, and OpenLDAP.
 - **loginModuleId** - This refers to the login module that requires password reset.
 - **forcePasswordChange** - Set this value to "true" to require users to change their password when they first log in after resetting their password. By default, or if set to
-

"false", users are not forced to change their password when they first log in after resetting their password.

- **loginModule** - This refers to the login module of the realm.
 - **handler** - Handler refers to the Login Module Handler class of the login module. Below is a list of some login module handler classes supported for CLP:
 - SystemPasswordBasic
 - LdapLogin
 - RadiusLogin
 - SMSOTPLLogin
 - VascoTokenLogin
 - OATHTokenLogin
 - ChinaOTPTokenLogin
 - CompositeLogin
 - ResponseOnlyTokenLogin
 - MultiChoiceLogin
 - SocialNetworkLogin
 - TrustExternalLoginHandler
 - FIDO2TokenLogin
 - Push2LoginModule
 - PseudoE2EEALoginModule
 - SSLClientCertificateLogin
 - **loginState** - This refers to the login state value which determines the client UI page to display during login. Below is a list of some login states supported for CLP:
 - login.start
 - login.otp
 - login.firstloginmoduleotp
 - login.qrscan
 - login.fido2
 - externalOIDClogin.main
 - vmdpushlogin.main



Note: Refer to [Login Module Handler](#) section to locate the login module handler in AccessMatrix.

-
- login.cert

i Note: For more detail about the relationship between handler and loginState, please refer to [Login Modules and Corresponding States](#).

- **externalLinks** - This refers to the external links presented to the user on the login page. They can be used for a variety of functions, such as redirecting users to an FAQ page. Refer to the [Display External Links on Login Page](#) section for the configuration details.

i Note: This feature is only available for authentication methods that have the value "login.start" under the loginState field.

- **useSessionTokenLifetimeForCookie** - This setting determines whether the AccessMatrix session token lifetime is used for the CLP cookie's session. This can be used to ensure the CLP cookie is persisted after the CLP browser is closed (provided the session token lifetime is long enough). Session token lifetime as well as session token idle timeout should be set to the same value as CLP does not periodically refresh SAML IDP session of user after user authentication.
 - Uncomment and set this to "true" to use the AccessMatrix session token's lifetime for the CLP cookie's session.

i Notes:

- Currently, this feature is only supported for the SAML protocol.
- The AccessMatrix session token lifetime is based on the configurations made in AccessMatrix, either under the configurations in **Global Setting** or the configurations for the realm-based session policy used in the authentication realm. You should refer to [Deployment Configurations - Configure Session Token Cookie to Use AccessMatrix Session Token Lifetime](#) for the full details on how to properly make these configurations.

- Set this to "false" (or leave the setting commented out) for the default behavior, where the CLP cookie will be removed each time the browser is closed. This means that CLP will require the user to log in again when the browser is relaunched.

Locate Login Module Handler in AccessMatrix

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane. All available Login Modules are displayed on the **Search Login Module Result** page.
 2. Select a login module to view.
 3. The "Handler" of the selected login module can be found on the top-right end of the **View Login Module** page.
-

6.2. 2FA (Email/SMS OTP)

i **Note:** In this configuration, it is assumed that the 1st login module (password-based) is already created. Refer [here](#) for the creation steps.

Configure the Login Module

The following steps will create a new login module for 2nd factor authentication. Alternatively, you can choose to use the pre-created login module "SMSOTPLLogin".

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select "i-Sprint SMS OTP Login" from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** (e.g. "EmailLM") and **Login Module Name** (e.g. "Email Login Module").
4. Specify the **Event Notification Policy**, e.g. "EmailOTP".

i **Note:** Ensure that the event notification policy is already configured correctly. Refer to [UAS SMS OTP Implementation Guide](#) for steps on how to configure the message template, delivery channel and event notification policy.

5. Configure the rest of the login module attributes, if necessary.
6. Save the changes.

Configure the Authentication Realm

The following steps will create a new authentication realm.

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane.
 2. Click **Create** and provide the **Authentication Realm Id** (e.g. "2FARealm") and **Authentication Realm Name** (e.g. "Two FA Realm").
 3. Specify the **User Lookup Module** value that will be used to validate the user identity, e.g. "SystemLookup".
 4. Specify the **First(1st) Login Module** that will be used for user authentication, e.g. "Password-based Login Module (PwdLM)".
-

-
5. Specify the **Second(2nd) Login Module** that will be used for user authentication, e.g. "Email Login Module (EmailLM)".
 6. Save the changes and assign this authentication realm to a user.

Configure the Authentication Realm in CLP

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML

```
<AuthnRealm>
  <id>2FARealm</id>
  <idLabel>Lbl_2FARealm</idLabel>

  <firstLoginModuleNeedsChallenge>>false</firstLoginModuleNeedsChallenge>

  <loginModuleIdForChangePassword>SystemPasswordBasic</loginModuleIdForChangePassword>
  <stateForChangePassword>login.chgpwd</stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>SystemPasswordBasic</handler>
    <loginState>login.start</loginState>
  </loginModule>
  <loginModule>
    <handler>SMSOTPLogin</handler>
    <loginState>login.otp</loginState>
  </loginModule>

  <!--
  <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>
  --><!-- This is to define whether to use the AM5 session token
  lifetime expiry for cookie session expiry. Default:false -->
</AuthnRealm>
```

3. Refer to [Enable Authentication Realm in CLP](#) section for the detailed information of the configuration above.
 4. Add the idLabel value (Lbl2FARealm) into **message_enUS.json/message_zh_CN.json** to properly display the label of the authentication realm in CLP. Refer to [Customise the Display Value of CLP Authentication Realm](#) section for the configuration steps.
 5. Save the changes.
 6. Restart the CLP server for changes to take effect.
-

Configure Delay Timer for Resend OTP

For SMS OTP page, there is a **Resend OTP** button/link that will be displayed. This button will only be clickable after the configured delay timer (in seconds). The purpose of this is to prohibit people from pressing this button as there is normally a delay in sending OTP via SMS.

The default delay timer is 30 seconds. Specifying a value less than 1 means there's no delay. The **Resend OTP** button/link will be replaced with a countdown timer and this text: "You may request to resend OTP in xx seconds". When the timer reaches 0, the text will be changed to **Resend OTP** button/link.

To configure:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the `ResendOTP` element and configure the interval or delay timer as shown below.

Example:

XML

```
<ResendOTP>
  <intervalInSecs>30</intervalInSecs>
</ResendOTP>
```

3. Save the changes.
4. Restart the CLP server for changes to take effect.

6.3. Integrated Windows Authentication (WIA)

Integrated Windows Authentication (IWA, or WIA) allows users to log in to CLP automatically if they are logged in to a workstation with their Active Directory credentials. The users do not need to re-enter their credentials provided they are in the same domain and authorized to log in.

 **Note:** WIA is supported as an authentication method in the following scenarios:

- CLP for AccessMatrix Applications
- CLP as SAML Identity Provider
 -  *Flows initiated from SAML Identity Provider available from AccessMatrix eGA.*
- CLP as OIDC OpenID Provider

Ensure that the necessary configurations, as described under [AccessMatrix CLP Application Integrations](#), are made before proceeding with the following steps.

Microsoft Internet Information Services (IIS) supports this Windows authentication mechanism. In Windows Server 2012/2016/2019/2022, IIS is already part of the standard deployment. Therefore, Windows integrated authentication in IIS can be easily turned on. In this solution, an ASP file from the CLP package is deployed in IIS to make use of the IIS's IWA feature. When a user chooses Windows integrated authentication, CLP redirects the user to the ASP endpoint in IIS to proceed with automatic Windows integrated authentication. The ASP endpoint will then fetch the user ID, perform a lookup to the AccessMatrix server and retrieve the AccessMatrix server session before redirecting to CLP.

The following sections present the detailed instructions on how to integrate CLP with IIS to use Windows Authentication.

Prerequisites

A working domain controller and at least one client computer in that domain are required for this solution, as testing windows integrated authentication from the domain controller will not work.

The following checklist must be completed before doing any configuration:

- A functioning Microsoft Windows Server 2022 Active Directory Domain including:
 - Domain Controller
 - Client workstation
-

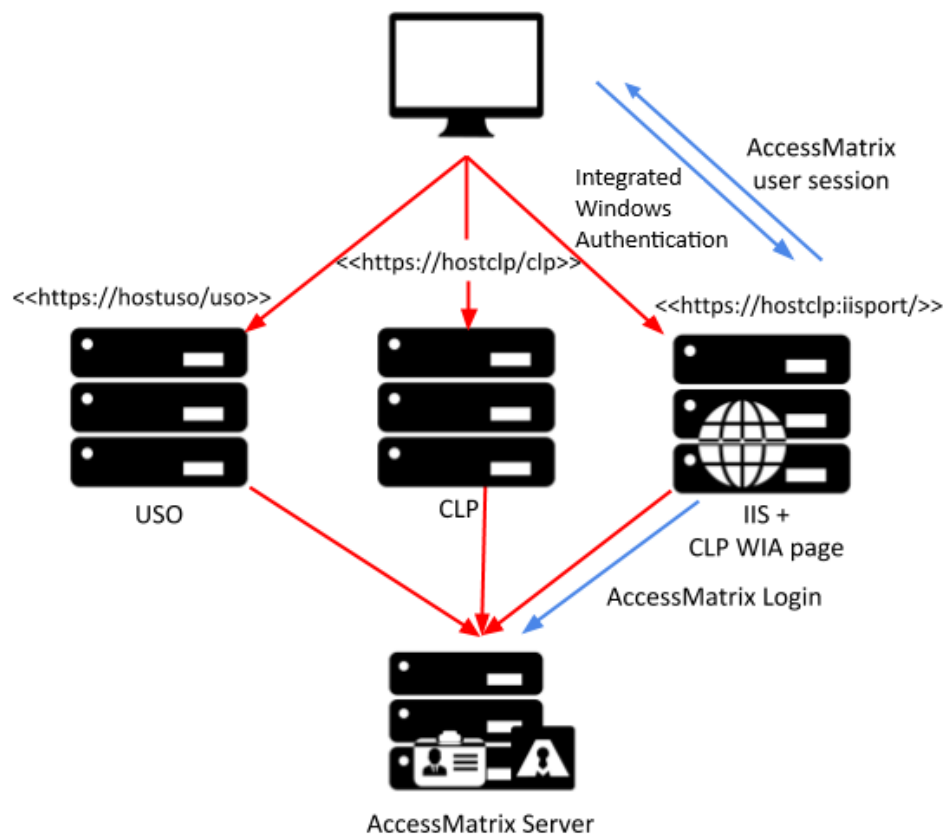
-
- Users that can log in to the client workstation
 - A Microsoft Windows Server 2022 with Web Server (IIS 10) installed and that is joined to Active Directory Domain. Ensure that the following has been installed in IIS:
 - Windows Authentication method
 - .Net Extensibility 4.X
 - ASP.NET 4.X
 - ISAPI Extension
 - ISAPI Filters
 - A Microsoft Windows 7/Windows 8/Windows 10 Client Workstation that is joined to Active Directory Domain.
 - Ensure that the client workstation is on the domain by checking the **Full computer name (FQDN)** and **Domain name** from System Properties (**Control Panel > System and Security > System**).
 - Ensure that the client workstation logs onto the domain and not locally.
 - IIS and CLP must be able to set and read the cookie on the same cookie domain.
 - Domain controller's DNS server is properly configured to have entries of Client workstation, IIS Server and CLP server running on a Windows server.
 - All hosts, except AccessMatrix server should be joined into the Active Directory Domain. Confirm this by doing a **PING** to the IIS server and CLP server using a fully qualified domain name (FQDN).
 - JDK 1.8 and above must be installed and used to run AccessMatrix server.
 - Confirm the version of the JRE and JDK installed by running each program (**java.exe** and **javac.exe**) with the **-version** flag.

Deployment

IIS and CLP can be deployed on the same machine or separately. There is no requirement that these two components must be on the same machine. If they are on the same machine, then they must be configured to listen to two different ports. If IIS and CLP are on different machines, then a reverse proxy can be deployed in front of IIS and CLP so that both can still use the same domain name.

It is very important to ensure that both CLP and IIS + CLP WIA page are deployed in HTTPS and not in the mix modes (e.g. CLP is deployed in HTTPS but IIS + CLP WIA page is deployed in HTTP - this will cause CLP fail to clean up authentication cookie of WIA which meant to be removed after user authentication and thus causing the unintended cookie left behind for undesirable SSO use).

USO and CLP can be deployed on the same web app server or a separate web app server. This is discussed in more details in [Deployment Alternatives](#).



Add Website or Application

To enable Windows authentication for a Web site, Web application, or Web service, you will need to:

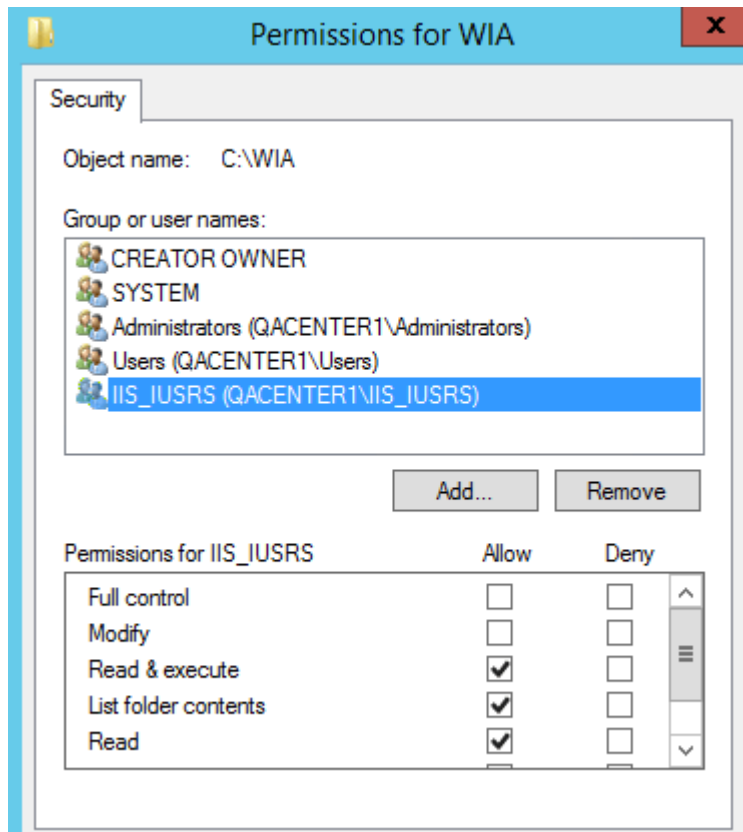
- [Configure IIS Manager](#)
- [Configure Internet Options](#)

Configure IIS Manager

Perform the following steps:

1. In the IIS server, create a local folder, e.g. `C:\WIA`.

i	Note: Ensure that the IIS user (IIS_IUSRS) has access to this folder by giving "Read and Execute" permissions.
----------	---



2. Open Internet Information Services (IIS) Manager.
3. In the **Connections** pane, expand the server name and right-click **Sites**.
4. Click **Add Website** and specify the **Site name** and **Physical Path**. The **Physical Path** should be pointing to the created folder, i.e. C:\WIA.



Note: A new application or a web service can also be added instead of adding a website.

-
5. Specify the **Port** to unused port number, e.g. "90".

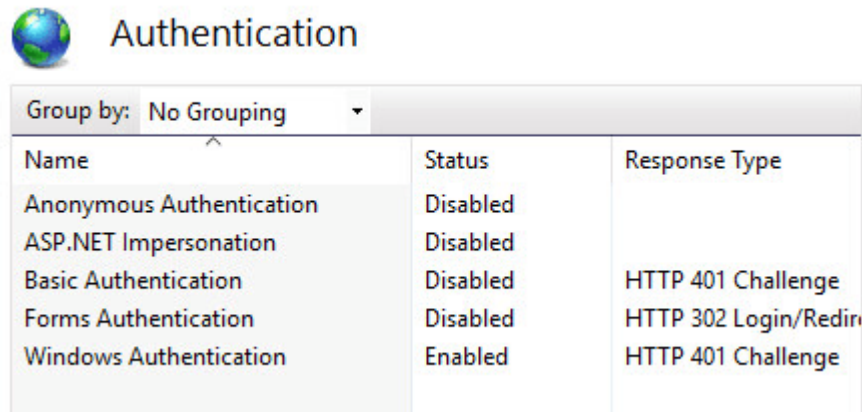
The screenshot shows the 'Add Website' dialog box. The 'Site name' field is 'MySite'. The 'Application pool' is 'MySite'. The 'Physical path' is 'C:\WIA'. The 'Port' is '90'. The 'Host name' field is empty. The example URL is 'www.contoso.com or marketing.contoso.com'.

6. In the same **Add Website** pane, specify the **Host name** for security purposes.

Note: Use https if configuring for a trusted site.

7. Click **OK** to continue the creation of the website.
 8. Click on the site, application or web service created.
 9. Under **Home** pane, scroll to **IIS** section and double click **Authentication**.
 10. Under **Authentication** pane, select **Windows Authentication** and click **Enable** in the **Actions** pane.
 11. In the same **Authentication** pane, click on **Advanced Settings** and uncheck **Enable Kernel-mode authentication**.
 12. In the same **Authentication** pane, click on **Providers**, select **Negotiate:Kerberos** in the drop-down list of **Available Providers**, and click the **Add** button.
-

-
13. In the same **Authentication** pane, select **Anonymous Authentication** and click **Disable** in **Actions** pane.

A screenshot of the 'Authentication' pane in a web management console. It features a globe icon and a title 'Authentication'. Below the title is a 'Group by' dropdown menu set to 'No Grouping'. A table lists five authentication methods: Anonymous Authentication, ASP.NET Impersonation, Basic Authentication, Forms Authentication, and Windows Authentication. Each method has a 'Status' column and a 'Response Type' column. Anonymous, ASP.NET Impersonation, Basic, and Forms Authentication are all 'Disabled', while Windows Authentication is 'Enabled'. The 'Response Type' for Basic and Forms Authentication is 'HTTP 401 Challenge', and for Windows Authentication, it is 'HTTP 302 Login/Redirect'.

14. Under **Actions** panel, click **Browse *:90 (http)** to check if the website is working.

Configure Internet Options

Perform the following steps:

1. Navigate to **Internet Options > Security** tab and click on either **Local intranet** or **Trusted sites**.
2. Click on the **Custom level...** button to display the **Security Settings** window.
3. Click on the **Automatic log-on with current username and password** radio button and click **OK**.

Configure Windows Integrated Authentication Handler File

Perform the following steps to configure the ASP file connection to AccessMatrix Server using the AccessMatrix REST API services.

1. In the AccessMatrix server, open the CLP's **wiahandler.asp** file from the following location in a text editor: `<CLP_HOME>/WEB-INF/wia/`
2. Update the `amRestURL` value with the actual AccessMatrix REST URL value. Example:

ASP.NET
<pre>amRestURL = "http://WIN2016-AM.win2016.com:8080/am5/rest"</pre>

-
3. Update the `amRealmId` value with the actual Integrated Windows Authentication realm value, e.g. 40171.

ASP.NET
<pre>amRealmId = "40171"</pre>

4. Update the `appURL` value with the actual CLP Kerberos Processor URL value. Example:

ASP.NET
<pre>appURL = "http://WIN2018-CLP.win2018.com:8080/clp/krb"</pre>

5. Update the `cookieDomain` value with the actual Domain value of the CLP server, e.g. WIN2018-CLP.win2018.com. Example:

ASP.NET
<pre>cookieDomain = "WIN2018-CLP.win2018.com"</pre>

6. Save the changes.
7. In the IIS server, copy the modified **wiahandler.asp** file to the previously created folder in IIS server, i.e. C:\WIA.

	Note: To secure the connection between WIA and AccessMatrix server, SSL client authentication is required. Refer to Secure the Connection to the AccessMatrix System section.
---	--

Configure IWA Authentication Realm

The Windows Integrated Authentication realm should be configured and added as one of the supported authentication realms of CLP. The 40171 authentication realm is created by default in Admin Console to support IWA , a suite of Microsoft security protocols intended to provide authentication, integrity, and confidentiality to users.

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
 2. Click on the 40171 authentication realm and make any changes to the configurations if necessary.
 3. Save the changes.
 4. Navigate to **Operation Dashboard > User & Segment > User**. Assign the realm to a user.
-

-
5. Save the changes.
 6. Navigate to <CLP_HOME>/WEB-INF/classes and open **am.commonlogin.config.xml**.
 7. Look for the AuthnRealms element and add the 40171 authentication realm configuration as shown below:

XML

```
<AuthnRealm>
  <id>40171</id>
  <idLabel>Kerberos</idLabel>

  <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

  <loginModuleIdForChangePassword></loginModuleIdForChangePassword>
  <stateForChangePassword></stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>TrustExternalLoginHandler</handler>
    <loginState>login.start</loginState>
  </loginModule>

  <!--
  <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>
  <!-- This is to define whether to use the AM5 session token
  lifetime expiry for cookie session expiry. Default:false -->
</AuthnRealm>
```

8. (Optional) Set 40171 as default realm Id for user selection.

XML

```
<defaultRealmId>40171</defaultRealmId>
```

9. Look for the sessionCookie element and update the domain parameter. This ensures that the session cookie domain of CLP has been set. For example:

XML

```
<SessionCookie>
  <domain>WIN2016-AM.win2016.com</domain> <!--
  Allow session cookie to set in this domain. If you
  using Internet Explorer, this value is required to be
  set. Default is empty-->
  <path></path> <!--Default is "/". Allow
  session cookie in all context path in current domain -->
  <SameSite></SameSite> <!-- Default is not set
```

```
for SameSite -->
    </SessionCookie>
```

10. Save the changes.

Enable CLP Support for Windows Integrated Authentication

1. Navigate to <CLP_HOME>/WEB-INF and open **web.xml**.
2. Look for the KerberosProcessor servlet name as shown below:

XML

```
<servlet>
    <servlet-name>KerberosProcessor</servlet-name>
    <servlet-
class>com.isprint.am.commonlogin.init.KerberosProcessor</servlet
-class>
    <init-param>
        <param-name>ntlmloginPageUrl</param-name>
        <param-value>/ssoredirect.jsp</param-value>
    </init-param>
    <init-param>
        <param-name>iisWIAEnabled</param-name>
        <param-value>>false</param-value>
    </init-param>
    <init-param>
        <param-name>iisWIARedirectPageUrl</param-name>
        <param-value>/iisredirect.jsp</param-value>
    </init-param>
    <init-param>
        <param-name>iisWIAPageUrl</param-name>
        <param-value>http://localhost/wiahandler.asp</param-
value>
    </init-param>
</servlet>
```

3. By default, the iisWIAEnabled parameter is configured as false. Set this parameter to true to enable Windows integrated authentication support.
 4. Modify the value of iisWIAPageUrl parameter to point to the actual IIS URL value where the **wiahandler.asp** is located.
 5. Save the changes.
 6. Restart the CLP server for changes to take effect.
-

Secure the Connection to the AccessMatrix System

Perform the following steps to set up the SSL client authentication between the WIA component and AccessMatrix server (mandatory for Production environment):

1. Convert the JKS keystore containing the client certificate (e.g. testagent.keystore) into PFX format (e.g. testagent.p12). Example command:

```
keytool -importkeystore -srckeystore testagent.keystore -destkeystore testagent.p12 -srcstoretype jks -deststoretype pkcs12
```
2. Right-click the PFX file (e.g. **testagent.p12**) and click **Install PFX** to import the certificate into the Windows certificate store.
3. Select "Local Machine" as the store location and follow the import steps in the Certificate Import Wizard.
4. Enter **certlm.msc** from the search bar. The Certificate Manager tool for the local device appears.
5. To view the imported certificate, under **Certificates - Local Computer** in the left pane, expand **Personal** and click **Certificates**.
6. Right-click **usoagent** and select **All Tasks > Manage Private Keys**.
7. Add **IIS_IUSRS** on testagent permission for private keys.
8. Open **wiahandler.asp** file and update the amRestURL value. Example:

Code

```
amRestURL = "https://WIN2016-AM.win2016.com:8443/am5/rest"
```

9. Then, add the code below into the restCall function:

objXML.setOption 3, "LOCAL_MACHINE\MY\usoagent" For example:

ASP.NET

```
Function restCall(url, body)
...
objXML.setOption 3, "LOCAL_MACHINE\MY\usoagent"
objXML.send "" & body & ""
```

For testing or troubleshooting, add the code below into the restCall function to ignore the server certificate-related error:

objXML.setOption 2, 13056 For example:

ASP.NET

```
Function restCall(url, body)
...
objXML.setOption 3, "LOCAL_MACHINE\MY\usoagent"
objXML.setOption 2, 13056
objXML.send "" & body & ""
```

Refer to [getOption](#) method.

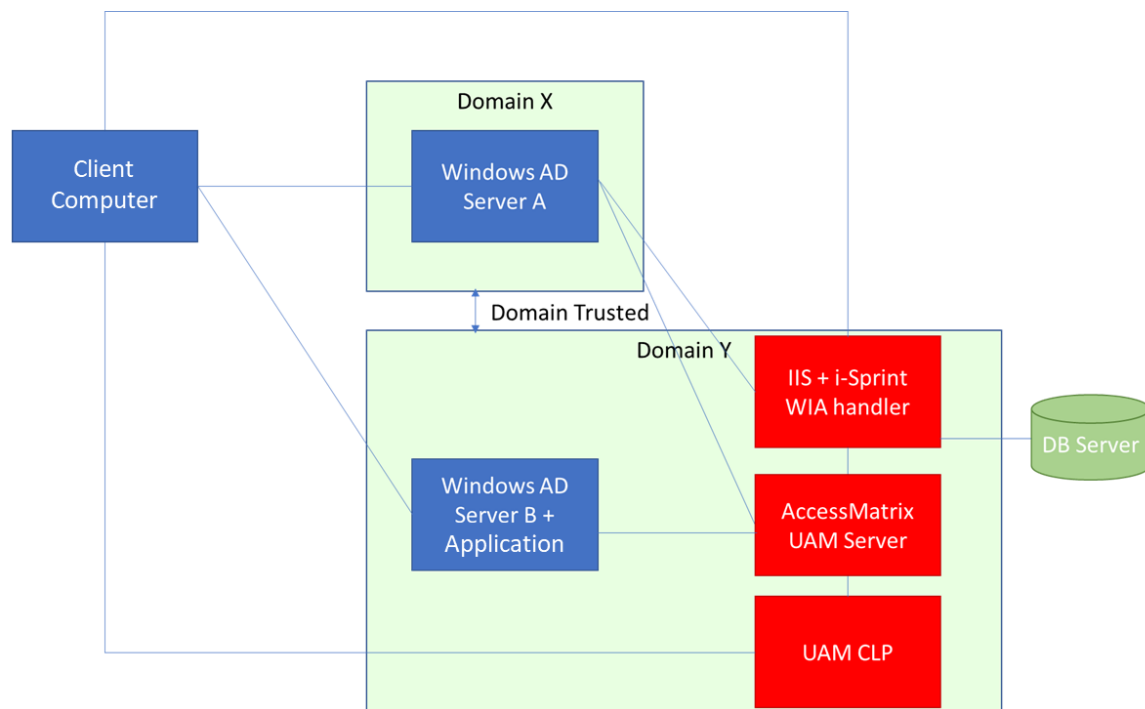
10. Save the changes made to the ASP file.
11. Check the **am.log** file. The log file will show the following information when WIA is using client-authentication SSL:

Code

```
2020-12-10 08:32:24.044 [https-jsse-nio-8443-exec-6]
DEBUG - (MethodLogger.java:22) REST:authn/login -
entry, realmId=40171, userId=app-sg-isi\chaeroddin
2020-12-10 08:32:24.073 [https-jsse-nio-8443-exec-6]
INFO - (LocalEventManager.java:191) User
chaeroddin/ActiveDire* (real actor chaeroddin) sesId
YknoYWI* triggered event Authentication-Login,
txId:null, result:S, msg=ForceChgPwd:
2020-12-10 08:32:24.081 [https-jsse-nio-8443-exec-6]
DEBUG - (MethodLogger.java:29) REST:authn/login - exit,
lapse(ms)=37
```

Configure WIA Trusted Domain Login with CLP

AccessMatrix CLP also supports trusted domain logins with WIA. In this scenario, AccessMatrix and CLP are deployed within Domain Y, an Active Directory Domain. Domain X, another Active Directory Domain without CLP, is configured as a trusted domain. A user within Domain X is able to access CLP in Domain Y using Windows Authentication. This is illustrated in the diagram below:

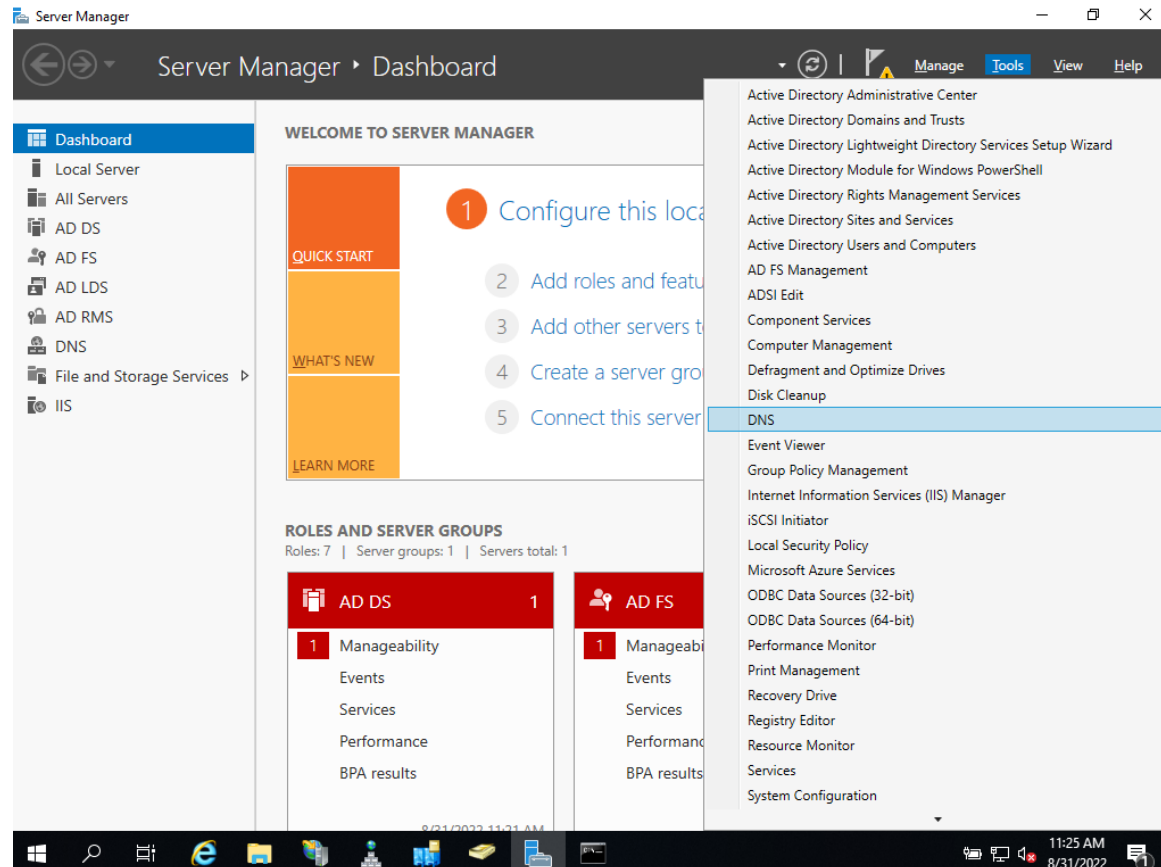


The following steps explain how to enable WIA trusted domain logins to CLP. This section assumes that you have prepared the Active Directory Domain with the AccessMatrix and CLP deployment as described in previous sections. It also assumes that you have set up the second Active Directory Domain without CLP and prepared a Client Workstation that is joined to this domain.

Add Conditional Forwarders

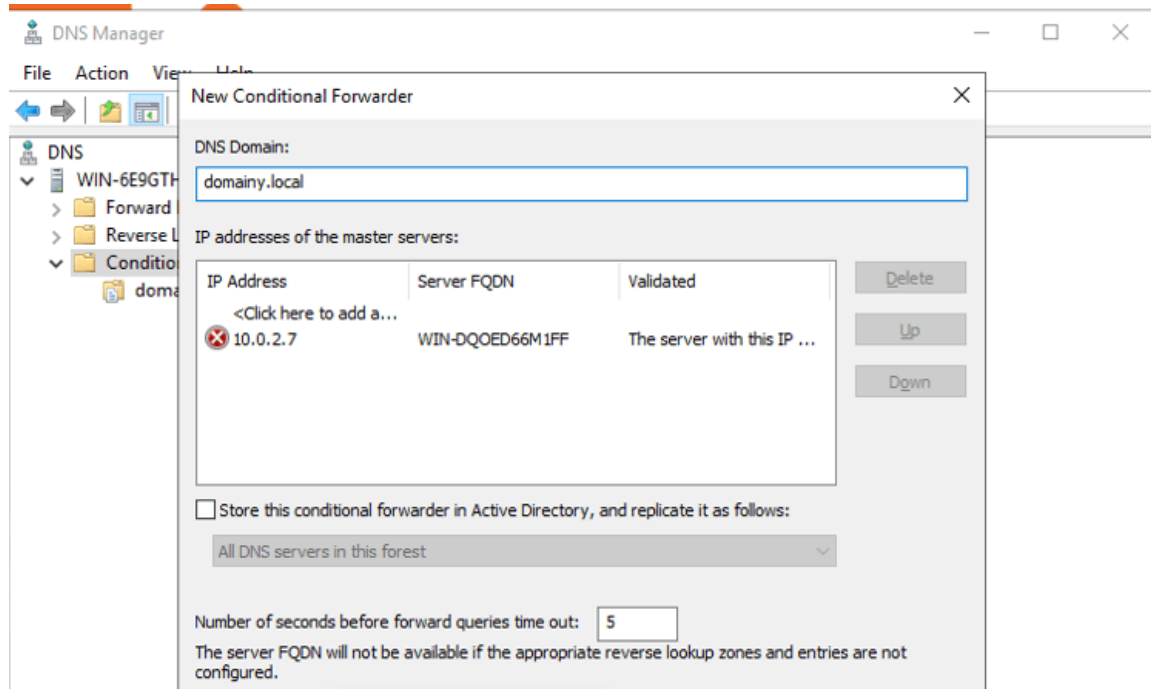
1. On Domain X, open the Server Manager application and click on **Tools > DNS** on the toolbar.

This opens up the DNS Manager interface.



2. On the DNS Manager, expand the options underneath your server name. Right-click the **Conditional Forwarders** folder and select **New Conditional Forwarder....**

3. Enter the domain name of Domain Y in the **DNS Domain** field and enter the IP Address of Domain Y in the box below. For example:



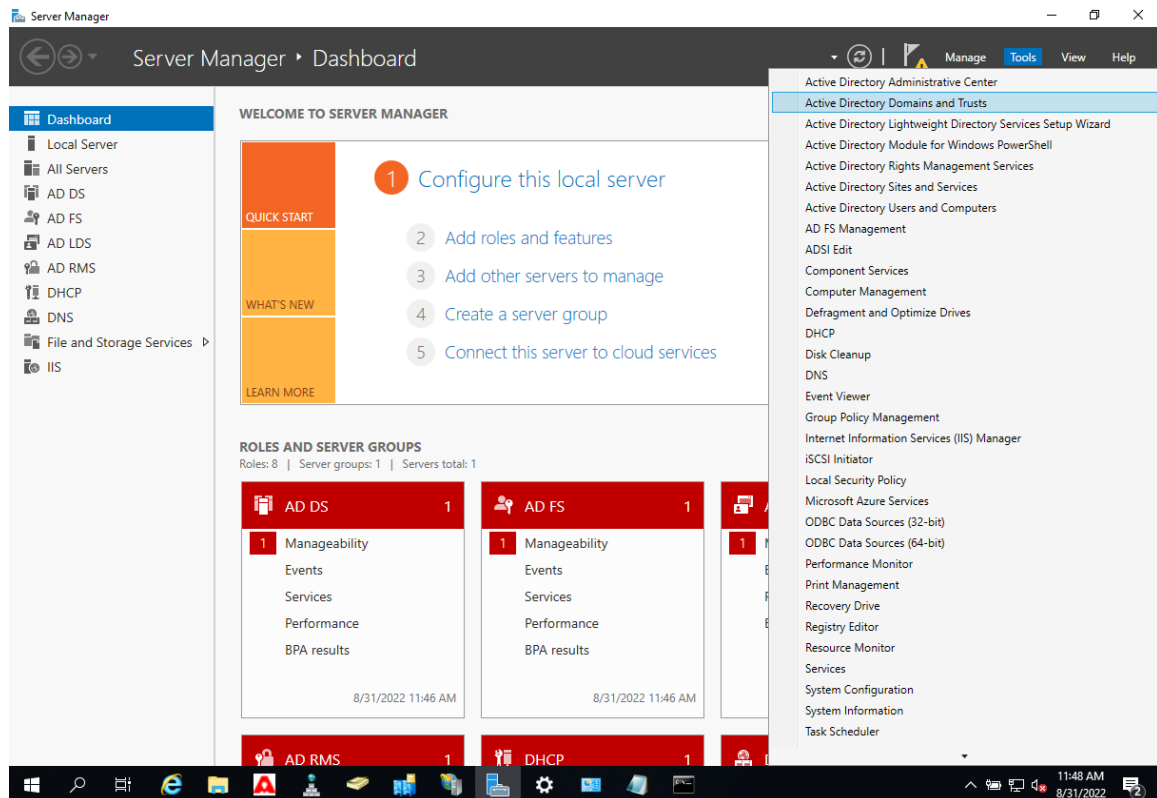
4. Click **OK** to save.
5. Perform a test by opening up the command prompt and entering `ping <DomainY_Name>`.
6. You should successfully receive a reply from the Domain Y server.

Note: If you are unable to receive a reply, ensure that the firewalls of the respective servers are not preventing the connection.

7. In Domain Y, repeat this process for Domain X.

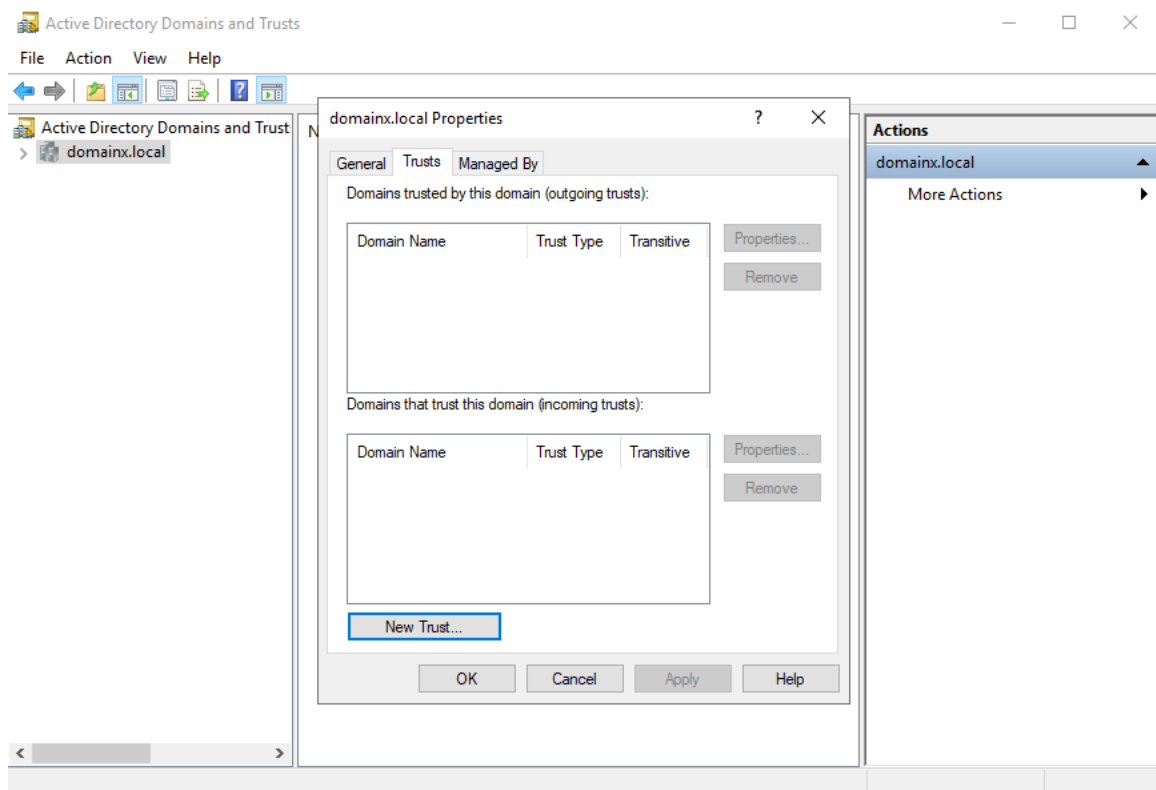
Configure Trusted Domain

1. In the Server Manager of Domain X, click on **Tools > Active Directory Domains and Trusts**. This opens the Active Directory Domains and Trusts interface.

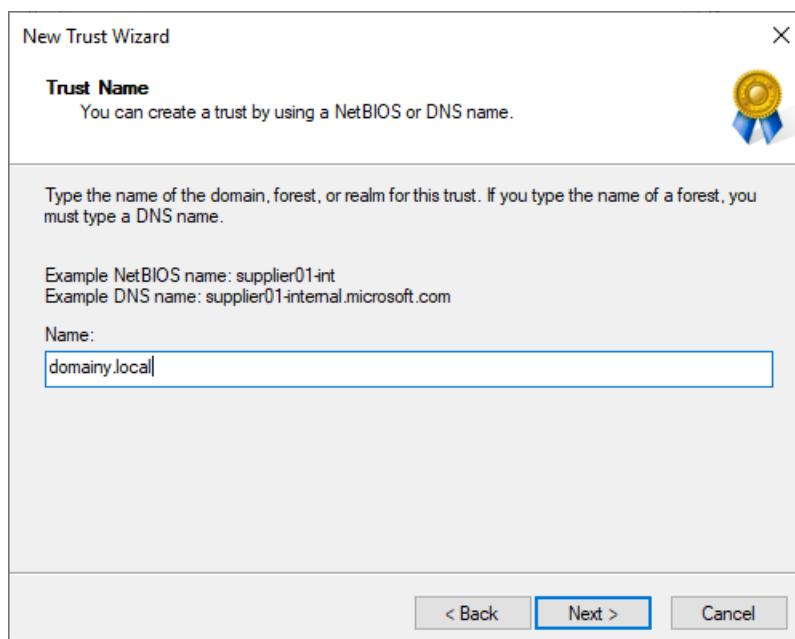


2. In the the Active Directory Domains and Trusts interface, right-click your domain name on the left pane and select **Properties**.

3. Navigate to the **Trusts** tab and click **New Trust....** This brings up the New Trust Wizard.



4. In the New Trust Wizard, enter the name of Domain Y (e.g. "domainy.local") under **Trust Name**.



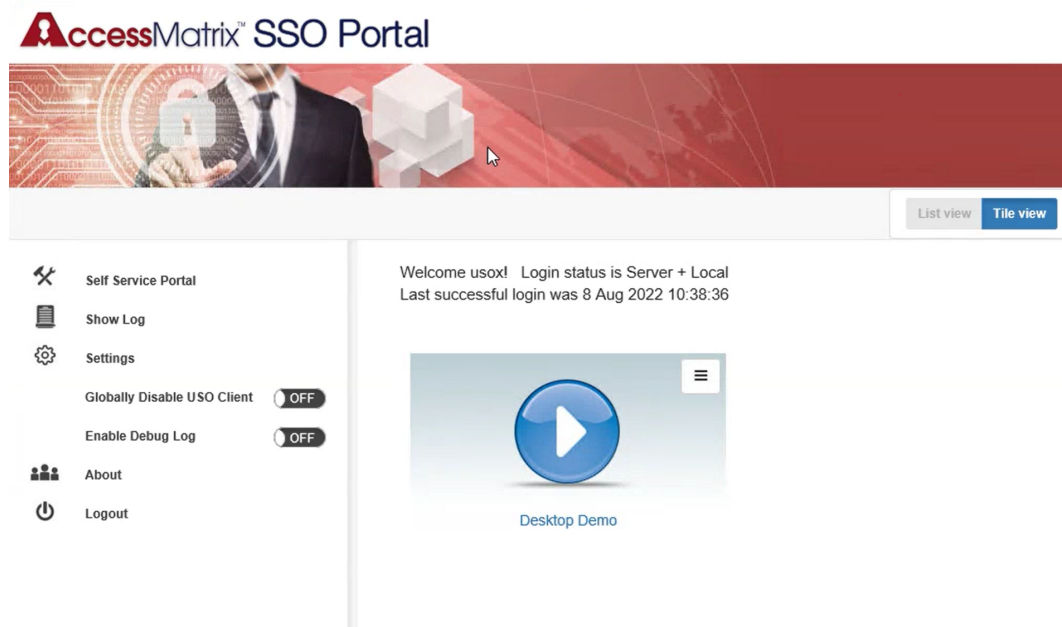
5. On the **Trust Type** page, choose **Forest trust**.
6. Follow the remaining steps in the New Trust Wizard, using the default settings on each page except:
7. On the **Confirm Outgoing Trust** page, select **Yes, confirm the outgoing trust**.

-
- Click **Finish** to complete the trust creation and click **OK** on the **Trusts** tab.
 - In Domain Y, repeat this process for Domain X.

Perform Test Login to CLP

If the configurations in the previous section were done correctly, the user should now be able to access web pages on Domain Y. Verify that CLP is now accessible from Domain X with the following steps:

- As a Domain X user, log into Domain X using the client workstation joined to Domain X.
- Access CLP via the appropriate application/website (depending on your deployment). The Windows Authentication process should take place and the user should be granted access:



FAQ & Troubleshooting

If you have followed the steps above and still encounter issues accessing CLP, ensure the following:

Configure Cookie Domain in USO server settings

Ensure that the cookie domain is configured in the USO server settings within the AccessMatrix Admin Console:

- Navigate to **Administration Dashboard > Configuration > Unified SSO > Enterprised SSO > USO Server Configuration**. The **View Configuration** page is displayed.
-

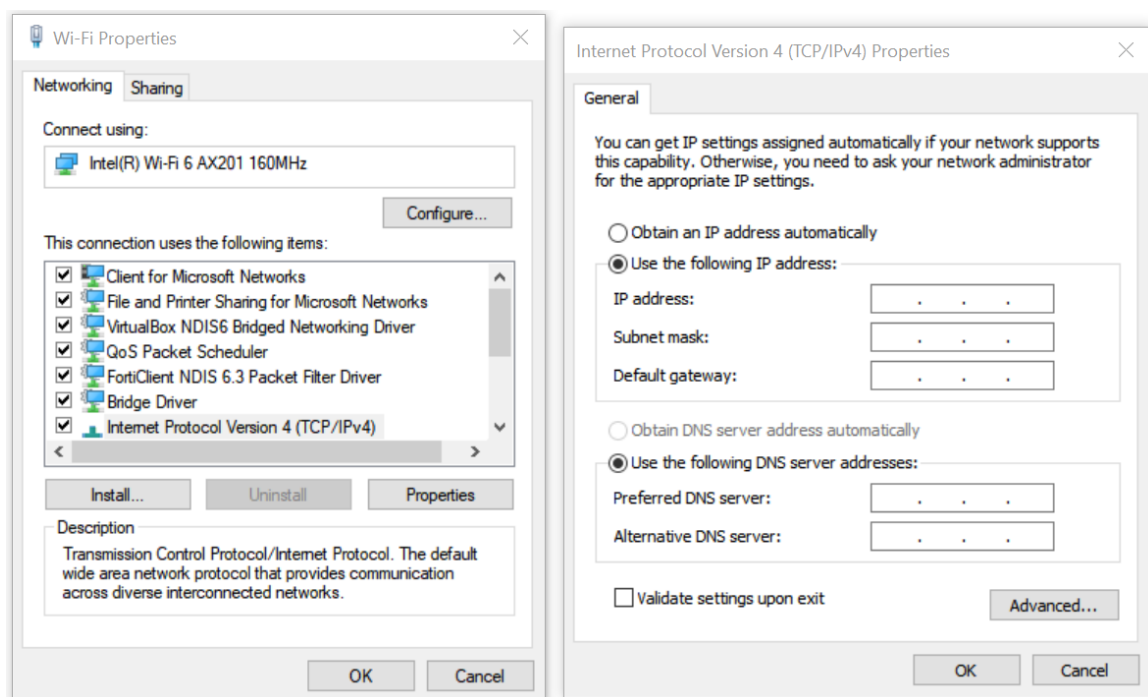
-
2. Under the **Cookie** header, ensure that the **Cookie.Domain** attribute is configured with the domain, e.g. "WIN2018-CLP.win2018.com".
 3. Save the changes.

If this attribute is not configured, after a user is successfully authenticated, they will be redirected to back to the CLP login page.

Use static IP address instead of a dynamic IP address

Ensure that the web server is configured to use a static IP address instead of a dynamic IP address:

1. On the search box of the Windows taskbar, type in "ncpa.cpl" and press Enter. This opens the Network Connections tool.
2. Right-click the network adapter currently in use and select **Properties**.
3. In the **Networking** tab, scroll down to the **Internet Protocol Version 4 (TCP/IPv4)** item and double-click it.



4. Select **Use the following IP address:** and manually enter the **IP address**, **Subnet mask** and **Default gateway** of your server. Retrieve these details by using the `ipconfig /all` command on the command prompt as necessary.
 5. Click **OK** on all the dialog boxes to save the changes.
-

-
6. Finally, open the command prompt and restart the Kerberos Key Distribution Center (KDC) and DNS Server. You can do so with the following commands:

```
net stop kdc
```

```
net stop dns
```

```
net start kdc
```

```
net start dns
```

6.4. 2FA (Token OTP)

i **Note:** In this configuration, it is assumed that the 1st login module (password-based) is already created. Refer [here](#) for the creation steps.

Configure the Login Module

The following steps will create a new login module for 2nd factor authentication.

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select the implementation based on the token OTP that will be used (e.g. "VascoTokenLogin", "OATHTokenLogin", etc.) from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** (e.g. "VascoLM") and **Login Module Name** (e.g. "Vasco Login Module").
4. Configure the rest of the login module attributes, if necessary.
5. Save the changes.

Configure the Authentication Realm

The following steps will create a new authentication realm.

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane.
 2. Click **Create** and provide the **Authentication Realm Id** (e.g. "Token2FA") and **Authentication Realm Name** (e.g. "Token 2FA Realm").
 3. Specify the **User Lookup Module** value that will be used to validate the user identity, e.g. "SystemLookup".
 4. Specify the **First(1st) Login Module** that will be used for user authentication, e.g. "Password-based Login Module (PwdLM)".
 5. Specify the **Second(2nd) Login Module** that will be used for user authentication, e.g. "Vasco Login Module (VascoLM)".
 6. Save the changes and assign this authentication realm to a user.
-

Note: Ensure that the token is already assigned to the user. (For OneSpan (Vasco) token, you can refer to [UAS OneSpan Token Implementation Guide](#) for the deployment steps and assignment of token to a user.)

Configure the Authentication Realm in CLP

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML

```
<AuthnRealm>
  <id>Token2FA</id>
  <idLabel>Lbl_Token2FA</idLabel>

  <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

  <loginModuleIdForChangePassword>SystemPasswordBasic</loginModuleIdForChangePassword>
  <stateForChangePassword>login.chgpwd</stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>SystemPasswordBasic</handler>
    <loginState>login.start</loginState>
  </loginModule>
  <loginModule>
    <handler>VascoTokenLogin</handler>
    <loginState>login.otp</loginState>
  </loginModule>

  <!--
  <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>
  --><!-- This is to define whether to use the AM5 session token
  lifetime expiry for cookie session expiry. Default:false -->
</AuthnRealm>
```

3. Refer to [Enable Authentication Realm](#) in CLP section for the detailed information of the configuration above.
 4. Add the idLabel value (Lbl_Token2FA) into **message_en_US.json/message_zh_CN.json** to properly display the label of the authentication realm in CLP. Refer to [Customise the Display Value of CLP Authentication Realm](#) section for the configuration steps.
 5. Save the changes.
 6. Restart the CLP server for changes to take effect.
-

6.5. OneSpan Scan and Login



Notes:

- The names OneSpan and VASCO are used interchangeably.
- Likewise, the names Mobile Authenticator Studio (MAS) and DIGIPASS for Mobile (DPMobile) refer to the same product.

The following sections present detailed instructions on how to integrate CLP with the OneSpan Mobile Authenticator Studio (MAS) to enable the Scan and Login authentication method. This authentication method will also identify the user based on the mobile token used for the authentication. Thus, it is suitable for a scan and login user experience where the user's identity is not yet known.



Note: OneSpan Scan and Login is supported as an authentication method in the following scenario only:

- CLP for AccessMatrix Applications
- CLP as SAML Identity Provider
- CLP as OIDC OpenID Provider
- CLP for WSA RP

Ensure that the necessary configurations, as described under [AccessMatrix CLP Application Integrations](#), are made before proceeding with the following steps.

How this OneSpan Scan and Login works is that CLP will make use of the Out-of-Band (OOB) session generated at the AccessMatrix Server. This OOB session will then be scanned by the mobile token so that the mobile token can generate the corresponding cryptographic response to CLP using the mobile channel (out-of-band). Upon successful verification, CLP web app will proceed to transform the OOB session to an authenticated AM session. This OOB session is generated with a random identifier, has a short expiry and has replay-attack prevention to increase the security of this authentication method. OneSpan Scan and Login uses challenge responses where the challenge is the OOB session identifier. For this reason, it is recommended that, when requesting for the token from OneSpan, the challenge length be long in the token parameter sheet.

Prerequisites and Preparation

In order to set up OneSpan scan and login, ensure that the following are ready:

- AccessMatrix Server deployed with OneSpan/Vasco Multi-Device libraries
 - AccessMatrix Server is configured with valid UAS license with OneSpan/Vasco Multi-Device token vendor
-

-
- DIGIPASS for Mobile token licenses (Token DPX file)
 - The OneSpan scan and login where the user identity is not yet known is using OneSpan mobile token's challenge-response for the authentication. For this reason, it is important to specify a long challenge length in the OneSpan parameters sheet when requesting for DIGIPASS for Mobile token licenses (DPX file).

Only activated DIGIPASS for Mobile token can be used for scan and login in CLP. DIGIPASS for Mobile token activation can be done using UAS Token Activation Portal (UAS TAP), refer to the UAS TAP documentation for its deployment and how to activate the token.

Configure OOB Lookup Module

To create an Out-of-Band (OOB) Lookup Module, navigate to **Administration Dashboard > Configuration > Authentication** and click **User Lookup Module** on the left pane. Click the **Create** button and select "Out-of-Band Lookup Module" implementation from the **Choose User Lookup Module Implementation** page. Then, click **Next** and provide the necessary values for the lookup module attributes before saving the changes.

Lookup Module Attribute Names	
*User Lookup Module Id	Example: OOBLookup
*User Lookup Module Name	Example: OOB Lookup
Attributes Tab	
Challenge Image Type	Example: VascoMultiDevice
OOB Session Id Length	Example: 10
OOB Token Vendor	Example: VASCO_MULTI_DEVICE

Configure Login Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
 2. Click **Create** and select the "Social Network Login implementation" from the **Choose Login Module Implementation** page.
 3. Enter a **Login Module Id** (e.g. "OOBLogin") and a **Login Module Name** (e.g. "OOB Login").
 4. Select the **Login Policy** and the **Authentication Level** values, if necessary.
-

-
5. Click **Save**. A new login module is created.

Configure Authentication Realm

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
 2. Click **Create**.
 3. Enter an **Authentication Realm Id** (e.g. "vmdscan") and an **Authentication Realm Name** (e.g. "OneSpan Scan and Login").
 4. Specify the **User Lookup Module** value that will be used to validate user identity. This should be the OOB Lookup Module created in earlier step, e.g. "OOB Lookup (OOBLookup)".
 5. Specify the **First(1st) Login Module** that will be used for user authentication. This should be the Social Network Login Module created in earlier step, e.g. "OOB Login (OOBLogin)".
 6. Click **Save**. A new authentication realm is created.
 7. Assign this realm to a user.
-

6.6. OneSpan Push to Login



Notes:

- The names OneSpan and VASCO are used interchangeably.
- Likewise, the names Mobile Authenticator Studio (MAS) and DIGIPASS for Mobile (DPMobile) refer to the same product.

The following sections present detailed instructions on how to integrate CLP with the OneSpan Mobile Authenticator Studio (MAS) to enable the Push to Login authentication method.



Note: OneSpan Push to Login is supported as an authentication method in the following scenarios:

- CLP for AccessMatrix Applications
- CLP as SAML Identity Provider
- CLP as OIDC OpenID Provider

Ensure that the necessary configurations, as described under [AccessMatrix CLP Application Integrations](#), are made before proceeding with the following steps.

Prerequisites

Before configuring OneSpan Push to Login, it is assumed that the following OneSpan-related prerequisites are met:

Prerequisite	Reference
The AccessMatrix server has a valid UAS license with the OneSpan/Vasco Multi-Device token vendor enabled.	-
The AccessMatrix server has been deployed with OneSpan/Vasco Multi-Device libraries.	Refer to the UAS OneSpan Multi-Device Implementation Guide .
The DPMobile Gateway has been deployed and configured.	Refer to the UAS OneSpan DPMobileGateway Implementation Guide .
The MAS mobile app has been generated and is working.	Refer to the OneSpan DIGIPASS for Mobile/ Mobile Authenticator Studio documentation.
Ensure a VASCO Multi-device token has been assigned to each of the relevant users and is activated.	The token activation can be done using the UAS Token Activation Portal (UAS TAP). Refer to UAS

Prerequisite	Reference
	Token Activation Portal Deployment Guide - VASCO Multi-Device Token Deployment.

Configure Login Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
2. Click **Create** and select the "Push to Login Module" from the **Choose Login Module Implementation** page.
3. Enter a **Login Module Id** (e.g. "P2Login") and a **Login Module Name** (e.g. "Push To Login").
4. Select an appropriate **Login Policy** and **Authentication Level** for the login module.
5. Configure the following attributes:
 - a. **Token Vendor** - Select "VASCO_MULTI_DEVICE".
 - b. **Notification Action Identifier** - Ensure that the value matches the action identifier setting in the **DIGIPASS.xml** file used to generate the MAS mobile application. To locate the value, search for the TransactionDataSigningAction id setting in the **DIGIPASS.xml** file.
 - c. **Version Tag** - Ensure that the value is set to "01".
6. Configure the remaining attributes, if necessary.
7. Click **Save**. A new login module is created.



Note: Refer to [AccessMatrix Common Administration Guide - Push to Login Module](#) for detailed information on the login module attributes.

Configure Authentication Realm

The following steps will create a new authentication realm.

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane.
2. Click **Create** and provide the **Authentication Realm Id** (e.g. "pushrlm") and **Authentication Realm Name** (e.g. "Push To Login").
3. Specify the **User Lookup Module** value that will be used to validate the user identity, e.g. "SystemLookup".

4. Specify the **First(1st) Login Module** that will be used for user authentication, e.g. "SystemPasswordBasic".
5. Specify the **Second(2nd) Login Module** that will be used for user authentication, e.g. "P2Login".
6. Configure the remaining attributes, if necessary.
7. Save the changes and assign this authentication realm to a user.



Note: Ensure that the same user has been assigned a VASCO Multi-Device token.

Configure Authentication Realm in CLP Config File

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the `AuthnRealms` element and add the `pushrlm` authentication realm, as shown below:

XML

```
<AuthnRealms>
...
  <AuthnRealm>
    <id>pushrlm</id>
    <idLabel>Push To Login</idLabel>

    <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

    <loginModuleIdForChangePassword>SystemPasswordBasic</loginModuleIdForChangePassword>

    <stateForChangePassword>login.chgpwd</stateForChangePassword>
    <resetPassword>
      <userStoreId></userStoreId>
      <loginModuleId></loginModuleId>
    </resetPassword>
    <loginModule>
      <handler>SystemPasswordBasic</handler>
      <loginState>login.start</loginState>
    </loginModule>
    <loginModule>
      <handler>Push2LoginModule</handler>
      <loginState>vmdplogin.main</loginState>
    </loginModule>

    <!--
    <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>
    --><!-- This is to define whether to use the AM5 session token
    lifetime expiry for cookie session expiry. Default:false -->
    </AuthnRealm>
  ...
</AuthnRealms>
```

-
- a. Ensure that the value of id matches the Id of the authentication realm configured in the previous section (e.g. "pushrlm").
 - b. Ensure the values of the attributes under each loginModule code block correspond to the login modules configured for 1FA and 2FA (e.g. "SystemPasswordBasic" and "Push2LoginModule" respectively).
3. Save the changes.
 4. Restart the CLP server for the changes to take effect.

Configure Push to Login OOB Polling Servlet in web.xml

1. Navigate to <CLP_HOME>/WEB-INF/ and open **web.xml**.
2. Add the Push2LoginOOBPollingServlet code block within the file, as follows:

XML

```
<servlet>
  <servlet-name>Push2LoginOOBPollingServlet</servlet-name>
  <servlet-
class>com.isprint.am.commonlogin.server.servlet.Push2LoginOOBPollin
gServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>Push2LoginOOBPollingServlet</servlet-name>
  <url-pattern>/push2loginOOB/check</url-pattern>
</servlet-mapping>
```

3. Save the changes.
-

6.7. YESsafe Scan and Login

The following sections present detailed instructions on how to integrate CLP with the YESsafe mobile application to enable the Scan and Login authentication method. This authentication method will also identify the user based on the mobile token used for the authentication. Thus, it is suitable for a scan and login user experience where the user's identity is not yet known.



Note: YESsafe Scan and Login is supported as an authentication method in the following scenario only:

- CLP for AccessMatrix Applications
- CLP as SAML Identity Provider
- CLP as OIDC OpenID Provider
- CLP for WSA RP

Ensure that the necessary configurations, as described under [AccessMatrix CLP Application Integrations](#), are made before proceeding with the following steps.

How this YESsafe Scan and Login works is that CLP will make use of the Out-of-Band (OOB) session generated at the AccessMatrix Server. This OOB session will then be scanned by the mobile token so that the mobile token application can generate an OTP and send it to CLP (via CLP endpoint: <CLP_URL>/oob/update) using the mobile channel (out-of-band) to be updated to the cache of AM5 backend for further verification. Subsequently, the CLP web app will perform OTP verification and return an authenticated AM session. This OOB session is generated with a random identifier, has a short expiry and has replay-attack prevention to increase the security of this authentication method.



Note: Ensure <CLP_URL>/oob/update & <CLP_URL>/oob/check is configured in CLP web.xml. <CLP_URL>/oob/update has to be internet accessible.

Prerequisites and Preparation

In order to set up YESsafe scan and login, ensure that the following are ready:

- AccessMatrix Server is configured with valid UAS license with OATH token vendor
- Token+ app has configured **Setting url** - https://:<UASTAP host:port>/uastap

Only activated YESsafe token can be used for scan and login in CLP. YESsafe token activation can be done using UAS Token Activation Portal (UAS TAP), refer to the UAS TAP documentation for its deployment and how to activate the token.

Configure OOB Lookup Module

Note: The OOB Lookup Module "YESsafeOOBLookup" is created by default during server initialization and can be used for this authentication method.

To create an Out-of-Band (OOB) Lookup Module, navigate to **Administration Dashboard > Configuration > Authentication** and click **User Lookup Module** on the left pane. Click the **Create** button and select "Out-of-Band Lookup Module" implementation from the **Choose User Lookup Module Implementation** page. Then, click **Next** and provide the necessary values for the lookup module attributes before saving the changes.

Lookup Module Attribute Names	
*User Lookup Module Id	Example: YESsafeLookup
*User Lookup Module Name	Example: YESsafeLookup
Attributes Tab	
Challenge Image Type	Example: YESsafe
OOB Session Id Length	Example: 10
OOB Session Expiry(in seconds)	Example: 90
OOB Token Vendor	Example: OATH

Configure Login Module

Note: The OATH Token Login Module "DefaultOATHTokenLogin" is created by default during server initialization and can be used for this authentication method.

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
 2. Click **Create** and select the "OATH Token Login implementation" from the **Choose Login Module Implementation** page.
 3. Enter a **Login Module Id** (e.g. "YesLogin") and a **Login Module Name** (e.g. "YESsafe Login").
 4. Set **Token Vendor** to "OATH".
 5. Enter appropriate **Mobile Callback URL** to call CLP oob/update endpoint:
<CLP_URL>/oob/update (e.g. "https://clp.i-sprint.com/clp/oob/update").
-

-
- a. If left empty, "<CLP_URL>/oob/update" is used as default.
 6. Select the **Login Policy** and the **Authentication Level** values, if necessary.
 7. Click **Save**. A new login module is created.

Configure Authentication Realm

Note: The Authentication Realm "DefaultYESsafeScanToLoginRealm" is created by default during server initialization and can be used for this authentication method.

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
2. Click **Create**.
3. Enter an **Authentication Realm Id** (e.g. "yessafe") and an **Authentication Realm Name** (e.g. "YESsafe Scan and Login").
4. Specify the **User Lookup Module** value that will be used to validate user identity. This should be the OOB Lookup Module created in earlier step, e.g. "YESsafeLookup ('YESsafeLookup')".
5. Specify the **First(1st) Login Module** that will be used for user authentication. This should be the OATH Token Login Module created in earlier step, e.g. "YESsafe Login ('YesLogin')".
6. Click **Save**. A new authentication realm is created.
7. Assign this realm to a user.

Configure the Authentication Realm in CLP

Note: The example shown below is created by default during server initialization and can be used for this authentication method.

1. Navigate to <CLP_HOME>/WEB-INF/classes and open **am.commonlogin.config.xml**.
2. Look for the AuthnRealms element and configure the YESsafe QR login authentication realm as shown below:

XML

```
<AuthnRealm>
  <id>DefaultYESsafeScanToLoginRealm</id> <!-- for YESsafe Scan and
Login-->
  <idLabel></idLabel>
```

```

<firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>
  <loginModuleIdForChangePassword></loginModuleIdForChangePassword>
  <stateForChangePassword></stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>OATHTokenLogin</handler>
    <loginState>login.qrscan</loginState>
  </loginModule>
  <!--
  <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>
  <!-- This is to define whether to use the AM5 session token
  lifetime expiry for cookie session expiry. Default:false -->
  </AuthnRealm>

```



3. Save the changes.
4. Restart the CLP server for changes to take effect.

Configure YESsafe OOB Servlet in web.xml

1. Navigate to <CLP_HOME>/WEB-INF/ and open **web.xml**.
2. Ensure the following code block is within the file, add it if not:

XML

```

<servlet>
  <servlet-name>OOBVerifyUpdateServlet</servlet-name>
  <servlet-
class>com.isprint.am.commonlogin.server.servlet.OOBVerifyUpdateServ
let</servlet-class>
</servlet>
<servlet>
  <servlet-name>OOBPollingServlet</servlet-name>
  <servlet-
class>com.isprint.am.commonlogin.server.servlet.OOBPollingServlet</
servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>OOBVerifyUpdateServlet</servlet-name>
  <url-pattern>/oob/update/*</url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name>OOBPollingServlet</servlet-name>

```

```
<url-pattern>/oob/check</url-pattern>  
</servlet-mapping>
```

3. Save the changes if insertion was made.

6.8. SingPass Mobile

The following sections present the detailed instructions on how to integrate CLP with SingPass Mobile.

Prerequisites and Preparation

In order to set up SingPass Mobile, ensure that the following are ready:

- SingPass Mobile OnBoarding Form that is submitted to SingPass
- SingPass Environment Information that is provided by SingPass
- (Optional) JWT Encryption Keystore (AccessMatrix requires the private key in Java Keystore) and its related information (passphrase and key alias)
- Outgoing connection from the AccessMatrix Server to your OP is allowed
- Ensure that the OIDCRP license feature is added under **UAM FEATURE - Social Network Login** to enable OIDC RP support and to display "OIDC Relying Party Lookup Module" in the lookup module implementation list.

Configure OIDC Relying Party (RP) Lookup Module

To create an OIDC Relying Party Lookup Module, navigate to **Administration Dashboard > Configuration > Authentication** and click **User Lookup Module** on the left pane. Click the **Create** button and select "OIDC Relying Party Lookup Module" implementation from the **Choose User Lookup Module Implementation** page. Then, click **Next** and provide the necessary values for the lookup module attributes before saving the changes, based on onboarding form provided to SingPass and environment specification provided by SingPass. Refer to [UAM OIDC Relying Party Implementation Guide - Configure Lookup Module](#) for the description of the lookup module attributes.



Note: To support client assertion JWT, refer to [UAM OIDC Relying Party Implementation Guide - Enable Client Authentication Using Client Assertion JWT](#) for details.

Attribute Name	Staging Environment	Production Environment
*User Lookup Module Id	Example: sgspstageLookup	Example: sgspprodLookup
*User Lookup Module Name	Example: SingPass Mobile - Staging	Example: SingPass Mobile - Prod

Attribute Name	Staging Environment	Production Environment
Attributes Tab		
*Client Id	<Staging_Client_ID>	<Production_Client_ID>
*Client Secret	<Staging_Client_Secret>	<Production_Client_Secret>
OpenID Provider		
OpenID Provider Configuration URL	https://stg-id.singpass.gov.sg/.well-known/openid-configuration	https://id.singpass.gov.sg/.well-known/openid-configuration
Token Endpoint	https://stg-id.singpass.gov.sg:9443/token Note: Only specify the value if "ClientSecret" is the token endpoint authentication method.	https://id.singpass.gov.sg:9443/token Note: Only specify the value if "ClientSecret" is the token endpoint authentication method.
Default Values of Authentication Request for Authorization Endpoint		
Redirection URI	<Staging_Callback_URL>	<Production_Callback_URL>
JWE Tab		
ID Token is JWE Encrypted	true	true
Key Pair in Keystore (Legacy)		
Keystore	<Staging_Encryption_Keystore>	<Production_Encryption_Keystore>
Keystore Passphrase	<Keystore_Passphrase>	<Keystore_Passphrase>
Key Alias	<Key_Alias_in_Keystore>	<Key_Alias_in_Keystore>
OIDC RP Servlet Tab		
Landing Page URL	<CLP_URL>/extoidcidp Example: http://clp-eng.i-sprint.com:8083/clp/extoidcidp	<CLP_URL>/extoidcidp Example: https://clp.i-sprint.com/clp/extoidcidp
Error Page URL	<CLP_URL>/error.jsp Example: http://clp-eng.i-sprint.com:8083/clp/error.jsp	<CLP_URL>/error.jsp Example: https://clp.i-sprint.com/clp/error.jsp

Configure Login Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
2. Click **Create** and select the "Social Network Login implementation" from the **Choose Login Module Implementation** page.
3. Enter a **Login Module Id** (e.g. "SocNetLM") and a **Login Module Name** (e.g. "Social Network Login Module").
4. Select the **Login Policy** and the **Authentication Level** values, if necessary.
5. Click **Save**. A new login module is created.

Configure Authentication Realm

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
2. Click **Create**.
3. Enter an **Authentication Realm Id** (e.g. "sgspstag") and an **Authentication Realm Name** (e.g. "Singpass Staging Realm").
4. Specify the **User Lookup Module** value that will be used to validate user identity (e.g. "SingPass Mobile - Staging (sgspstageLookup)").
5. Specify the **First(1st) Login Module** that will be used for user authentication (e.g. "Social Network Login Module (SocNetLM)").
6. Click **Save**. A new authentication realm is created.
7. Assign this realm to a user.

Configure OAuth RP Servlet Settings

1. Navigate to `<CLP_HOME>/WEB-INF` and open **web.xml**.
2. Look for the OAuth RP Servlet Configuration comment as shown below:

XML

```
...
    <!-- Setting used for OAuth RP Servlet Configuration-->
    <context-param>
        <param-name>stateCookieTimeOutInSec</param-name>
        <param-value>180</param-value>
    </context-param>
```

```

        <context-param>
            <param-name>errorPageUrl</param-name>
            <param-value>/clp/error.jsp</param-value>
        </context-param>
        <!-- End setting used for OAuth RP Servlet
        Configuration-->
        ...
        <!-- OAuth RP Servlet Configuration -->
        <servlet>
            <servlet-name>OAuthRPInitServlet</servlet-name>
            <servlet-
class>com.isprint.oauth.rp.OAuthRPInitServlet</servlet-
class>
            </servlet>
            <servlet>
                <servlet-name>OAuthRPCallBackServlet</servlet-
name>
                <servlet-
class>com.isprint.oauth.rp.OAuthRPCallBackServlet</servlet-
class>
                </servlet>
            <servlet>
                <servlet-name>OAuthRPJwksServlet</servlet-name>
                <servlet-
class>com.isprint.oauth.rp.OAuthRPJwksServlet</servlet-
class>
                </servlet>
            <servlet-mapping>
                <servlet-name>OAuthRPInitServlet</servlet-name>
                <url-pattern>/rp/init/sgspstag</url-pattern>
                <url-pattern>/rp/rest/init</url-pattern>
            </servlet-mapping>
            <servlet-mapping>
                <servlet-name>OAuthRPCallBackServlet</servlet-
name>
                <url-pattern>/rp/cb/sgspstag</url-pattern>
            </servlet-mapping>
            <servlet-mapping>
                <servlet-name>OAuthRPJwksServlet</servlet-name>
                <!--Servlet path "<realmId>" followed by "/.well-
known/jwks.json". Configure the realmId where the lookup
module is an OIDC Relying Party Lookup Module handler -->
                <url-pattern>/sgspstag/.well-known/jwks.json</url-
pattern>
            </servlet-mapping>

        <!--End RP Servlet Configuration -->

```

3. Configure the value for the `StateCookieTimeOutInSec` parameter. This setting specifies the state cookie timeout for requests via `rp/rest/init` (for CLP compatibility). By default, it is set to 180 seconds.
 4. Modify the url-pattern for `OAuthRPInitServlet`, `OAuthRPCallBackServlet`, and `OAuthRPJwksServlet` to specify the init URL (`/rp/init/<authnRealmId>`), callback URL (`/rp/cb/<authnRealmId>/`), and JWKS
-

URL (/<authnRealmId>/well-known/jwks.json) based on the Authentication Realm Id. In the example above, sgspstag is the Authentication Realm Id.

Refer to *UAM OIDC Relying Party Implementation Guide - Configure OIDC RP Configuration File* for the descriptions of the properties to configure.

5. Save the changes.

Configure the Authentication Realm for SingPass QR Code Login

1. Navigate to <CLP_HOME>/WEB-INF/classes and open **am.commonlogin.config.xml**.
2. Look for the AuthnRealms element and configure the SingPass Mobile login authentication realm as shown below:

XML

```
<AuthnRealm>
  <id>sgspstag</id> <!-- for SingPass Staging Login-->
  <idLabel></idLabel>

  <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

  <loginModuleIdForChangePassword></loginModuleIdForChangePassword>
  <stateForChangePassword></stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>SocialNetworkLogin</handler>
    <loginState>externalOIDClogin.main</loginState>
  </loginModule>

  <!--
  <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>--><!-- This is to define whether to use the AM5 session token
  lifetime expiry for cookie session expiry. Default:false -->
</AuthnRealm>
```

3. Save the changes.
 4. Restart the CLP server for changes to take effect.
-

6.9. FIDO2 Authentication

The following sections provide detailed instructions on how to integrate AccessMatrix Common Login Page (CLP) with FIDO2 authentication. In this configuration, AccessMatrix and CLP act as the Relying Party (RP), and a FIDO2-compliant authenticator is used for authentication. Both of the following types of authenticators are supported:

- **Platform authenticators** - Authenticators built into and restricted to a specific device (e.g. fingerprint scanner on a laptop for Windows Hello).
- **Cross-platform authenticators** - External authenticators that can be used with multiple devices (e.g. physical security key).

i **Notes:** FIDO2 authentication is supported as an authentication method in the following scenarios:

- CLP for AccessMatrix Applications
- CLP as SAML Identity Provider
- CLP as OIDC OpenID Provider
- CLP for WSA RP

Ensure that the necessary configurations, as described under [AccessMatrix CLP Application Integrations](#), are made before proceeding with the following steps.

! **Important Note:** Instead of performing the configurations below, FIDO2 authentication can also be used out-of-the-box with no additional configurations. A default FIDO2 authentication realm (DefaultFIDO2Realm) is created in AccessMatrix upon the initial startup. It is already configured with a pre-created:

- Lookup module (DefaultFIDO2MultiChoiceLookup)
- Lookup module (DefaultFIDO2DCLookup)
- Login module (DefaultFIDO2TokenLogin)
- Token policy (DefaultFIDO2TokenPolicy)
- Token batch (DefaultFIDO2TokenBatch)
- Token group (DefaultFIDO2TokenGroup)

If you wish to use FIDO2 without further configurations, simply skip to the [Next Steps](#) section.

If you wish to create your own FIDO2 authentication realm, or learn more about the configurable attributes for each of the above, you can proceed to view all of the steps below.

Prerequisites and Preparation

Ensure that the following is ready:

- AccessMatrix Server is configured with a valid UAS license with the FIDO2 token vendor.

Configure FIDO2 Token Policy

1. Navigate to **Administration Dashboard > Security Policy > Token** and click **Token Policy** on the left pane.
2. Click **Create**, select the "FIDO2 Token Policy" implementation on the **Choose Token Policy Implementation** page and click **Next** button.
3. Enter a **Token Policy Id** (e.g. "FIDO2Policy") and a **Token Policy Name** (e.g. "FIDO2 Policy").
4. Provide values for the remaining attributes, if necessary.
5. Click **Save**.



Note: Refer to the following resources for more information:

- [FIDO2 Token Implementation Guide - Security Considerations](#) for security considerations to note while configuring this token policy.
- [FIDO2 Token Implementation Guide - Configure Token Policy](#) for the full attribute details of this token policy.

Configure Token Batch

1. Navigate to **Operation Dashboard > Token > Token Batch** and click the **Create** button.
2. Enter a **Token Batch Id** (e.g. "FIDO2Batch") and a **Token Batch Name** (e.g. "FIDO2 Batch").
3. Under **Token Policy**, specify the Id of the previously-created policy (e.g. "FIDO2Policy").
4. Provide details for the remaining token batch attributes, if necessary.
5. Click **Save**.



Note: For more details on the remaining token batch attributes, refer to [UAS Administration Guide - Token Batch](#).

Configure Token Group

1. Navigate to **Operation Dashboard > Token > Token Group** and click the **Create** button.
 2. Enter a **Token Group Id** (e.g. "FIDO2Group") and a **Token Group Name** (e.g. "FIDO2 Group").
 3. Provide details for the remaining token group attributes, if necessary.
-

-
4. Click **Save**.

i **Note:** For more details on the remaining token group attributes, refer to [UAS Administration Guide - Token Group](#).

Configure Lookup Module

CLP supports usernameless authentication for FIDO2, wherein the user is not required to provide a username in the authentication process. This feature is also known as ID-less authentication. To enable this feature, you are required to configure the FIDO2 discoverable credentials lookup module. The instructions are detailed below:

Configure FIDO2 Discoverable Credentials Lookup Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module** and click **Create**.
2. Select the "FIDO2 Discoverable Credentials Lookup" implementation on the **Choose User Lookup Module Implementation** page and click **Next**.
3. Enter a **User Lookup Module Id** (e.g. "FIDO2DCLookup") and a **User Lookup Module Name** (e.g. "FIDO2 Discoverable Credentials Lookup").
4. Click **Save**.

i **Note:** You do not need to configure the user store for this lookup module.

Configure Multiple Choice Lookup Module

The multiple choice lookup module functions as a mechanism to link lookup modules so they may be used together. To enable usernameless authentication for FIDO2, the multiple choice lookup module must be used to link the FIDO2 discoverable credentials lookup module to a second lookup module. The second lookup module should be the lookup module corresponding to the user store in which the user is located (e.g. "SystemLookup"). Ensure that the lookup modules have been created before proceeding.

i **Note:** You can refer to [AccessMatrix Common Administration Guide - Configure User Lookup Modules](#) for information on the lookup modules that may be relevant.

Next, perform the following steps.

-
1. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module** and click **Create**.
 2. Select the "Multiple Choice Lookup" implementation on the **Choose User Lookup Module Implementation** page and click **Next**.
 3. Enter a **User Lookup Module Id** (e.g. "FIDO2MCLookup") and a **User Lookup Module Name** (e.g. "FIDO2 Multiple Choice Lookup").
 4. Under the **Configuration** tab, for the attribute **First Lookup Module**, select the lookup module that was previously configured (e.g. "FIDO2DCLookup").



Note: For better performance, the first lookup module should be the FIDO2 discoverable credentials lookup.

5. For the attribute **Second Lookup Module**, select the secondary lookup module corresponding to the user store the user is located in (e.g. "SystemLookup").
6. Click **Save**.

Configure Login Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
2. Click **Create** and select the "FIDO2 Token Login" implementation from the **Choose Login Module Implementation** page.
3. Enter a **Login Module Id** (e.g. "FIDO2LM") and a **Login Module Name** (e.g. "FIDO2 Login Module").
4. Configure the following attributes under the **Relying Party Configuration** header:
 - **Relying Party ID** - Specify the Relying Party server domain (e.g. "fido2.i-sprint.com").
 - **Relying Party Port** - Specify the port of the Relying Party server (e.g. "8443").
5. Configure the following attributes under the **FIDO2 Token Configuration** header:
 - **Token Batch** - The FIDO2 token batch Id (e.g. "FIDO2Batch").



Note: This should match the Id of the token batch configured in the respective [Prerequisites](#) step.

- **Token Group** - The FIDO2 token group Id (e.g. "FIDO2Group").



Note: This should match the Id of the token group configured in the respective [Prerequisites](#) step.

6. Click **Save**.
-

i **Note:** Refer to [FIDO2 Token Implementation Guide - Configure Login Module](#) for the full attribute details of this login module.

Configure Authentication Realm

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
2. Click **Create**.
3. Enter an **Authentication Realm Id** (e.g. "FIDO2Realm") and an **Authentication Realm Name** (e.g. "FIDO2 Realm").
4. Specify the **User Lookup Module** value that will be used to validate user identity (e.g. "SystemLookup").

i **Note:** To allow flexible FIDO authentications (both ID and ID-less authentication), use the multiple choice lookup module configured in the earlier step (e.g. FIDO2MCLookup). This lookup module should contain a FIDO2 discoverable credentials lookup module.

5. Specify the newly-created **First(1st) Login Module** that will be used for user authentication (e.g. "FIDO2LM").
6. Click **Save**. A new authentication realm is created.
7. Assign this realm to a user.

Configure USO Self-Service Portal Policy

1. Navigate to **Administration Dashboard > Configuration > Unified SSO** and click **Enterprised SSO > USO Self-Service Portal** on the left pane.
2. Ensure that the **Enable "FIDO Registration" function** attribute is set to "true". It should be enabled by default.
3. Ensure that the the configured login module is assigned to the **FIDO Registration Login Module Id** attribute (e.g. "FIDO2LM").
4. Click **Save**.

i **Note:** Refer to [AccessMatrix SSO Portal Implementation Guide - Configure USO Self-Service Portal Policy](#) for the detailed steps, if necessary.

Configure CLP Config File

This section details configurations required for CLP.

1. Navigate to `<AM5_ROOT>/tomcat/webapps/clp/WEB-INF/classes/` and open **am.commonlogin.config.xml**.
2. Look for the `AuthnRealms` element and configure it as shown below:

XML

```
...
<!-- Default realms for user selection during common login
-->
<AuthnRealms>
...
    <AuthnRealm>
        <id>FIDO2Realm</id> <!-- for Fido2 Login -->
        <idLabel></idLabel>

        <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

        <loginModuleIdForChangePassword></loginModuleIdForChangePassword>
        <stateForChangePassword></stateForChangePassword>
        <resetPassword>
            <userStoreId></userStoreId>
            <loginModuleId>FIDO2LM</loginModuleId>
        </resetPassword>
        <loginModule>
            <handler>FIDO2TokenLogin</handler>
            <loginState>login.fido2</loginState>
        </loginModule>

        <!--
        <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>
        <--><!-- This is to define whether to use the AM5 session token
        lifetime expiry for cookie session expiry. Default:false -->
        </AuthnRealm>
    ...
</AuthnRealms>
```

Ensure that the following values are correct:

- `id` - This value should match the authentication realm Id of the [configured authentication realm](#) (e.g. "FIDO2Realm").
- `loginModuleId` - This value should match the login module Id of the [configured login module](#) (e.g. "FIDO2LM").



Note: If you opted to use the pre-created authentication realm, do not update the above fields. Leave them as their default values.

3. Save the changes.
4. Restart the CLP server for the changes to take effect.

Next Steps

After FIDO2 authentication has been configured, end users can perform FIDO2 self-registration by doing the following:

- **Register FIDO2 Authentication Method via SSO Portal**

Click on the **FIDO2** icon in the SSO Portal and click on **Register Credential**. Follow the instructions from the FIDO2 authenticator when prompted. Refer to [AccessMatrix SSO Portal User Guide - FIDO2 Credential Management](#) for the detailed steps, if necessary.

- **Log In with FIDO2 Authentication Method via CLP**

Select the relevant FIDO2 authentication realm on CLP, enter the user Id and log in via the newly-registered FIDO2 sign-in method.

For the API details of FIDO2 Authentication with CLP, refer to Authentication Registration and Authentication Assertion flows under [UAS FIDO2 Token Implementation Guide - UAS FIDO2 Token Implementation Using API](#).

6.10. External OP Authentication via OIDC

The following sections provide instructions on how to integrate CLP with an external OpenID Provider (OP) for OIDC authentication. This authentication method uses the OAuth 2.0/OIDC Authorization Code flow.

Prerequisites and Preparation

In order to set up external Identity Provider authentication using OIDC, ensure that the following are ready:

- Outgoing connection from the AccessMatrix Server to your OP is allowed
- Ensure that the OIDCRP license feature is added under **UAM FEATURE - Social Network Login** to enable OIDC RP support and to display "OIDC Relying Party Lookup Module" in the lookup module implementation list.

Configure OIDC Relying Party (RP) Lookup Module

To create an OIDC Relying Party Lookup Module:

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **User Lookup Module** on the left pane.
2. Click the **Create** button and select the "OIDC Relying Party Lookup Module" implementation from the **Choose User Lookup Module Implementation** page.
3. Enter a suitable value for **User Lookup Module Id** (e.g. "CLPOIDCLookup") and **User Lookup Module Name** (e.g. "CLP - OIDC RP Lookup").
4. Configure the values for the following attributes based on your setup:
 - a. Under the **Attributes** tab:
 - **OpenID Provider Configuration URL:** "<OIDC_OP_Configuration_URL>"
Example: "https://login.microsoftonline.com/clp.i-sprint/v2.0/.well-known/openid-configuration"
 - **Redirection URI:** "<CLP_URL>/rp/cb/<authnRealmId>"
Example: "https://clp.i-sprint.com/clp/rp/cb/oidcOP"



Note: This should match the value of **Authentication Realm Id**, as configured in the [next section](#).

b. Under the **OIDC RP Servlet** tab:

- **Landing Page URL:** "<CLP_URL>/extoidcop"
Example: "https://clp.i-sprint.com/clp/extoidcop"
- **Error Page URL:** "<CLP_URL>/error.jsp"
Example: "https://clp.i-sprint.com/clp/error.jsp"

5. Provide the values for the remaining lookup module attributes as needed. Refer to [UAM OIDC Relying Party Implementation Guide - Configure Lookup Module](#) for details on each of the attributes and the steps to configure them.



Note: To support client assertion JWT, refer to [UAM OIDC Relying Party Implementation Guide - Enable Client Authentication Using Client Assertion JWT](#) for details.

6. Save your changes and exit.

Configure Login Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
2. Click **Create** and select the "Social Network Login implementation" from the **Choose Login Module Implementation** page.
3. Enter a **Login Module Id** (e.g. "SocNetLM") and a **Login Module Name** (e.g. "Social Network Login Module").
4. Select the **Login Policy** and the **Authentication Level** values, if necessary.
5. Click **Save**. A new login module is created.

Configure Authentication Realm

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
 2. Click **Create**.
 3. Enter an **Authentication Realm Id** (e.g. "oidcOP") and an **Authentication Realm Name** (e.g. "External OIDC Realm").
 4. Specify the **User Lookup Module** value that will be used to validate user identity, as configured in the previous section (e.g. "CLP - OIDC RP Lookup (CLPOIDCLookup)").
 5. Specify the newly-created **First(1st) Login Module** that will be used for user authentication (e.g. "Social Network Login Module (SocNetLM)").
-

-
6. Click **Save**. A new authentication realm is created.
 7. Assign this realm to a user.

Configure OAuth RP Servlet Settings

1. Navigate to `<CLP_HOME>/WEB-INF` and open **web.xml**.
2. Look for the OAuth RP Servlet Configuration comment as shown below:

```
XML

<!-- OAuth RP Servlet Configuration -->
    <servlet>
        <servlet-name>OAuthRPInitServlet</servlet-name>
        <servlet-
class>com.isprint.oauth.rp.OAuthRPInitServlet</servlet-
class>
    </servlet>
    <servlet>
        <servlet-name>OAuthRPCallBackServlet</servlet-
name>
        <servlet-
class>com.isprint.oauth.rp.OAuthRPCallBackServlet</servlet-
class>
    </servlet>
    <servlet>
        <servlet-name>OAuthRPJwksServlet</servlet-name>
        <servlet-
class>com.isprint.oauth.rp.OAuthRPJwksServlet</servlet-
class>
    </servlet>
    <servlet-mapping>
        <servlet-name>OAuthRPInitServlet</servlet-name>
        <url-pattern>/rp/init/oidcOP</url-pattern>
        <url-pattern>/rp/rest/init</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>OAuthRPCallBackServlet</servlet-
name>
        <url-pattern>/rp/cb/oidcOP</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>OAuthRPJwksServlet</servlet-name>
        <!--Servlet path "<realmId>" followed by "/.well-
known/jwks.json". Configure the realmId where the lookup
module is an OIDC Relying Party Lookup Module handler -->
        <url-pattern>/oidcOP/.well-known/jwks.json</url-
pattern>
    </servlet-mapping>
```

3. Modify the url-pattern for OAuthRPInitServlet, OAuthRPCallBackServlet, and OAuthRPJwksServlet to specify the init URL (/rp/init/<authnRealmId>), callback URL (/rp/cb/<authnRealmId>) and JWKS URL (/<authnRealmId>/.well-known/jwks.json) based on the Authentication Realm Id. In the
-

example above, oidcOP is the Authentication Realm ID.

Refer to *UAM OIDC Relying Party Implementation Guide - Configure OIDC RP Configuration File* for the description of the properties to configure.

4. Save the changes.

Configure Authentication Realm in CLP Config File

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the `AuthnRealms` element and add the oidcOP authentication realm, as shown below:

XML

```
<AuthnRealm>
  <id>oidcOP</id> <!-- for External OIDC Login-->
  <idLabel></idLabel>

  <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

  <loginModuleIdForChangePassword></loginModuleIdForChangePassword>
  <stateForChangePassword></stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>SocialNetworkLogin</handler>
    <loginState>externalOIDClogin.main</loginState>
  </loginModule>

  <!--
  <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>--><!-- This is to define whether to use the AM5 session token
  lifetime expiry for cookie session expiry. Default:false -->
</AuthnRealm>
```

3. Save the changes.
 4. Restart the CLP server for changes to take effect.
-

6.11. End-to-End Encryption Authentication (E2EEA)

AccessMatrix Universal Authentication Server (UAS) provides the End-to-End Encryption Authentication (E2EEA) feature, where an end-to-end application layer encryption protects passwords and other sensitive data between a client device and AccessMatrix UAS.

The following sections contain information on how to integrate CLP with E2EEA.

Prerequisites and Preparation

In order to set up E2EEA login with CLP, ensure that the following are ready:

- If you are using an HSM, ensure that it has been deployed and integrated with the AccessMatrix UAS E2EEA solution. The details on how to do so are beyond the scope of this guide; you should refer to the respective sections for your HSM in the [UAS E2EEA Developer Guide](#).
- AccessMatrix Server is configured with a valid UAS license with the **E2EEA Base** license feature enabled.

Configure Keys

Pseudo E2EEA	Configure the Key Provider Refer to UAS Pseudo E2EEA Implementation - Configure Key Provider for detailed instructions on creating a key provider for the wrapping key. Configure the Wrapping Key Refer to UAS Pseudo E2EEA Implementation - Configure Encryption Key for detailed instructions on creating a storage key via the Admin Console. Configure the Storage Key Refer to UAS Pseudo E2EEA Implementation - Configure Encryption Key for detailed instructions on creating a storage key via the Admin Console. For the Purpose attribute, select "PIN Encryption" instead of "Key Wrapping".
SafeNet ProtectServer E2EEA	Generate Public Key Refer to UAS E2EEA with SafeNet ProtectServer HSM Implementation - Generate Public Key for detailed instructions on creating the public key.

	Generate Storage PIN Encryption Key Refer to UAS E2EEA with SafeNet ProtectServer HSM Implementation - Generate Storage PIN Encryption Key for detailed instructions on creating the storage PIN encryption key.
Utimaco CryptoServer E2EEA	Generate Public/Private Key Pair Refer to UAS E2EEA with Utimaco CryptoServer HSM Implementation - Generate Public/Private Key Pair for detailed instructions on creating the public/private key pair. Generate Secret Key Refer to UAS E2EEA with Utimaco CryptoServer HSM Implementation - Generate Secret Key for detailed instructions on creating the secret key.

Configure the Login Module

Pseudo E2EEA	The following steps will create a new login module. 1. In the Admin Console, navigate to Administration Dashboard > Configuration > Authentication and click Login Module on the left pane. 2. Click Create and select the "Pseudo E2EEA Login Module" implementation from the Choose Login Module Implementation page 3. Then, click Next and provide the Login Module Id (e.g. "E2EEALM") and Login Module Name (e.g. "E2EEA Login Module"). 4. Configure the following attributes: ○ *Wrapping Key - Specify the key that will be used to encrypt (wrap) the asymmetric keys, created in the Configure the Wrapping Key section. ○ *Storage Key - Specify the key that will be used to encrypt the PIN/password in storage, created in Configure the Storage Key section. 5. Configure the rest of the login module attributes, if necessary. 6. Save the changes.  Note: Refer to UAS Pseudo E2EEA Implementation - Configure Login Module for the full descriptions of every Pseudo E2EEA login module attribute.
SafeNet ProtectServer E2EEA	The following steps will create a new login module. 1. In the Admin Console, navigate to Administration Dashboard > Configuration > Authentication and click Login Module on the left pane. 2. Click Create and select the "SafeNet ProtectServer E2EEA Login Module" implementation from the Choose Login Module Implementation page

	- Then, click Next and provide the Login Module Id (e.g. "E2EEALM") and Login Module Name (e.g. "E2EEA Login Module"). - Configure the following attributes: *HSM User PIN - Specify the user PIN used to login to Key Management Utility (KMU). This is the same PIN used during the key creation step. *HSM Slot Name - The slot or token labels for each HSM; if label is not unique, prefix with HSM serial no.(e.g. "408801:Slot01"). *Public Key Names - List of public key name(s) or ID(s) to use to encrypt the client-side PIN, created in Generate Public Key section. *Storage PIN Encryption Key Names - List of PIN encryption key name(s) or ID(s) to use for encrypting the stored PIN, created in Generate Storage PIN Encryption Key section. - Configure the rest of the login module attributes, if necessary. - Save the changes. i Note: Refer to UAS SafeNet ProtectServer E2EEA Implementation - Configure Login Module for the full descriptions of every SafeNet ProtectServer E2EEA login module attribute.
Utimaco CryptoServer E2EEA	The following steps will create a new login module. - In the Admin Console, navigate to Administration Dashboard > Configuration > Authentication and click Login Module on the left pane. - Click Create and select the "Utimaco CryptoServer E2EEA Login Module" implementation from the Choose Login Module Implementation page - Then, click Next and provide the Login Module Id (e.g. "E2EEALM") and Login Module Name (e.g. "E2EEA Login Module"). - Configure the following attributes: *Device - Specify the CryptoServer HSM device address (e.g. "3001@192.168.1.49", where "192.168.1.49" is the IP address and "3001" is the port of CryptoServer LAN address). *HSM User PIN - Specify the HSM user PIN. This is the same user PIN you used to log in to PKCS#11 CAT tool. *Public Key Label(s) - Specify the name(s) or ID(s) of the public key(s) to use to encrypt the client-side PIN, created in Generate Public/Private Key Pair section. *TPIN Key Label(s) - Specify the name(s) or ID(s) of the PIN encryption key(s) to use for encrypting the stored PIN, created in Generate Secret Key section. - Configure the rest of the login module attributes, if necessary. - Save the changes. i Note: Refer to UAS Utimaco CryptoServer E2EEA Implementation - Configure Login Module for the full descriptions of every Utimaco CryptoServer E2EEA login module attribute.

Configure the Authentication Realm

The following steps will create a new authentication realm in Admin Console.

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane.
2. Click **Create** and provide the **Authentication Realm Id** (e.g. "E2EEARL") and **Authentication Realm Name** (e.g. "E2EEA Realm").
3. Specify the **User Lookup Module** value that will be used to validate the user identity (e.g. "SystemLookup").
4. Specify the **First(1st) Login Module** that was created in the previous section (e.g. "E2EEALM").
5. Save the changes.
6. Assign this authentication realm to a user.

Configure the Authentication Realm in CLP

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML

```
<AuthnRealm>
  <id>E2EEARL</id>
  <idLabel>Pseudo E2EEA</idLabel>

  <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

  <loginModuleIdForChangePassword>E2EEALM</loginModuleIdForChangePassword>

  <stateForChangePassword>login.chgpwd</stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>PseudoE2EEALoginModule</handler> <!--applicable to
the following handlers: UtimacoCSE2EELoginModule,
SafenetPSGLoginModule as well -->
    <loginState>login.start</loginState>
    <!--<PQC>required</PQC>--> <!-- Only applicable to
PseudoE2EEALoginModule, value with required or preferred. Defaults
with RSA keys if undefined.-->
  </loginModule>
</AuthnRealm>
```

```

        <!-- Add external link to the login page -->
        <!--<externalLinks>
            <externalLink>
                <order>1</order>
                <url>http://google.com</url>
                <label>Lbl_ExLink_1</label>
            </externalLink>
            <externalLink>
                <order>2</order>
                <url>https://xx</url>
                <label>Lbl_ExLink_2</label>
            </externalLink>
        </externalLinks> -->
    </loginModule>

    <!--
    <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>--><!-- This is to define whether to use the AM5 session token
    lifetime expiry for cookie session expiry. Default:false -->
    </AuthnRealm>

```

Ensure that the following values are updated:

- a. id - This value should match the authentication realm Id of the [configured authentication realm](#) (e.g. "E2EEARL").
- b. loginModuleIdForChangePassword - To allow the use of the change password feature using the E2EEA login module, ensure this value matches the Id of the login module used for E2EEA login (e.g. "E2EEALM").

i **Note:** Additionally, ensure the change password setting for CLP is enabled. Refer to the [AccessMatrix CLP Other Authentication Configurations - Enable Change Password](#) section for the details.

- c. handler - Update the value to one the following supported handlers, depending on the type of login module used:
 - i. **Pseudo E2EEA login module** - "PseudoE2EEALoginModule"
 - ii. **SafeNet ProtectServer E2EEA login module** - "SafenetPSGLoginModule"
 - iii. **Utimaco CryptoServer E2EEA login module** - "UtimacoCSE2EELoginModule"
- d. (Optional) PQC - Verifies if the PseudoE2EEALoginModule handler is Post Quantum Cryptography (PQC) enabled.

i **Note:** This configuration is only applicable to the PseudoE2EEALoginModule handler. To enable this configuration, uncomment and specify with a relevant value. Otherwise, RSA keys will always be returned instead. For more details on configuring PQC for the

Pseudo E2EEA Login Module, refer to Configure Pseudo E2EEA Login Module .	
PQC Value	Description
required	If specified as required, PQC keys will be used. An error will be returned if there are no PQC keys available.
preferred	if specified as preferred, PQC keys will be used. If there are no PQC keys available, RSA keys will be used instead.

- e. (Optional) `externalLink` - It is possible to add links to external pages on the E2EEA login page. If required, uncomment the code block and configure the values. Refer to the [Display External Links on Login Page](#) section for the full configuration details.
3. Save the changes.
 4. Restart the CLP server for changes to take effect.
-

6.12. Certificate-based Authentication

This page provides instructions on how to enable CLP to allow the use of SSL client certificate-based authentication for user logins. Certificate-based authentication refers to the use of an SSL/TLS client certificate (a data file used to verify the identity of an entity) as a method of authenticating the user. In a production environment, the certificate should be digitally signed by a trusted Certificate Authority (CA) to ensure its authenticity.

In this page, the instructions detail how to enable certificate-based authentication via CLP for end users. Once enabled, the overall flow for certificate-based authentication is as follows:

1. An end user (an interactive user) initiates the login process on CLP.
2. The CLP server presents its own certificate to the browser for validation.
3. Once the browser validates its certificate, the CLP server makes an authentication request asking the end user to authenticate themselves.
4. The end user selects their certificate via the browser, and the browser presents it to the CLP server for validation.
5. The CLP server calls the AccessMatrix server via REST API, where the certificate's Distinguished Name (DN) is used to look up the corresponding AccessMatrix user.
6. The AccessMatrix server further validates and authenticates the user's identity before the user is granted access.

Prerequisites and Assumptions

In order to set up certificated-based authentication, ensure that the following are ready:

End User Prerequisites

- The necessary certificate files should already be prepared. In a production environment, this refers to:
 - The end-entity certificate for the user, issued by a CA or its intermediate CA.
 - The certificate of the root CA.
 - The certificate of the intermediate CA, if the user certificate was issued by an intermediate CA.
 - A CLP truststore containing all of the above certificates (e.g. **truststore.jks**).
-

-
- Additionally, all of the above files should be placed into the directory <AM_ROOT>/tomcat/conf/.

CLP Prerequisites

- CLP should already be securely deployed on HTTPS. It is assumed that port 8443 (the default port for secure deployment) is used.
 - As part of a secure CLP deployment, you should have prepared the necessary agent configurations for CLP as well. These configurations are described in detail in [Deployment Configurations - Configure AccessMatrix Server Settings](#) and will not be covered here. You must complete all of those steps before proceeding.
- You should also configure the CLP server settings as follows:
 1. Navigate to <AM_ROOT>/tomcat/conf/ and open **server.xml**.
 2. Add the following code block into the file's "Connector" section:

XML

```
<Connector port="8444" minSpareThreads="10"
protocol="com.isprint.amutil.tomcat.AMCustomNIO2Protocol"
    enableLookups="false"
    disableUploadTimeout="true"
    keepAliveTimeout="900000"
    maxKeepAliveRequests="-1"
    acceptCount="100" maxThreads="200"
    scheme="https" secure="true" SSLEnabled="true"
>
    <SSLHostConfig
certificateVerification="optional" sslProtocol="TLS"
protocols="TLSv1.2,TLSv1.3"
ciphers="TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_RSA_W
ITH_AES_128_GCM_SHA256,TLS_AES_256_GCM_SHA384,TLS_AES_128_GCM_
SHA256" truststoreFile="conf/truststore.jks"
truststorePassword="AesEncryptor:txNbulL6Ip9JtvEFzUZ1UsK2LzK8L
JUyF2RQgyTe2RU=" truststoreType="JKS">
        <Certificate
certificateKeystoreFile="conf/testserver.keystore"
certificateKeystorePassword="AesEncryptor:txNbulL6Ip9JtvEFzUZ1
UsK2LzK8LJUyF2RQgyTe2RU=" certificateKeystoreType="JKS"/>
    </SSLHostConfig>
</Connector>
```

Ensure the values of the following fields are accurate:

- α. truststoreFile - The file name of CLP truststore (e.g. "truststore.jks").



Note: You must ensure that the root CA and intermediate CA certificates have been imported into the same keystore. Failing to do so may result in

an error when the end user attempts certificate-based login, such as the following error in the **am.log** file:
sun.security.validator.ValidatorException: PKIX path building failed:
sun.security.provider.certpath.SunCertPathBuilderException: unable to find valid certification path to requested target

- β. truststorePassword - The password of the truststore.

i **Note:** You may wish to use the bundled Encryptor Utility Tool to encrypt your truststore password. Refer to [AccessMatrix Common Deployment Guide - Using AesEncryptor to Encrypt TrustStore and KeyStore Password](#) for the instructions on how to do so.

3. Save the changes.

Configure Certificate DN Lookup Module

To configure the certificate DN lookup module:

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **User Lookup Module** on the left pane.
 2. Select the pre-created lookup module "CertificateDnLookup".
 3. Configure the values for the following attributes based on your setup:
 - **Certificate Filter** - Provide the attribute in the certificate to be mapped with the attribute in the user store for user lookup. The general format used is "<field name of user store>=<field name of certificate>" (e.g. "DN=Subject"). You can leave the attribute blank to use the default value "DN=Subject".
 - **Java Regular Expression group** - If a Java Regular Expression is used as part of the **Certificate Filter** value, this attribute is used to define the Java Regular Expression.
 - **Allow Trusted App PKI User Authentication** - Enable this attribute by setting it to "true". This allows trusted app PKI user authentication, which is necessary for certificate-based authentication via CLP. You should already have created a CLP agent (as part of the secure CLP deployment requirements noted in the [Prerequisites and Assumptions](#) section).
 - **Agent-based Access Restriction** - Select the CLP agent (e.g. "clpagent") to be whitelisted. This agent will serve as the agent of the trusted application (i.e. CLP) for PKI authentication.
-



Note: It is strongly recommended that you refer to [AccessMatrix Common Administration Guide - Certificate DN Lookup](#) for the full details on each of the above attributes.

4. Save your changes and exit.

Configure Login Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
2. Select the pre-created login module "ClientCertificateOverSSL".
3. Click **Edit** and select the **Login Policy** and the **Authentication Level** values, if necessary.
4. Save your changes and exit.

Configure Authentication Realm

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
2. Select the pre-created authentication realm "40153".
3. Ensure that the **User Lookup Module** value matches the name of the lookup module configured in the previous section (e.g. "Certificate DN Lookup (CertificateDnLookup)").
4. Ensure that the **First(1st) Login Module** value matches the name of the login module configured in the previous section (e.g. "ClientCertificateOverSSL (ClientCertificateOverSSL)").
5. Save your changes and exit.

Configure End User

The following instructions assume there is an existing user. If not, create a new user at **Operation Dashboard > User & Segment > User** and follow steps 3-6.

1. Navigate to **Operation Dashboard > User & Segment > User**.
 2. Select the end user that will use certificate-based authentication and click the **Edit** button.
 3. Ensure that ***Interactive** is set to "Yes".
 4. Under **Distinguished Name**, specify the user's DN (e.g. "CN=demouser, OU=Security, O=i-sprint, L=Singapore, ST=Singapore, C=SG"). Ensure these fields and values match those in the user's certificate, based on the mapping previously defined in the certificate DN lookup module's **Certificate Filter** attribute.
-

-
5. Under the **Login Accounts** tab, click the **Add** button and select the "HTTPS Certificate (40153)" authentication realm.
 6. Click the **Save** button.

Configure Authentication Realm in CLP Config File

1. Navigate to `<CLP_HOME>/WEB-INF/classes/` and open **am.commonlogin.config.xml**.
2. Look for the `AuthnRealms` element and uncomment the 40153 authentication realm.
3. Configure it as shown below:

XML

```
<AuthnRealm>
  <id>40153</id>
  <idLabel>Client Certificate</idLabel>

  <firstLoginModuleNeedsChallenge>false</firstLoginModuleNeedsChallenge>

  <loginModuleIdForChangePassword></loginModuleIdForChangePassword>
  <stateForChangePassword></stateForChangePassword>
  <resetPassword>
    <userStoreId></userStoreId>
    <loginModuleId></loginModuleId>
  </resetPassword>
  <loginModule>
    <handler>SSLClientCertificateLogin</handler>
    <loginState>login.cert</loginState>
  </loginModule>

  <!--
  <useSessionTokenLifetimeForCookie></useSessionTokenLifetimeForCookie>
  --><!-- This is to define whether to use the AM5 session token
  lifetime expiry for cookie session expiry. Default:false -->
</AuthnRealm>
```

4. Save the changes.
5. Restart the CLP server for changes to take effect.

Import Certificates to Browser

This section provides general instructions for importing certificates to the end user's browser for certificate-based authentication. In addition to importing the user's certificate, the certificate of the root CA that issued the user certificate must also be imported so that the user's certificate can be trusted. If the user certificate was issued by an intermediate CA, the intermediate CA's certificate should be imported along with the root CA's certificate.

i Note: The specific steps for certificate imports may vary, depending on your browser and operating system. Refer to the relevant documentation for your specific browser as needed.

1. **Prepare the user and CA certificates** - Prepare the certificates of the user and root CA (and intermediate CA if applicable). These certificates files should be in a format that the browser recognizes.

i Note: In particular, CA certificates are usually provided as .PEM or .DER files.

2. **Navigate to your browser's security settings** - In your web browser, access the settings menu and open the security settings. Certificate import-related options are usually located within these settings, although this may differ from browser to browser.
 3. **Locate the certificate management settings** - Within the security settings, locate the certificate settings and open your browser's certificate management interface.
 4. **Import the certificates** - Look for an option to import certificates. Follow the instructions and complete the import for each of the certificates (user, intermediate CA, and root CA). In particular, you may wish to ensure that the following configurations are correctly performed:
 - a. **Select the appropriate certificate file** - Locate and select the certificate file you want to import.
 - b. **Select the import location** - Specify where the certificate should be stored based on the type of the certificate:
 - User certificate: "Personal" store.
 - Intermediate CA certificate: "Intermediate Certification Authorities" store.
 - Root CA certificate: "Trusted Root Certification Authorities" store.
 - c. **Configure import options** - You may be prompted to configure other import options. You can select the relevant options as needed.
 5. **Verify the success of the certificate imports** - After successfully importing the certificates, they should appear in the list of trusted certificates in your browser's certificate manager.
 6. **Enable trust for CA for SSL/TLS** - Your browser may also have an option to enable trust for a root CA - enable it, if necessary.
 7. **Restart your browser** - You may wish to restart your browser, in case the browser requires it for the changes to take effect.
-

-
8. **Test your user certificate** - You can use the URL *https://<CLP Host Name>:8444/clp/clientcert.jsp* to test if the user cert has been imported and set up successfully. If all of the previous steps in this page have been properly completed, you will receive a prompt from your browser to select a certificate for authentication. You should see the certificate you have imported in the list of certificates.
-

7. AccessMatrix CLP Re-Authentication Method

Each method listed below is fully supported by AccessMatrix Applications, SAML, and OIDC.

Reauthentication Method	AccessMatrix Applications	SAML	OIDC
System Password (Basic)	Supported	Supported	Supported
Vasco Token (OTP)	Supported	Supported	Supported
SMS OTP	Supported	Supported	Supported
External OIDC (Relying Party)	Supported	Supported	Supported
FIDO2 Token	Supported	Supported	Supported
RADIUS (1FA OTP)	Supported	Supported	Supported
PseudoE2EEA (with PQC/RSA)	Supported	Supported	Supported

Refer to the [summary table](#) for a quick configuration guide.

7.1. Reauthentication Overview

Configure the Login Module in Admin Console

Follow the steps below to create a new login module in the Admin Console:

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select "System Password Basic" from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** and **Login Module Name**.
4. Configure the rest of the login module attributes, if necessary. Refer to [Manage User Authentication Service - Configure Login Module Section](#) for more details.
5. Save the changes.

Configure the Re-Authentication Login Module in CLP

Re-Authentication Login Module Configurations

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Reauthentication configuration is all under `<Reauthenticate>` tag.
3. Add the newly created authentication login module in this CLP configuration file.

Example configuration::

XML

```
<Reauthenticate>
  <loginModule>
    <loginModuleId>SystemPasswordBasic</loginModuleId>

    <loginModuleNeedsChallenge>false</loginModuleNeedsChallenge>
    <handler>SystemPasswordBasic</handler>
  </loginModule>
  .....
  .....
```

```
.....  
<Reauthenticate>
```

4. Save the changes.
5. Restart the CLP server for changes to take effect.

Login Module Elements

Below is the list of elements and attributes for Authentication Realm configuration:



Note: All the values (`loginModuleId`, `handler`) should **map consistently to the Login Module definition in the Admin Console.**

If `loginModuleId` is `SystemPasswordBasic` in the XML, the Admin Console must also have a login module configured with ID = `SystemPasswordBasic`.

1. **<loginModuleId>**
 - Description: Unique identifier of the login module. Value must **map consistently to the Login Module Id in the Admin Console.**
 - Example: `SystemPasswordBasic`
 2. **<loginModuleNeedsChallenge>**
 - Description: Indicates whether the login module requires a challenge (e.g., OTP) before authentication.
 - Example: `true` or `false`
 3. **<handler>**
 - Description: The handler name of login module. Value must **map consistently to the Login Module handler in the Admin Console.**
 - Example: `SystemPasswordBasic`
 4. **<loginState>**
-

-
- Description: Defines the login state that redirects users to a specific login HTML page. Each handler is mapped to its respective state. These values are system-defined and **cannot be modified**. For more details, refer to [Reauthentication Login Modules – Summary Table](#).
 - Example: login.start
-

7.2. System Password Basic

Configure the Login Module

The following steps will create a new login module for System Password Basic re-authentication.

Alternatively, you can choose to use the pre-created login module "SystemPasswordBasic".

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select "System Password Basic" from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** (e.g. "PwdLM") and **Login Module Name** (e.g. "Password-based Login Module").
4. Configure the rest of the login module attributes, if necessary.
5. Save the changes.

Configure the Re-Authentication Login Module in CLP

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML

```
<loginModule>
  <loginModuleId>SystemPasswordBasic</loginModuleId>

  <loginModuleNeedsChallenge>false</loginModuleNeedsChallenge>
  <handler>SystemPasswordBasic</handler>
  <loginState>login.start</loginState>
</loginModule>
```

3. Save the changes.
 4. Restart the CLP server for changes to take effect.
-

Explanation of Elements

- **<loginModuleId>**

Description: Must match the Login Module ID defined in the Admin Console.

Value: `SystemPasswordBasic`

- **<loginModuleNeedsChallenge>**

Description: Determines if the login module requires an additional challenge step.

Value: `false`

- **<handler>**

Description: The handler associated with the login module. It must correspond to the handler configured in the Admin Console.

Value: `SystemPasswordBasic`

- **<loginState>**

Description: Defines the login UI that will be displayed to the user. This value are system-defined and **cannot be modified**.

Value: `login.start`

7.3. VascoTokenLogin

Configure the Login Module

The following steps will create a new login module for System Password Basic re-authentication.

Alternatively, you can choose to use the pre-created login module "DefaultVascoTokenLogin".

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select "Vasco Token Login" from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** (e.g. "DefaultVascoTokenLogin") and **Login Module Name** (e.g. "VascoTokenLogin").
4. Configure the rest of the login module attributes, if necessary.
5. Save the changes.

Configure the Re-Authentication Login Module in CLP

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML

```
<loginModule>
  <loginModuleId>DefaultVascoTokenLogin</loginModuleId>

  <loginModuleNeedsChallenge>false</loginModuleNeedsChallenge>
    <handler>VascoTokenLogin</handler>
    <loginState>login.otp</loginState>
</loginModule>
```

3. Save the changes.
 4. Restart the CLP server for changes to take effect.
-

Explanation of Elements

- **<loginModuleId>**

Description: Must match the Login Module ID defined in the Admin Console.

Value: DefaultVascoTokenLogin

- **<loginModuleNeedsChallenge>**

Description: Determines if the login module requires an additional challenge step.

Value: false

- **<handler>**

Description: The handler associated with the login module. It must correspond to the handler configured in the Admin Console.

Value: VascoTokenLogin

- **<loginState>**

Description: Defines the login UI that will be displayed to the user. This value are system-defined and **cannot be modified**.

Value: login.otp

7.4. Reauthentication Login Modules – Summary Table

Reauthentication Login Modules – Summary Table

Handler	loginModuleId	Needs Challenge	Login State	PQC	Notes
VascoTokenLogin	DefaultVascoTokenLogin	false	login.otp	-	Token-based reauthentication (Vasco).
SystemPasswordBasic	SystemPasswordBasic	false	login.start	-	Basic system password reauthentication.
SMSOTPLogin	SMSOTPLogin	true	login.firstlogin moduleotp	-	Requires OTP challenge via SMS.
OIDCRelyingPartyLoginModule	<i>(empty)</i>	false	externalOIDCReauth.main	-	External OIDC reauthentication (Relying Party).
FIDO2TokenLogin	DefaultFIDO2TokenLogin	false	login.fido2	-	Reauthentication with FIDO2 tokens (security keys/biometric).
RadiusLogin	RADIUSLogin	false	login.otp	-	1FA only. 2FA not supported for reauthentication.
PseudoE2EEALoginModule	<i>(empty)</i>	false	login.start	preferred	Supports PQC (required/pr

Handler	loginModuleId	Needs Challenge	Login State	PQC	Notes
					preferred). Falls back to RSA if PQC not available.

7.5. SMSOTPLLogin

Configure the Login Module

The following steps will create a new login module for SMSOTPLLogin

re-authentication. Alternatively, you can choose to use the pre-created login module "SMSOTPLLogin".

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select "SMSOTPLLogin" from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** (e.g. "SMSOTPLLogin") and **Login Module Name** (e.g. "SMSOTPLLogin").
4. Configure the rest of the login module attributes, if necessary.
5. Save the changes.

Configure the Re-Authentication Login Module in CLP

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML
<pre><loginModule> <loginModuleId>SMSOTPLLogin</loginModuleId> <loginModuleNeedsChallenge>true</loginModuleNeedsChallenge> <handler>SMSOTPLLogin</handler> <loginState>login.firstloginmoduleotp</loginState> </loginModule></pre>

3. Save the changes.
4. Restart the CLP server for changes to take effect.

Explanation of Elements

- **<loginModuleId>**

Description: Must match the Login Module ID defined in the Admin Console.

Value: SMSOTPLLogin

- **<loginModuleNeedsChallenge>**

Description: Determines if the login module requires an additional challenge step.

Value: true

- **<handler>**

Description: The handler associated with the login module. It must correspond to the handler configured in the Admin Console.

Value: SMSOTPLogin

- **<loginState>**

Description: Defines the login UI that will be displayed to the user. This value are system-defined and **cannot be modified**.

Value: login.firstloginmoduleotp

7.6. OIDCRelyingPartyLoginModule

CLP also supports reauthentication using an external OIDC OpenID Provider. This authentication method uses the OAuth 2.0/OIDC Authorization Code flow or the Authorization Code flow with PKCE. Using the PKCE flow is strongly recommended. The `OIDCRelyingPartyLoginModule` reauthentication **is currently supported only in AccessMatrix Applications**.

	Note: Ensure that the OIDCRP license feature is added under UAM FEATURE - Social Network Login to enable OIDC RP support and to display "OIDC Relying Party Login Module" in the login module implementation list.
---	---

The following is a checklist of the relevant steps needed to enable external OP reauthentication:

Step	Reference Guide
Create OIDC Relying Party Login Module	OIDC Relying Party Implementation Guide - Configure OIDC RP Login Module for Reauthentication via External OIDC OP
Verify That Reauthentication Is Enabled in <code>am.commonlogin.config.xml</code>	-
Configure USO Application Reauthentication Attributes	USO Administration Guide - USO Admin Console Administration
Configure OIDC RP Servlet <code>web.xml</code>	OIDC Relying Party Implementation Guide - Configure OIDC RP Configuration File
(Additional) OAuth RP Servlet Reauthentication Endpoints	OIDC Relying Party Implementation Guide - Use OIDC Relying Party Servlet for Reauthentication

Create OIDC Relying Party Login Module

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
 2. Click the **Create** button and select the "OIDC Relying Party Login Module" implementation from the **Choose Login Module Implementation** page.
 3. Enter a suitable value for **Login Module Id** (e.g. "CLPOIDCLogin") and **Login Module Name** (e.g. "CLP - OIDC RP Login").
 4. Select "Yes" under the **PKCE Flow is Used** attribute if the Authorization Code flow with PKCE is used. "No" is selected by default.
-



Note: It is highly recommended that the Authorization Code flow with PKCE is used.

5. Configure the values for the following attributes based on your setup:
 - a. Under the **Attributes** tab:
 - **OpenID Provider Configuration URL:** "<OIDC_OP_Configuration_URL>"
Example: "https://login.microsoftonline.com/clp.i-sprint/v2.0/.well-known/openid-configuration"
 - **Authorization Endpoint :** Example: "https://clp.i-sprint.com/clp/auth"
 - **Token Endpoint:** Example: "https://clp.i-sprint.com/clp/token"
 - **Redirection URI:** "<CLP_URL>/rp/reauthn/cb/<loginModuleId>"
Example: "https://clp.i-sprint.com/clp/rp/reauthn/cb/CLPOIDCLogin"
 - **User Mapping Lookup Option:** Map ID Token's Claim to AccessMatrix user by user attribute
 - b. Under the **OIDC RP Servlet** tab:
 - **Landing Page URL:** "<CLP_URL>/extoidcop"
Example: "https://clp.i-sprint.com/clp/extoidcop"
 - **Error Page URL:** "<CLP_URL>/error.jsp"
Example: "https://clp.i-sprint.com/clp/error.jsp"
6. Provide the values for the remaining login module attributes as needed. Refer to *UAM OIDC Relying Party Implementation Guide - Configure Login Module for Reauthentication via External OIDC OP* for details on each of the attributes and the steps to configure them.



Note: To support client assertion JWT, refer to *UAM OIDC Relying Party Implementation Guide - Enable Client Assertion JWT* for details.

7. Save your changes and exit.

Verify That Reauthentication Feature Is Configured in am.commonlogin.config.xml

1. Navigate to <CLP_HOME>/WEB-INF/classes and open **am.commonlogin.config.xml**.
2. Look for the Reauthenticate element.
3. Ensure that the following login module for reauthentication is present:

XML
<pre><loginModule> <loginModuleNeedsChallenge>false</loginModuleNeedsChallenge> <handler>OIDCRelyingPartyLoginModule</handler> <loginState>externalOIDCReauth.main</loginState> </loginModule></pre>

4. Save the changes, if necessary.
5. Restart the CLP server for changes to take effect if changes were made.

Configure USO Application Reauthentication Attributes

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Enterprised SSO > USO Application** and click on the relevant USO application Id. The **View USO Application** page is displayed.
2. Under the **Reauthentication** header, configure the following attributes:
 - a. **Verify replay** - Set to "Enable".
 - b. **Verify replay method** - Select the OIDC RP login module Id of the login module configured in the steps above (e.g. "CLPOIDCLogin").
 - c. **Verify replay before launch** - Set to "Yes".
3. Save the changes.



Note: Refer to [USO Administration Guide - USO Admin Console Administration](#) for details on each of the attributes.

Configure OIDC RP Servlet web.xml

1. Navigate to `<ACMATRIX_ROOT>\tomcat\webapps\oauth-rp\WEB-INF\` and open the **web.xml** file.
2. Search for the OAuthRPReauthnJwksServlet servlet mapping and update the url-pattern value by adding the login module Id of the OIDC RP login module configured in the steps above (e.g. "CLPOIDCLogin"):
 - a. **OAuthRPReauthnJwksServlet** - Configure the url-pattern value as `/reauthn/<loginModuleId>/.well-known/jwks.json`
For example:

XML

```
<servlet-mapping>
  <servlet-name>OAuthRPReauthnJwksServlet</servlet-name>
  <!--Servlet path "/reauthn/<moduleId>" followed by "/.well-known/jwks.json". Configure the moduleId where the login module is an OIDC Relying Party Login Module handler -->
  <url-pattern>/reauthn/CLPOIDCLogin/.well-known/jwks.json</url-pattern>
</servlet-mapping>
```

3. Save the changes.



Note: Refer to the *OIDC Relying Party Implementation Guide - Configure OIDC RP Configuration File* for more information on the OIDC RP Servlet **web.xml** file.

(Additional) OAuth RP Servlet Reauthentication Endpoints

The application must pass the OIDC Relying Party (RP) login module Id to CLP for that login module to be used for external reauthentication.

If necessary, you may refer to [UAM OIDC Relying Party Implementation Guide - Use OIDC Relying Party Servlet for Reauthentication](#) for the full details on the OIDC RP Servlet endpoints and request parameters involved in the reauthentication process.

7.7. FIDO2TokenLogin

Configure the Login Module

The following steps will create a new login module for System Password Basic re-authentication.

Alternatively, you can choose to use the pre-created login module "FIDO2TokenLogin".

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select "Vasco Token Login" from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** (e.g. "DefaultFIDO2TokenLogin") and **Login Module Name** (e.g. "FIDO2TokenLogin").
4. Configure the rest of the login module attributes, if necessary.
5. Save the changes.

Configure the Re-Authentication Login Module in CLP

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML

```
<loginModule>
  <loginModuleId>DefaultFIDO2TokenLogin</loginModuleId>

  <loginModuleNeedsChallenge>false</loginModuleNeedsChallenge>
  <handler>FIDO2TokenLogin</handler>
  <loginState>login.fido2</loginState>
</loginModule>
```

3. Save the changes.
 4. Restart the CLP server for changes to take effect.
-

Explanation of Elements

- **<loginModuleId>**

Description: Must match the Login Module ID defined in the Admin Console.

Value: DefaultFIDO2TokenLogin

- **<loginModuleNeedsChallenge>**

Description: Determines if the login module requires an additional challenge step.

Value: false

- **<handler>**

Description: The handler associated with the login module. It must correspond to the handler configured in the Admin Console.

Value: FIDO2TokenLogin

- **<loginState>**

Description: Defines the login UI that will be displayed to the user. This value are system-defined and **cannot be modified**.

Value: login.fido2

7.8. RadiusLogin

Configure the Login Module

The following steps will create a new login module for System Password Basic re-authentication.

Alternatively, you can choose to use the pre-created login module "RADIUSLogin".

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select "RadiusLogin" from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** (e.g. "RADIUSLogin") and **Login Module Name** (e.g. "RadiusLogin").
4. Configure the rest of the login module attributes, if necessary.
5. Save the changes.

Configure the Re-Authentication Login Module in CLP

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML

```
<loginModule>
  <loginModuleId>RADIUSLogin</loginModuleId>

  <loginModuleNeedsChallenge>false</loginModuleNeedsChallenge>
  <!-- This setting applies to RadiusLogin using 1FA
  (OTP), loginModuleNeedsChallenge must set to false otherwise
  might get "request-reject-unknown" error. Note: 2FA is not
  supported for reauthentication, otherwise might get "Access
  Challenge->Authentication required" . -->
    <handler>RadiusLogin</handler>
    <loginState>login.otp</loginState>
</loginModule>
```

3. Save the changes.
 4. Restart the CLP server for changes to take effect.
-

Explanation of Elements

- **<loginModuleId>**

Description: Must match the Login Module ID defined in the Admin Console.

Value: RADIUSLogin

- **<loginModuleNeedsChallenge>**

Description: Specifies whether the login module requires an additional challenge step.

- For RADIUS Login using **1FA (OTP)**, this must be set to `false`.
- If set to `true`, errors such as `"request-reject-unknown"` may occur.
- **Note:** 2FA is not supported for reauthentication with this module. Attempting to use 2FA may cause `"Access Challenge -> Authentication required"` errors.

Value: `false`

- **<handler>**

Description: The handler associated with the login module. It must correspond to the handler configured in the Admin Console.

Value: RadiusLogin

- **<loginState>**

Description: Defines the login UI that will be displayed to the user. This value are system-defined and **cannot be modified**.

Value: login.otp

7.9. PseudoE2EEALoginModule

Configure the Login Module

The following steps will create a new login module for PseudoE2EEALoginModule re-authentication.

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select "PseudoE2EEALoginModule" from the **Choose Login Module Implementation** page.
3. Then, click **Next** and provide the **Login Module Id** (e.g. PseudoE2EEALoginModule") and **Login Module Name** (e.g. "PseudoE2EEALoginModule").
4. Configure the rest of the login module attributes, if necessary.
5. Save the changes.

Configure the Re-Authentication Login Module in CLP

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Configure the newly created authentication realm in this CLP configuration file as shown below:

XML

```
<loginModule>
  <loginModuleId></loginModuleId>

  <loginModuleNeedsChallenge>false</loginModuleNeedsChallenge>
  <handler>PseudoE2EEALoginModule</handler>
  <loginState>login.start</loginState>
  <PQC>preferred</PQC><!-- Only applicable to
PseudoE2EEALoginModule, value with required or preferred. If
specified as required, PQC keys will be used. An error will
be returned if there are no PQC keys available. If specified
as preferred, PQC keys will be used. If there are no PQC
keys available, RSA keys will be used instead. RSA keys will
be used if undefined.-->
</loginModule>
```

3. Save the changes.
 4. Restart the CLP server for changes to take effect.
-

Explanation of Elements

-

- **<loginModuleId>**

Description: Unique identifier for the login module. This value must match the Login Module ID defined in the Admin Console.

Value: *(to be defined by configuration)*

- **<loginModuleNeedsChallenge>**

Description: Indicates whether the login module requires an additional challenge step. For this module, it should be set to `false`.

Value: `false`

- **<handler>**

Description: The handler associated with this login module. It must correspond to the handler configured in the Admin Console.

Value: `PseudoE2EEALoginModule`

- **<loginState>**

Description: Defines the login UI state that will be displayed to the user. This value is system-defined and cannot be modified.

Value: `login.start`

- **<PQC>**

Description: Specifies the use of PQC (Post-Quantum Cryptography) keys. Only applicable when using `PseudoE2EEALoginModule`.

- `required` → PQC keys must be used. Authentication fails if PQC keys are not available.
- `preferred` → PQC keys will be used if available; otherwise, RSA keys will be used.
- *(undefined)* → RSA keys will be used by default.

Value: `preferred`

8. AccessMatrix CLP Other Authentication Configurations

Enable Change Password

The configuration below enables the change password feature. To configure:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the `ChgPwdLoginModule` element and set the `enable` child element to "true" to display the change password option on the user login page. The default value is "true".

XML

```
<ChgPwdLoginModule>
  <enable>true</enable>
</ChgPwdLoginModule>
```

3. Save the changes.
4. Restart the CLP server for changes to take effect.

Configure Login Module ID for Change Password

If the change password feature is enabled as described in [Enable Change Password](#) section, then the login module ID of the authentication realm should also be configured. To configure:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the `AuthnRealms` element and find the authentication realm ID that will be used to login to CLP.
3. Find the `loginModuleIdForChangePassword` element under this authentication realm and specify the login module ID to be used for change password. For example, if the authentication realm ID is "40151" and the login module ID for change password is "SystemPasswordBasic", then the configuration should look like this:

XML

```
<AuthnRealm>
  <id>40151</id>
  ...
  <loginModuleIdForChangePassword>SystemPasswordBasic</loginModuleIdForChangePassword>
  ...
</AuthnRealm>
```

-
4. Save the changes.
 5. Restart the CLP server for changes to take effect.

Enable Segment Login

The configuration below enables user login with segment feature. To configure:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the `SegmentLogin` element and set the `enable` child element to "true" to display the segment text field on the user login page. The default value is "false".

XML

```
<SegmentLogin>
  <enable>>false</enable>
</SegmentLogin>
```

3. Save the changes.
4. Restart the CLP server for changes to take effect.

Configure Reauthentication



Note: Currently, CLP only supports reauthentication for USO.

Reauthentication enables your application to confirm a user's identity even if it had been verified previously. For example, the application may ask the user to re-enter their password when attempting to complete a sensitive transaction. This feature is used to prevent cases where a user leaves a device logged in or when a third-party hijacks someone's session.

To configure the reauthentication feature:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the `Reauthenticate` element. This will contain all the login modules that can be used for reauthentication.
3. Configure the login module that will be used for reauthentication as shown below:

XML

```
<loginModule>
  <loginModuleId>SystemPasswordBasic</loginModuleId>
</loginModule>
```

```
<loginModuleNeedsChallenge>false</loginModuleNeedsChallenge>
  <handler>SystemPasswordBasic</handler>
  <loginState>login.start</loginState>
  <!--<PQC>required</PQC>--> <!-- Only applicable to
PseudoE2EEALoginModule, value with required or preferred.
Defaults with RSA keys if undefined.-->
</loginModule>
```

The loginModule child element contains the following configurations:

- **loginModuleId**
 - The value displayed in the selection list when choosing a login module.
- **loginModuleNeedsChallenge**
 - Set this to "true" if the login module challenge is required. The default value is "false".



Important Note: When configuring the loginModule child element for Radius login, this value needs to be specified as "false".

- **handler**

Handler refers to the Login Module Handler class of the login module. The login module handler classes supported for CLP reauthentication is listed below:

 - SystemPasswordBasic
 - LdapLogin
 - RadiusLogin
 - SMSOTPLLogin
 - VascoTokenLogin
 - OATHTokenLogin
 - ChinaOTPTokenLogin
 - CompositeLogin
 - ResponseOnlyTokenLogin
 - BRTokenLogin
 - MultiChoiceLogin
 - SocialNetworkLogin
 - OIDCRelyingPartyLoginModule
-

-
- FIDO2TokenLogin
 - PseudoE2EEALoginModule
 - SafenetPSGLoginModule
 - UtimacoCSE2EELoginModule
- Refer to [Login Module Handler](#) section to locate the login module handler in AccessMatrix.
 - **loginState**

The login state value determines the client UI page to display during the reauthentication process.



Note: The username displayed in the UI during the reauthentication process is non-editable.

Login State Value	Handler Class	Description
login.start	▪ SystemPasswordBasic ▪ TrustExternalLoginHandler ▪ PseudoE2EEALoginModule	The UI displays the user Id and text box for password-based reauthentication.
login.otp	▪ OATHTokenLogin ▪ VascoTokenLogin	The UI displays the user Id and text box for reauthentication using an OTP.
	▪ RadiusLogin	CLP display a page to allow user to enter OTP. Note: Reauthentication only supported for RadiusLogin using 1st factor authentication.
login.firstloginmoduleotp	▪ SMSOTPLogin	The UI displays the user Id and text box for entering the SMS OTP. This is used if

Login State Value	Handler Class	Description
		the login module requires a challenge-response such as SMS OTP.
login.fido2	FIDO2TokenLogin	The UI displays the user Id and "Login With Passkey" button for reauthentication using a FIDO2 passkey.

○ **PQC**

Verifies if the PseudoE2EEALoginModule handler is Post Quantum Cryptography (PQC) enabled.

i Note: This configuration is only applicable to the PseudoE2EEALoginModule handler. To enable this configuration, uncomment and specify with a relevant value. Otherwise, RSA keys will always be returned instead. For more details on configuring PQC for the Pseudo E2EEA Login Module, refer to [Configure Pseudo E2EEA Login Module](#).

PQC Value	Description
required	If specified as required, PQC keys will be used. An error will be returned if there are no PQC keys available.
preferred	if specified as preferred, PQC keys will be used. If there are no PQC keys available, RSA keys will be used instead.

4. Save the changes.
5. Restart the CLP server for changes to take effect.

In this use case, it is the application that passes the login module Id to CLP to decide which login module will be used.

Configure Reauthentication via External OIDC OP

CLP also supports reauthentication using an external OIDC OpenID Provider. This authentication method uses the OAuth 2.0/OIDC Authorization Code flow or the Authorization Code flow with PKCE. Using the PKCE flow is strongly recommended.



Note: Ensure that the **OIDCRP** license feature is added under **UAM FEATURE - Social Network Login** to enable OIDC RP support and to display "OIDC Relying Party Login Module" in the login module implementation list.

The following is a checklist of the relevant steps needed to enable external OP reauthentication:

Step	Reference Guide
Create OIDC Relying Party Login Module	OIDC Relying Party Implementation Guide - Configure OIDC RP Login Module for Reauthentication via External OIDC OP
Verify That Reauthentication Is Enabled in am.commonlogin.config.xml	-
Configure USO Application Reauthentication Attributes	USO Administration Guide - USO Admin Console Administration
Configure OIDC RP Servlet web.xml	OIDC Relying Party Implementation Guide - Configure OIDC RP Configuration File
(Additional) OAuth RP Servlet Reauthentication Endpoints	OIDC Relying Party Implementation Guide - Use OIDC Relying Party Servlet for Reauthentication

Create OIDC Relying Party Login Module

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click the **Create** button and select the "OIDC Relying Party Login Module" implementation from the **Choose Login Module Implementation** page.
3. Enter a suitable value for **Login Module Id** (e.g. "CLPOIDCLogin") and **Login Module Name** (e.g. "CLP - OIDC RP Login").
4. Select "Yes" under the **PKCE Flow is Used** attribute if the Authorization Code flow with PKCE is used. "No" is selected by default.



Note: It is highly recommended that the Authorization Code flow with PKCE is used.

5. Configure the values for the following attributes based on your setup:
 - a. Under the **Attributes** tab:

-
- **OpenID Provider Configuration URL:** "<OIDC_OP_Configuration_URL>"

Example: "https://login.microsoftonline.com/clp.i-sprint/v2.0/.well-known/openid-configuration"

- **Redirection URI:** "<CLP_URL>/rp/reauthn/cb/<loginModuleId>"

Example: "https://clp.i-sprint.com/clp/rp/reauthn/cb/CLPOIDCLogin"

b. Under the **OIDC RP Servlet** tab:

- **Landing Page URL:** "<CLP_URL>/extoidcop"

Example: "https://clp.i-sprint.com/clp/extoidcop"

- **Error Page URL:** "<CLP_URL>/error.jsp"

Example: "https://clp.i-sprint.com/clp/error.jsp"

6. Provide the values for the remaining login module attributes as needed. Refer to *UAM OIDC Relying Party Implementation Guide - Configure Login Module for Reauthentication via External OIDC OP* for details on each of the attributes and the steps to configure them.



Note: To support client assertion JWT, refer to *UAM OIDC Relying Party Implementation Guide - Enable Client Assertion JWT* for details.

7. Save your changes and exit.

Verify That Reauthentication Feature Is Configured in am.commonlogin.config.xml

1. Navigate to <CLP_HOME>/WEB-INF/classes and open **am.commonlogin.config.xml**.
2. Look for the Reauthenticate element.
3. Ensure that the following login module for reauthentication is present:

XML

```
<loginModule>

<loginModuleNeedsChallenge>false</loginModuleNeedsChallenge>
  <handler>OIDCRelyingPartyLoginModule</handler>
  <loginState>externalOIDCReauth.main</loginState>
</loginModule>
```

4. Save the changes, if necessary.
 5. Restart the CLP server for changes to take effect if changes were made.
-

Configure USO Application Reauthentication Attributes

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Enterprised SSO > USO Application** and click on the relevant USO application Id. The **View USO Application** page is displayed.
2. Under the **Reauthentication** header, configure the following attributes:
 - a. **Verify replay** - Set to "Enable".
 - b. **Verify replay method** - Select the OIDC RP login module Id of the login module configured in the steps above (e.g. "CLPOIDCLogin").
 - c. **Verify replay before launch** - Set to "Yes".
3. Save the changes.



Note: Refer to [USO Administration Guide - USO Admin Console Administration](#) for details on each of the attributes.

Configure OIDC RP Servlet web.xml

1. Navigate to `<ACMATRIX_ROOT>\tomcat\webapps\oauth-rp\WEB-INF` and open the **web.xml** file.
2. Search for the OAuthRPReauthnJwksServlet servlet mapping and update the url-pattern value by adding the login module Id of the OIDC RP login module configured in the steps above (e.g. "CLPOIDCLogin"):
 - a. **OAuthRPReauthnJwksServlet** - Configure the url-pattern value as `/reauthn/<loginModuleId>/.well-known/jwks.json`
For example:

XML

```
<servlet-mapping>
  <servlet-
name>OAuthRPReauthnJwksServlet</servlet-name>
  <!--Servlet path "/reauthn/<moduleId>" followed
by "/.well-known/jwks.json". Configure the moduleId
where the login module is an OIDC Relying Party Login
Module handler -->
  <url-pattern>/reauthn/CLPOIDCLogin/.well-
known/jwks.json</url-pattern>
</servlet-mapping>
```

3. Save the changes.
-

i **Note:** Refer to the *OIDC Relying Party Implementation Guide - Configure OIDC RP Configuration File* for more information on the OIDC RP Servlet **web.xml** file.

(Additional) OAuth RP Servlet Reauthentication Endpoints

The application must pass the OIDC Relying Party (RP) login module Id to CLP for that login module to be used for external reauthentication.

If necessary, you may refer to [UAM OIDC Relying Party Implementation Guide - Use OIDC Relying Party Servlet for Reauthentication](#) for the full details on the OIDC RP Servlet endpoints and request parameters involved in the reauthentication process.

Configure Self-Service Password Reset

The Self-Service Password Reset (SSPR) feature allows users, who have either forgotten their password or whose account is locked, to change or reset their password without administrator involvement. This feature can be used for a variety of use cases:

Channel	Description
Internal	SSPR is triggered from CLP's login page.
External	SSPR is triggered from an external application (such as the Admin Console) and the user is redirected to the SSPR page to perform the password reset. SSPR can be used as a standalone feature, such that the external application is not required to integrate with CLP or its supported protocols. The URL <code>http://<IP address>:<port>/clp/sspr</code> is used (e.g. <code>http://localhost:8080/clp/ssr</code>) to directly access the SSPR page. Note: This feature is only supported from AccessMatrix 5.7.5-GA onwards.
Verification Method	Description
Self-service Password Reset via OTP Authentication	This method uses the Email/SMS delivery channel to send an OTP via email or SMS. The user must use the OTP for verification before the password reset can be completed.
Self-service Password Reset via Security	This method uses a security question + OTP Authentication as a verification method before the password reset can be completed.

Channel	Description
Question + OTP Authentication	
Self-service Password Reset via Security Question Only	This method uses a security question as a verification method before the password reset can be completed.
Self-service Password Reset via Token OTP Authentication	This method uses token OTP authentication as a verification method to trigger the password reset.

The steps to configure the SSPR feature are detailed in the next section, while the various verification methods are discussed in the section after that.

Enable SSPR

To configure this feature:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Search for the `SelfServicePasswordReset` element.
3. Set the `enable` element to "true" to enable the self-service password reset feature. The default value is "false". For example:

XML
<pre> <SelfServicePasswordReset> <enable>true</enable> ... </SelfServicePasswordReset> </pre>

4. Set the `selections` element to "true" to allow users to select between multiple verification methods for resetting their password. The default value is "false". For example:

XML
<pre> <SelfServicePasswordReset> <enable>true</enable> <selections>true</selections> ... </SelfServicePasswordReset> </pre>

5. Uncomment the relevant verification methods under the `ResetPassword` element.



Note: Ensure that all `authnRealmId` elements specified here have been configured. For more details on how to configure each verification method, refer to [Configure Verification Methods](#).

- a. If the `selections` element is set to "true", multiple verification methods can be configured. For example:

XML

```
<ResetPassword>
  <OTPAuthentication>
    <otpChannel>
      <idLabel>Lbl_EmailOtp</idLabel>
      <authnRealmId>EmlRealm</authnRealmId>

    <eventNotificationPolicyId></eventNotificationPolicyId>
    </otpChannel>
    <otpChannel>
      <idLabel>Lbl_SmsOtp</idLabel>
      <authnRealmId>SMSRealm</authnRealmId>

    <eventNotificationPolicyId>smsNotificationPolicy</eventNotificationPolicyId>
    </otpChannel>
  </OTPAuthentication>
  <!--<SecurityQuestionAndOTP>-->
    <!--<enableOTP>true</enableOTP>-->
    <!--<authnRealmId>oneS</authnRealmId>-->
  <!--</SecurityQuestionAndOTP>-->
  <TokenOTP>
    <authnRealmId>DefaultOATHToken</authnRealmId>
  </TokenOTP>
</ResetPassword>
```

- b. If the `selections` element is set to "false", ensure that only one verification method has been configured. For example:

XML

```
<ResetPassword>
  <OTPAuthentication>
    <otpChannel>
      <idLabel>Lbl_EmailOtp</idLabel>
      <authnRealmId>EmlRealm</authnRealmId>

    <eventNotificationPolicyId></eventNotificationPolicyId>
    </otpChannel>
    <otpChannel>
      <idLabel>Lbl_SmsOtp</idLabel>
      <authnRealmId>SMSRealm</authnRealmId>

    <eventNotificationPolicyId>smsNotificationPolicy</eventNotificationPolicyId>
  </OTPAuthentication>
</ResetPassword>
```

```

        </otpChannel>
    </OTPAuthentication>
    <!--<SecurityQuestionAndOTP>-->
        <!--<enableOTP>true</enableOTP>-->
        <!--<authnRealmId>oneS</authnRealmId>-->
    <!--</SecurityQuestionAndOTP>-->
    <!--<TokenOTP>-->
        <!--<authnRealmId>DefaultOATHToken</authnRealmId>-->
    <!--</TokenOTP>-->
</ResetPassword>

```

6. Search for the AuthnRealm element of the authentication realm that is used for CLP login, e.g. 40151.



Note: CLP will redirect the user to the password reset page if the specific authentication realm has been configured for password reset.

7. Set the values for the userStoreId and loginModuleId under the resetPassword child element. For example:

XML

```

<AuthnRealm>
    <id>40151</id>
    ...
    <resetPassword>
        <userStoreId>DefaultStore</userStoreId>

        <loginModuleId>SystemPasswordBasic</loginModuleId>
    </resetPassword>
    ...
</AuthnRealm>

```



Notes:

- The example above sets "DefaultStore" as the user store ID and "SystemPasswordBasic" as the login module ID.
- userStoreId refers to the ID of the user store to which the user belongs, e.g. DefaultStore, ActiveDirectory.
- loginModuleId refers to the ID of the login module that requires password reset. The password reset feature is only applicable for password-based login modules.
- For self-service password reset to work, it is required to set the **Allow resetting password without force password change** attribute of the Basic Login Policy for the login module to "Enable in API only" or "Enable in API and Admin Console"

8. (Optional) To require users to change their password when they first log in after a password reset, add the line <forcePasswordChange>true</forcePasswordChange> under resetPassword. For example:

XML

```
<AuthnRealm>
  <id>40151</id>
  ...
  <resetPassword>
    <userStoreId>DefaultStore</userStoreId>

    <loginModuleId>SystemPasswordBasic</loginModuleId>
    <forcePasswordChange>true</forcePasswordChange>
  </resetPassword>
  ...
</AuthnRealm>
```



Note: If the `<forcePasswordChange>true</forcePasswordChange>` line is already present, ensure it is uncommented and the value is "true" for the setting to take effect.

9. Save the changes.

Configure Verification Methods

There are four types of verification methods that are supported for the self-service password reset feature:

- [Self-service Password Reset via OTP Authentication](#)
- [Self-service Password Reset via Security Question + OTP Authentication](#)
- [Self-service Password Reset via Security Question Only](#)
- [Self-service Password Reset via Token OTP Authentication](#)



Note: The default verification method is OTP Authentication.

Self-service Password Reset via OTP Authentication

This method uses Email/SMS delivery channel to send OTP via Email or SMS. The OTP will be used for verification before a password reset will be done.



Note: Before configuring this verification method, ensure that SMS OTP has been implemented correctly. For more details, refer to [UAS SMS OTP Implementation Guide - Implementation Checklist](#).

To configure:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
-

-
2. Look for the `ResetPassword` element. Under this element is the `OTPAuthentication` element containing the OTP Channel configurations. You may configure both or either Email or SMS OTP channels.
 3. To support the Email OTP channel, modify the XML file as shown below:

XML

```
<otpChannel>
  <idLabel>Lbl_EmailOtp</idLabel>
  <authnRealmId>EmlRealm</authnRealmId>

  <eventNotificationPolicyId></eventNotificationPolicyId>
</otpChannel>
```

4. To support the SMS OTP channel, modify the XML file as shown below:

XML

```
<otpChannel>
  <idLabel>Lbl_SmsOtp</idLabel>
  <authnRealmId>SMSRealm</authnRealmId>

  <eventNotificationPolicyId>smsNotificationPolicy</eventNotificationPolicyId>
</otpChannel>
```



Notes:

- `idLabel` refers to the label ID of the Email/SMS OTP channel. This value can be configured as `Lbl_xxx` and it is configurable in:
 - `<CLP_HOME>/assets/i18n/message_en_US.json`
 - `<CLP_HOME>/assets/i18n/message_zh_CN.json`The label value is displayed in the **Select OTP Channel** drop-down list of the **Self Service Password Reset** dialog box.
- `authnRealmId` refers to the ID of the authentication realm that is configured with a login module that implements "SMSOTPLLogin" handler only. This authentication realm must be assigned to the CLP user. Refer to [Login Module Handler](#) section to locate the login module handler in AccessMatrix.
- `eventNotificationPolicyId` refers to the ID of the Event Notification Policy, an event-based trigger for sending email/SMS OTP. This setting is optional. By default, CLP will use the Event Notification Policy configured in the login module. However, if this setting is configured, then it will overwrite the Event Notification Policy configured in the login module.

5. Save the changes.
-

-
- Restart the CLP server for changes to take effect.

Self-service Password Reset via Security Question + OTP Authentication

This method uses Security Question + OTP Authentication for verification before a password reset will be done.

i **Note:** Before configuring this verification method, ensure that SMS OTP has been implemented correctly. For more details, refer to [UAS SMS OTP Implementation Guide - Implementation Checklist](#).

To configure:

- Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
- Look for the `ResetPassword` element. Under this element is the `SecurityQuestionAndOTP` element containing the Security Question + OTP configuration. Comment out the `OTPAuthentication` configuration and uncomment the `SecurityQuestionAndOTP` configuration.
- To enable the Security Question + OTP Authentication feature, modify the XML file as shown below:

XML

```
<SecurityQuestionAndOTP>
  <enableOTP>true</enableOTP>
  <authnRealmId>oneS</authnRealmId>
</SecurityQuestionAndOTP>
```

i **Note:** `authnRealmId` refers to the ID of the authentication realm assigned to the user which contains 2 login modules: the first login module is for Security Question which is implemented using the "QAndAKBALoginInBulkAnswers" handler, and the second login module is for OTP Authentication which is implemented using the "SMSOTPLogin" handler.
Refer to [Set Up Answers for Knowledge-based Authentication](#) section for Q&A KBA login configuration.

- Save the changes.
- Restart the CLP server for changes to take effect.

Self-service Password Reset via Security Question Only

This method uses Security Question for verification before a password reset will be done.

To configure:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the `ResetPassword` element. Under this element is the `SecurityQuestionAndOTP` element containing the Security Question + OTP configuration. Comment out the `OTPAuthentication` configuration and uncomment the `SecurityQuestionAndOTP` configuration.
3. Modify the XML file as shown below, ensuring that the `enableOTP` attribute is set to "false":

XML

```
<SecurityQuestionAndOTP>
  <enableOTP>false</enableOTP>
  <authnRealmId>oneS</authnRealmId>
</SecurityQuestionAndOTP>
```



Note:

- `authnRealmId` refers to the ID of the authentication realm that is configured with a login module that implements the "QAndAKBALoginInBulkAnswers" handler only. Refer to [Set Up Answers for Knowledge-based Authentication](#) section for Q&A KBA login configuration.

4. Save the changes.
5. Restart the CLP server for changes to take effect.

Self-service Password Reset via Token OTP Authentication

This method uses token OTP authentication for verification to trigger a password reset.

Before configuring this verification method:

- If an OATH Token is used, ensure that the user has activated their authenticator token using the UAS Token Activation Portal.



Notes:

- Refer to the relevant section (e.g. *Google Authenticator Token Deployment* section) in the *UAS Token Activation Portal Deployment Guide* for more information.
- Examples of OATH token implementations include:
 - Google Authenticator Token
 - Microsoft Authenticator Token

-
- YESsafe Token

Once the above steps have been completed, perform the following steps:

1. Navigate to `<CLP_HOME>/WEB-INF/classes` and open **am.commonlogin.config.xml**.
2. Look for the `ResetPassword` element. Under this element is the `TokenOTP` element containing the Token OTP configuration. Comment out the `OTPAuthentication` configuration and uncomment the `TokenOTP` configuration:

XML

```
<TokenOTP>
  <authnRealmId>DefaultOATHToken</authnRealmId>
</TokenOTP>
```



Note:

- `authnRealmId` refers to the ID of the authentication realm that is configured with a login module that implements the respective token handler only. This authentication realm must be assigned to the CLP user. Refer to the [Login Module Handler](#) section for details on how to locate the login module handler in AccessMatrix.

3. Save the changes.
4. Restart the CLP server for changes to take effect.

Display External Web Links on Login Page

It is possible to display external links on the login page. They can be used for a variety of functions, such as redirecting users to an FAQ page. This can be configured via the `externalLinks` code block under the `loginModule` element.



Note: This feature is only available for authentication methods that have the value "login.start" under the `loginState` field.

To do so, perform the following steps:

1. Add the following `externalLinks` code block under the `loginModule` element:

XML

```

<AuthnRealm>
...
  <loginModule>
    <handler>SystemPasswordBasic</handler>
    <loginState>login.start</loginState>
    <externalLinks>
      <externalLink>
        <order>1</order>
        <url>http://google.com</url>
        <label>Lbl_ExLink_1</label>
      </externalLink>
      <externalLink>
        <order>2</order>
        <url>https://microsoft.com</url>
        <label>Lbl_ExLink_2</label>
      </externalLink>
    </externalLinks>
  </loginModule>
</AuthnRealm>

```



Notes:

- If the code block is already present but is commented out, uncomment it.
- You can omit the second externalLink element if only one external link is required.

2. Specify the URL of the desired external web page in the url field under the externalLink element (e.g. "https://google.com").
3. Note down the corresponding label value of the external link (e.g. "Lbl_ExLink_1").
4. Navigate to <CLP_HOME>\assets\i18n\ and open the json file for the language you wish to make the update for (e.g. **message_en_US.json**).
5. Search for the label you noted down (e.g. "Lbl_ExLink_1") and change the value to the link text that will be presented to the user (e.g. "Forgot User ID or PIN"). If the label does not exist, add it into the file.

For example:

JSON

```
"Lbl_ExLink_1": "Forgot User ID or PIN"
```



Note: If you support multiple languages for your login page, you must repeat steps 4 and 5 with the other language json files.

-
6. Repeat steps 2-5 for any other external links you wish to configure. You can add up to 5 external links.

Set Up Answers for Knowledge-based Authentication (KBA)

The Q&A KBA-based login method can be configured using the **Q&A KBA Login (one step)** login module. This login module allows the user to configure their own Q&As (up to 10 questions). Then, users can log in to answer the defined questions in one submission. This module will then verify multiple answers all at once (one step/in one call). The administrator can reset the users' questions and answers to cater to the users' requirements.

To set up questions for KBA:

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane.
2. Click **Create** and select the "Q&A KBA Login (one step)" implementation from the **Choose Login Module Implementation** page.
3. Then, click Next and provide the **Login Module Id** (e.g. "Q&AOneS") and **Login Module Name** (e.g. "Q&A One Step").
4. Set the value for the **Minimum No. of Correct Responses** attribute.
5. Set up the questions (maximum of 10 questions).
6. Save the changes.
7. Assign this login module to the authentication realm (e.g. oneS) configured in the previous section ([Use Case 2: Self-service Password Reset via Security Question + OTP Authentication](#)).
8. Restart the CLP server for changes to take effect.

To set up answers for KBA:

1. Ensure that the authentication realm with Q&A KBA Login (one step) implementation is assigned to the CLP user.
 2. Launch CLP and log in.
 3. If the login is successful, the user will be prompted to set up answers for the security questions.
 4. Click **Submit** once done.
-



Note: Clicking the **Forgot Password?** link prior to CLP login will display the **Self Service Password Reset** dialog box. The user will be prompted to enter his User ID and the answers to the security questions. The user can then reset his password once verified.
